NORTHWESTERN UNIVERSITY

Contract Checking as Effects

A DISSERTATION

SUBMITTED TO THE GRADUATE SCHOOL

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

Master of Science

Field of Computer Science

By

Anh Khoa Nguyen

EVANSTON, ILLINOIS

December 2026

Copyright by Anh Khoa Nguyen 2026

All Rights Reserved

ABSTRACT

Contract Checking as Effects

Anh Khoa Nguyen

Contracts checking can be turned into effects and handled with handlers.

Preface

This is the preface (optional).

Dedication

This is the dedication (optional).

Table of Contents

ABSTRACT **3**
Preface **4**
Dedication **5**
Table of Contents **6**
List of Tables **9**
List of Figures **10**
1 Introduction **11**
1.1 Contributions 12
1.2 Base Language 13
2 Contracts Language **20**
2.1 Example 20
2.2 Language Model 21
3 Effects Language **28**
3.1 Example 28
3.2 Language Model 29
4 Compiler **37**
4.1 Formal Model 39
4.2 Contracts Hijacking (and Option Contracts) 42
4.3 States (and Trace Contracts) 43
4.4 Connection to Effectful Contracts 45
5 Implementation **47**
5.1 Redex models 47
5.2 OCaml implementation 49
5.2.1 OCaml Attributes and Rewritters 50
5.2.2 OCaml Contracts as Attributes 50

5.2.3 States encoding contracts . . . 52
5.2.4 Boxes and Vectors contracts . . . 53
5.2.5 Mismatch with the formal models . . . 53
6 Conclusion . . . 55
6.1 Contract checking as Effects is Possible . . . 55
6.2 Shallow or Deep handlers . . . 56
6.3 Effectful Contracts . . . 57
6.4 Practicality . . . 57
6.5 Comparison to Racket . . . 58
Bibliography . . . 59
A Base Safety Theorem . . . 62
A.1 Type Safety . . . 62
B Contracts Safety Theorems . . . 65
C Contracts Erasure Theorem . . . 70
C.1 Relation . . . 70
C.2 Main Theorem Proof . . . 75
D Effects Safety Theorem . . . 80
D.1 Evaluation Context Typing Lemmas . . . 80
D.2 Type Safety Theorem . . . 81
E Contracts to Effects Compiler Typesoundness . . . 87
E.1 Overview . . . 87
E.2 Compiler Type Preservation . . . 87
E.3 Wrapped Compiled Program Typing (Deep Handler) . . . 90
E.4 Wrapped Compiled Program Typing (Shallow Handler) . . . 91
F Contracts to Effects Compiler Correctness (Deep Handler) . . . 93
F.1 Overview . . . 93

F.2 Relations 93

F.3 Compiler Correctness 103

G Contracts to Effects Compiler Correctness (Shallow Handler) 111

G.1 Overview 111

G.2 Relation 111

G.3 Compiler Correctness 115

H Contracts to Effects Compiler Correctness (Local Handler) 125

H.1 Relations 125

H.2 Correctness Theorem 129

List of Tables

Table 1 Attributes for Contracts 51

List of Figures

Figure 1 Base Language 14
Figure 2 Type Rules for Base 16
Figure 3 Reduction Rules for Base 18
Figure 4 Contracts in action 20
Figure 5 Contracts language 22
Figure 6 Type Rules for Contracts 24
Figure 7 Reduction Rules for Contracts 26
Figure 8 Handler in action 28
Figure 9 Handler with state 29
Figure 10 Type Rules for Effects 33
Figure 11 Naive handler stuck 38
Figure 12 Handler placement analysis case 39
Figure 13 Compiler Rules 41
Figure 14 Shallow check handler with contract-controlled state 43
Figure 15 Shallow check handler with contract-controlled multiple states . 44
Figure 16 Extend Contracts with contract states 45
Figure 17 Structure of the contract erasure proof. Dashed vertical lines denote ER. 75
Figure 18 Structure of the compiler correctness proof. Dashed lines rel states by EQ. 104
Figure 19 Structure of the compiler correctness proof. Dashed lines rel states by EQ. 116

CHAPTER 1

Introduction

Software contracts, or *behavioral contracts*, allow programmers to attach assertions to values, and functions. These assertions are runtime guarantees, on violation, an error is thrown, blaming the violating party. Contracts are most useful when placed on a function, enforcing the function's input and output values. While first-order functional contracts are useful, they are too simple. Higher-order functional contracts allows programmers to enforce contracts for higher-order functions. Dependent contracts are functional contracts allowing usage of the function argument during post-condition contract checking.

Recently, algebraic effect handlers has gained a large attraction, and is adopted in OCaml since version 5.0. Effect handlers open a new way to identify effects and handling them. Handlers are defined to handle one or many effects. A handler for an effect can stop the program or continue with some result. Handlers do not have state by default, but it can be implemented by applying the new states to the continuation closure, and let the underlying computation receives a state argument.

Combining the contract system and effect handlers is a new approach. Effectful Contract [13] demonstrates how both systems can be combined, providing contracts for effectful code. Contracts in their system remain a language feature. In this dissertation, instead of combining the effects and contracts, we ask ourselves whether contract checking itself can become an effect, and reuse the capabilities of handlers.

> *While encoding contracts as effects does not increase the expressiveness of effects, delicate handler constructions make it possible to unify contracts into a single con-*

text. This unification enables contract states and communication between contracts without relying on boxes.

1.1 Contributions

This dissertation provides concrete evidence to the thesis claim. We model the two languages Contracts representing software contracts, and `Effects` representing effect handlers. Then a formal model for a compiler between Contracts and `Effects` is presented. This compiler is proven correct, and effect safe. Moreover, we provide an implementation of such compiler by providing a preprocessor to convert OCaml code annotated with contracts into run-able OCaml code based off this compiler model. Lastly, we provide a Redex PLT model to test our theorems.

Structure …

1.2 Base Language

We define Base language and its syntax in Figure 1. The language extends the PCF language with tuples. It has no mutable cells. This will be the base language for both Contracts and `Effects`.

Base

$$
\begin{array}{rcl}
\tau & ::= & o \\
& | & \tau \to \tau \\
& | & \langle \tau, \tau \rangle \\
& | & \mathsf{unit} \\
\\
o & ::= & \mathrm{num} \\
& | & \mathrm{bool} \\
\\
e & ::= & n \mid b \mid \mathsf{unit} \mid \langle e, e \rangle \mid x \\
& | & \lambda x : \tau . e \mid \mu x : \tau . e \\
& | & e + e \mid e - e \mid e \wedge e \mid e \vee e \\
& | & e\, e \mid \mathrm{if}\ e\ \mathrm{then}\ e\ \mathrm{else}\ e \\
& | & \mathsf{zero?}(e) \\
& | & \mathsf{inj.l}(e) \mid \mathsf{inj.r}(e)
\end{array}
$$

$$
\begin{array}{rcl}
n & ::= & 0 \mid -1 \mid 1 \mid \ldots \\
\\
b & ::= & \mathsf{true} \mid \mathsf{false} \\
\\
v & ::= & n \mid b \mid \mathsf{unit} \mid \langle v, v \rangle \mid x \mid \lambda x : \tau . e \\
\\
E & ::= & \square \\
& | & E + e \mid v + E \mid E - e \mid v - E \\
& | & E \wedge e \mid v \wedge E \mid E \vee e \mid v \vee E \\
& | & E\, e \mid v\, E \mid \mathrm{if}\ E\ \mathrm{then}\ e\ \mathrm{else}\ e \mid \mathsf{zero?}(E) \\
& | & \langle E, e \rangle \mid \langle v, E \rangle, \mathsf{inj.l}(E) \mid \mathsf{inj.r}(E)
\end{array}
$$

Figure 1: Base Language

$$\frac{}{\Gamma \vdash n : \mathsf{num}}\ \text{T-Num} \qquad \frac{}{\Gamma \vdash b : \mathsf{bool}}\ \text{T-Bool} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash \langle e_1, e_2 \rangle : \langle \tau_1, \tau_2 \rangle}\ \text{T-Tuple}$$

$$\frac{}{\Gamma \vdash \mathsf{unit} : \mathsf{unit}}\ \text{T-Unit} \qquad \frac{o : \tau_a \times \tau_b \to \tau_o \quad \Gamma \vdash e_1 : \tau_a \quad \Gamma \vdash e_2 : \tau_b}{\Gamma \vdash e_1\ o\ e_2 : \tau_o}\ \text{T-Op}$$

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ \text{T-Var} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x : \tau_1 . e : \tau_1 \to \tau_2}\ \text{T-Lambda} \qquad \frac{\Gamma, x : \tau \vdash e : \tau}{\Gamma \vdash \mu x : \tau . e : \tau}\ \text{T-Fix}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}\ \text{T-App} \qquad \frac{\Gamma \vdash e_1 : \mathsf{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau}\ \text{T-If}$$

$$\frac{\Gamma \vdash e : \mathsf{num}}{\Gamma \vdash \mathsf{zero?}(e) : \mathsf{bool}}\ \text{T-Zero}$$

$$\frac{\Gamma \vdash e : \langle \tau_1, \tau_2 \rangle}{\Gamma \vdash \mathsf{inj.l}(e) : \tau_1}\ \text{T-Inj-Left} \qquad \frac{\Gamma \vdash e : \langle \tau_1, \tau_2 \rangle}{\Gamma \vdash \mathsf{inj.r}(e) : \tau_2}\ \text{T-Inj-Right}$$

Figure 2: Type Rules for Base

$\lambda x.e\ v$	$\dashrightarrow$	$e[x := v]$	S-App
$\mu x.e$	$\dashrightarrow$	$e[x := \mu x.e]$	S-Fix
if true then e_1 else e_2	$\dashrightarrow$	e_1	S-If-True
if false then e_1 else e_2	$\dashrightarrow$	e_2	S-If-False
$\mathsf{inj.l}(\langle v_1, v_2 \rangle)$	$\dashrightarrow$	v_1	S-Inj-Left
$\mathsf{inj.r}(\langle v_1, v_2 \rangle)$	$\dashrightarrow$	v_2	S-Inj-Right
$v_1 o v_2$	$\dashrightarrow$	$\delta(o, v_1, v_2)$	S-Op
$\mathsf{zero?}(n)$	$\dashrightarrow$	b	S-Zero

$$\frac{e_1 \dashrightarrow e_2}{E[e_1] \longrightarrow E[e_2]}\ \text{Step}$$

Figure 3: Reduction Rules for Base

CHAPTER 2

Contracts Language

2.1 Example

Contracts provide a run-time validation of programs, similar to how assertions work. Contracts extend basic assertions to support functions, especially higher-order ones, correctness. Past research has move from pre/post condition placing on functions, into obligations [6], or monitors. These constructions support blame tracking, enabling the developer to know *who*, which function, is responsible for a violation.

Figure 4 illustrates a function protected with a monitor. This function should return a value that is even. When applied with 10, it is checked by the function's *domain contract*. Since the function does not impose any properties for its argument, the contract passes. The body of the function is executed normally, but its result must pass the function's *range contract*. Because 11 is not an even number, the program terminates with a blame.

$$
\begin{array}{ll}
 & \mathsf{mon}_j^{k,l}(\mathsf{flat}(\lambda x.\ \mathsf{true}) \to \mathsf{flat}(\mathsf{iseven}), \lambda x.x+1)\ 10 \\
\to & \mathsf{guard}_j^{k,l}(\mathsf{flat}(\lambda x.\ \mathsf{true}) \to \mathsf{flat}(\mathsf{iseven}), \lambda x.x+1)\ 10 \\
\to & \mathsf{mon}_j^{k,l}\Big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{mon}_j^{l,k}(\mathsf{flat}(\lambda x.\ \mathsf{true}), 10)\Big) \\
\to & \mathsf{mon}_j^{k,l}\Big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{guard}_j^{l,k}(\lambda x.\ \mathsf{true}, 10)\Big) \\
\to & \mathsf{mon}_j^{k,l}\Big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{check}_j^{l}((\lambda x.\ \mathsf{true})\ 10, 10)\Big) \\
\to & \mathsf{mon}_j^{k,l}\Big(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ \mathsf{check}_j^{l}(\mathsf{true}, 10)\Big) \\
\to & \mathsf{mon}_j^{k,l}(\mathsf{flat}(\mathsf{iseven}), (\lambda x.x+1)\ 10) \\
\to^* & \mathsf{mon}_j^{k,l}(\mathsf{flat}(\mathsf{iseven}), 11) \\
\to^* & \mathsf{blame}_j^{k}
\end{array}
$$

Figure 4: Contracts in action

2.2 Language Model

We define Contracts, in Figure 5, by extending the Base language with contracts. Contracts protect an expression, ensuring that the evaluated value conform to the contracts defined. Contracts are defined for all possible types of value. Base values, including booleans and numbers, are trivial to enfore a contract on them. While functions need to check for both its argument and return result. Not only that, functions are first-class citizen in the language, and we can form higher-order functions. A contract for function may want to "see" the argument when checking the result, such is the idea of dependent contracts. Both contracts higher-order functions, and dependent contracts semantics are discussed in [6].

The language supports contracts for tuples. Their contracts will be applied individually during execution. Mutable cells contracts follow the work of [2], where a special contract type ref/c(κ) is used to protect the cell's value.

Contracts extends Base

$$
\begin{array}{lcl}
\tau & ::= & \ldots \\
 & | & \mathsf{con}(\tau) \\
\\
e & ::= & \ldots \\
 & | & \mathsf{mon}_j^{k,l}(\kappa, e) \\
 & | & \mathsf{check}_j^k(e, v) \\
 & | & \mathsf{blame}_p^l \\
\\
\kappa & ::= & \mathsf{flat}(e) \\
 & | & \kappa \to \kappa \mid \kappa \to_i \lambda x : \tau.\kappa \\
\\
v & ::= & \ldots \\
 & | & \mathsf{guard}_j^{k,l}(\kappa, v) \\
\\
E & ::= & \ldots \\
 & | & \mathsf{mon}_j^{k,l}(\kappa, E) \\
 & | & \mathsf{check}_j^k(E, v)
\end{array}
$$

Figure 5: Contracts language

$$\frac{\Gamma \vdash e : \tau \to \mathsf{bool}}{\Gamma \vdash \mathsf{flat}(e) : \mathsf{con}(\tau)}\ \text{Type-C-Flat} \qquad \frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \quad \Gamma \vdash \kappa_2 : \mathsf{con}(\tau_2)}{\Gamma \vdash \kappa_1 \to \kappa_2 : \mathsf{con}(\tau_1 \to \tau_2)}\ \text{Type-C-Fun}$$

$$\frac{\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1) \quad \Gamma \vdash \lambda x : \tau_1 . \kappa_2 : \tau_1 \to \mathsf{con}(\tau_2)}{\Gamma \vdash \kappa_1 \to_i \lambda x . \kappa_2 : \mathsf{con}(\tau_1 \to \tau_2)}\ \text{Type-C-DepFun}$$

$$\frac{\Gamma \vdash \kappa : \mathsf{con}(\tau) \quad \Gamma \vdash e : \tau}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa, e) : \tau}\ \text{Type-C-Mon} \qquad \frac{}{\Gamma \vdash \mathsf{blame}_l^k : \tau}\ \text{Type-C-Blame}$$

$$\frac{\Gamma \vdash e : \mathsf{bool} \quad \Gamma \vdash v : \tau}{\Gamma \vdash \mathsf{check}_l^k(e, v) : \tau}\ \text{Type-C-Check} \qquad \frac{\Gamma \vdash v : \tau \quad \Gamma \vdash \kappa : \mathsf{con}(\tau)}{\Gamma \vdash \mathsf{guard}_j^{k,l}(\kappa, v) : \tau}\ \text{Type-C-Guard}$$

Figure 6: Type Rules for Contracts

When a contract violation occurs, a blame is thrown with the party, caller or callee. The semantics for correct blaming has been studied under [2] and [4]. Following previous works, We use *indy* semantics for dependent contracts. We use $\mathsf{guard}_j^{k,l}(\kappa, v)$ as the run-time proxy to value v procted under a contract κ. Guard is a value, the semantics adapt to usage of guards, and expand further, sometimes back to monitor to evaluate inner expressions. A guard on a basic value will trigger a $\mathsf{check}_j^k(e\ v, v)$ and returns the value v if it passes the predicate e, or returns a blame_j^k to party k with contract location j. The evaluation rules for Contracts is presented in Figure 7.

$$\mathsf{check}^{k}_{j}(\mathrm{tt}, v) \quad \dashrightarrow \quad v \qquad \text{Check-True}$$

$$\mathsf{check}^{k}_{j}(\mathrm{ff}, v) \quad \dashrightarrow \quad \mathsf{blame}^{k}_{j} \qquad \text{Check-False}$$

$$\mathsf{mon}^{k,l}_{j}(\mathsf{flat}(e), v) \quad \dashrightarrow \quad \mathsf{check}^{k}_{j}(e\ v, v) \qquad \text{Mon-Flat}$$

$$\mathsf{mon}^{k,l}_{j}(\kappa, v) \quad \dashrightarrow \quad \mathsf{guard}^{k,l}_{j}(\kappa, v) \qquad \text{Mon-Guard}$$

$$\text{where } \kappa \neq \mathsf{flat}(e)$$

$$\mathsf{guard}^{k,l}_{j}(\kappa_1 \to \kappa_2, v_1)\ v_2 \quad \dashrightarrow \quad \mathsf{mon}^{k,l}_{j}\Big(\kappa_2, v_1\ \mathsf{mon}^{l,k}_{j}(\kappa_1, v_2)\Big) \qquad \text{Guard-Func}$$

$$\mathsf{guard}^{k,l}_{j}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)\ v_2 \quad \dashrightarrow \quad \mathsf{mon}^{k,l}_{j}\Big(\kappa_3, v_1\ \mathsf{mon}^{l,k}_{j}(\kappa_1, v_2)\Big) \qquad \text{Guard-Dep}$$

$$\text{where } \kappa_3 = \kappa_2\Big[x := \mathsf{mon}^{l,j}_{j}(\kappa_1, v_2)\Big]$$

$$\frac{E \neq \square}{E\big[\mathsf{blame}^{k}_{j}\big] \to \mathsf{blame}^{k}_{j}}\ \text{Step-C-Blame}$$

$$\frac{e_1 \dashrightarrow e_2}{E[e_1] \to E[e_2]}\ \text{Step-C} \qquad \frac{e_1 \dashrightarrow e_2}{E[e_1] \to E[e_2]}\ \text{Step-C-Base}$$

Figure 7: Reduction Rules for Contracts

CHAPTER 3

Effects Language

3.1 Example

In Figure 8, we present a simple example: a program wants to compute $(\mathscr{D}\ 2) + 1$ is registered with a handler h for effect $\mathscr{D}$, basically *duplicating* its input. When an effect is perform, the main program pauses the execution, and push everything later into a continuation $k = \lambda y.\texttt{handle}_d(h, (y + 1))$, at (i). Here, the continuation registers the handler h again, in case the rest of the program uses $\mathscr{D}$. When the program is a value, the handler removes itself (ii).

$$
\begin{array}{lll}
h & = & \{\mathscr{D} \to \lambda v.\lambda k.k\ (v + v)\} \\
 & \texttt{handle}_d(h, ((\texttt{perform}\ \mathscr{D}\ \mathsf{num}\ 2) + 1)) & \\
\Longrightarrow & \texttt{handle}_d(h, \texttt{perform}\ \mathscr{D}\ \mathsf{num}\ 2) & \\
\Longrightarrow & (\lambda v.\lambda k.k\ (v + v))\ 2\ (\lambda y.\texttt{handle}_d(h, (y + 1))) & \text{(i)} \\
\Longrightarrow^* & (\lambda y.\texttt{handle}_d(h, (y + 1)))\ (2 + 2) & \\
\Longrightarrow^* & \texttt{handle}_d(h, 5) & \text{(ii)} \\
\Longrightarrow & 5 &
\end{array}
$$

Figure 8: Handler in action

Handlers can encode states. This is a nice feature to provide a common state across the code. We illustrate the example in Figure 9, where a simple state can be managed by two effects *get* $\mathscr{G}$, and *set* $\mathscr{S}$. As can be seen, the continuation now returns a function expecting a state. During handler execution, this state is applied automatically if the handler is installed with initial state. The continuation resumes the program with the new state, made possible because handler is installed with new state.

Without careful setup, the program might enter the stuck state because the handler removes itself when the execution completes, making the result applies with the state. For example, this program $\texttt{handle}_d(h, v)\ s \Longrightarrow^* v\ s$ stucks if v is not callable. There are many ways to resolve this problem. The first approach requires the program awareness of the state. The programmer explicitly returns a function taking the result and the state but only returns the result.

$$\texttt{handle}_d(h, ((\lambda x.\lambda s.x)\ e))\ s \Longrightarrow^* \texttt{handle}_d(h, ((\lambda x.\lambda s.x)\ v))\ s \Longrightarrow^* (\lambda s.v)\ s \Longrightarrow v$$

The second approach is to use *parameterized handlers* [9], which puts the state into the handler expression. The third approach is to augment the handler with a special effect `return`, which modifies the value before removing the handler. The example Figure 8 chooses the first approach, as it keeps the handler semantics simple, although a programmer needs to pay more attention when dealing with handlers receiving arguments.

$$h \quad = \quad \{\mathscr{G} \to \lambda v.\lambda k.\lambda s.k\ s\ s; \mathscr{S} \to \lambda v.\lambda k.\lambda s.k\ s\ v\}$$

$$\begin{array}{ll}
 & e = ((\texttt{perform}\ \mathscr{S}\ \mathsf{num}\ 42) + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}) + 100) \\
 & (\texttt{handle}_d(h, ((\lambda x.\lambda s.x)\ e)))\ 0 \\
\Longrightarrow^* & (\lambda v.\lambda k.\lambda s.k\ s\ v)\ 42\ \lambda y.\texttt{handle}_d(h, ((\lambda x.\lambda s.x)\ (y + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}) + 100)))\ 0 \\
\Longrightarrow^* & (\lambda y.\texttt{handle}_d(h, ((\lambda x.\lambda s.x)\ (y + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}) + 100))))\ 0\ 42 \\
\Longrightarrow^* & \texttt{handle}_d(h, ((\lambda x.\lambda s.x)\ ((0 + (\texttt{perform}\ \mathscr{G}\ \mathsf{num}\ \mathsf{unit}) + 100))))\ 42 \\
\Longrightarrow^* & \texttt{handle}_d(h, ((\lambda x.\lambda s.x)\ 142))\ 42 \\
\Longrightarrow^* & (\lambda s.142)\ 42 \\
\Longrightarrow^* & 142
\end{array}$$

Figure 9: Handler with state

3.2 Language Model

`Effects`, inspired by [18], equips Base with effects and effect handlers. We extend the type system into a kinded type system based on System F, mainly to separate between effect types and normal types. System F also provides polymorphism. The type system

adapts its syntax of functions into $\tau_1 \to^{\varepsilon} \tau_2$ denotes an effect ε (may) occurs during the execution of the function.

Following [18], effects are group of "operations", distinct by its label. An operation $\mathcal{O}$ of effect ε is invoked by applying `perform` $\mathcal{O}\ \tau$ with an argument, and produces an effect ε. Given an expression e that would perform some effect ε, a handler h providing all definitions for operations of ε must be installed. Each handler is tagged with a semantic identifier $m \in \{d, s\}$ where d denotes deep handler semantic and s uses shallow handler semantic. The main difference between these two handler semantics is that the continuation automatically registers the handler again (deep) or not (shallow). The language provides two ways to install a handler, $\texttt{handle}_m(h, e)$ and $\texttt{handler}_m(h)\ (\lambda_.e)$. The former is the base syntax, during evaluation of e, the handler h is used. The other syntax is useful for suspending a computation, and run it only when a handler is applied, providing flexibility such as different handlers.

A handler is a mapping between operation names and their definitions. An operation is a function with the template $\Lambda\alpha.\lambda x : \tau_1.\lambda k : \tau_2.e$. The polymorphic type variable α is used to make operations polymorphic. The argument x is the operation argument. The argument k is the *continuation* or *resumption*, enclosing the rest of the program after the operation invocation. When k is called, the program resumes with the argument provided to k.

Because k is a function, a handler can return k and let the program use continuation anyhow the programer wants. This is described in [18] as "undesirable in practice" and define a restriction named *scoped resumption*. Our language also follows the same principle and does not allow returning the continuation k (directly or indirectly through closure). However, we do not enforce this and left out for the language implementators.

`Effects` extends Base

$$
\begin{array}{rcl}
\tau & ::= & \ldots \\
& | & \alpha^k \mid c^k \tau \ldots \mid \tau \to^\varepsilon \tau \mid \forall \alpha^k . \tau \\
& | & \texttt{blab} \\
\\
k & ::= & * \\
& | & k \to k \\
& | & \texttt{eff} \\
& | & \texttt{lab} \\
\\
m & ::= & d \\
& | & s \\
\\
e & ::= & \ldots \\
& | & \lambda^\varepsilon x : \tau . e \\
& | & e[\tau] \\
& | & \texttt{handle}^\varepsilon_m(h, e) \\
& | & \texttt{error}_p \\
\\
h & ::= & \{\texttt{op} \to f, \ldots, \} \\
\\
f & ::= & \Lambda \alpha . \lambda^\varepsilon x : \tau_1 . \lambda^\varepsilon k : (\tau_2 \to^\varepsilon \tau) . e
\end{array}
$$

$$
\begin{array}{rcl}
l & ::= & \text{Error Label} \\
\\
v & ::= & \ldots \\
& | & \lambda^\varepsilon x : \tau . e \\
& | & \Lambda \alpha^k . v \\
& | & \texttt{handler}^\varepsilon_m(h) \\
& | & \texttt{perform}^\varepsilon\ \texttt{op}\ \tau \\
& | & \texttt{error}_l \\
\\
E & ::= & \ldots \\
& | & E[\tau] \\
& | & \texttt{handle}^\varepsilon_m(h, E) \\
\\
F & ::= & \square \\
& | & F + e \mid v + F \mid F - e \mid v - F \\
& | & F \wedge e \mid v \wedge F \mid F \vee e \mid v \vee F \\
& | & F\ e \mid v\ F \\
& | & \text{if } F \text{ then } e \text{ else } e \\
& | & F[\tau]
\end{array}
$$

Listing 1: `Effects` language

A function can use multiple effects, and are represented as rows with equivalence up to reordering. An empty effect is represented as $\langle\rangle$. Effects can be concatenated by $\langle l, \varepsilon\rangle$ where effect row ε is extended with effect l. For ease of writing, we allow writing $\langle l_1, l_2, ..., l_n\rangle$ as shorthand instead of $\langle l_1, \langle l_2, ...\langle l_n\rangle\rangle\rangle$. We let effect equivalence is up to same size, and ordering, that is $\langle l_1, l_2\rangle \equiv \langle l_2, l_1\rangle$. Effects are checked during type derivations, by passing inwards. Therefore, functions can state it has more effect than their bodies do.

The language is explicit about the effects. Lambdas are annotated with effects, as well as perform (operation invocation), and handler installation. These effect annotations are used only during type checking, they do not affect the execution.

Typing for an effect language requires some consideration. Following the works in [18], we define the type judgement relationship $\Gamma \vdash e : \tau \mid \varepsilon$ between a type context Γ, an expression e and assert it against type τ with some possible effects ε. Variables and base values do not have effects, they can take any effects. For this reason, we define $\Gamma \vdash_{\text{val}} v : \tau$. Handlers are typed using $\Gamma \vdash_{\text{ops}} m : h : \tau \mid l \mid \varepsilon$, that it handles a *single* effect l, other effects in ε remains unhandled, and all operations returns τ, and there are two different rules depending on the handler semantics m. All effects signatures are defined in a global set Σ for fast accessing the type of the operations. This set can be built by going through all handlers, and its existence is assumed during the start of type checking. All type rules unique to `Effects` is presented in Figure 10.

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash_{\text{val}} x : \tau}\ \text{T-E-Var} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \mid \varepsilon}{\Gamma \vdash_{\text{val}} \lambda^{\varepsilon} x : \tau_1 . e : \tau_1 \to^{\varepsilon} \tau_2}\ \text{T-E-Abs}$$

$$\frac{\Gamma \vdash_{\text{val}} v : \tau}{\Gamma \vdash v : \tau \mid \varepsilon}\ \text{T-E-Val} \qquad \frac{k \neq \texttt{lab} \quad \Gamma \vdash_{\text{val}} v : \tau}{\Gamma \vdash_{\text{val}} \Lambda \alpha^{k} . v : \forall \alpha^{k} . \tau}\ \text{T-E-TAbs}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to^{\varepsilon} \tau_2 \mid \varepsilon \quad \Gamma \vdash e_2 : \tau_1 \mid \varepsilon}{\Gamma \vdash e_1\ e_2 : \tau \mid \varepsilon}\ \text{T-E-App} \qquad \frac{\vdash_{\text{wf}} \tau : k \quad \Gamma \vdash e : \forall \alpha^{k} . \tau_1 \mid \varepsilon}{\Gamma \vdash e[\tau] : \tau_1[\alpha := \tau] \mid \varepsilon}\ \text{T-E-Tapp}$$

$$\frac{\mathsf{op} : \forall \alpha . \tau_1 \to \tau_2 \in \Sigma(l) \qquad \alpha \notin \text{ftv}(\Gamma)}{\Gamma \vdash_{\text{val}} \texttt{perform}^{\varepsilon}\ \mathsf{op}\ \tau : \tau_1[\alpha := \tau] \to^{\langle l \mid \varepsilon \rangle} \tau_2[\alpha := \tau]}\ \text{T-E-Perform}$$

$$\frac{\mathsf{op}_i : \forall \alpha . \tau_1 \to \tau_2 \in \Sigma(l) \qquad \alpha \notin \text{ftv}(\varepsilon, \tau) \quad \Gamma \vdash_{\text{val}} f_i : \forall \alpha . \tau_1 \to^{\varepsilon} ((\tau_2 \to^{\varepsilon} \tau) \to^{\varepsilon} \tau)}{\Gamma \vdash_{\text{ops}} d : \{\mathsf{op}_1 \to f_1, ..., \mathsf{op}_n \to f_n\} : \tau \mid l \mid \varepsilon}\ \text{T-E-Ops-Deep}$$

$$\frac{\mathsf{op}_i : \forall \alpha . \tau_1 \to \tau_2 \in \Sigma(l) \qquad \alpha \notin \text{ftv}(\varepsilon, \tau) \quad \Gamma \vdash_{\text{val}} f_i : \forall \alpha . \tau_1 \to^{\varepsilon} \left((\tau_2 \to^{\langle l,\ \varepsilon \rangle} \tau) \to^{\varepsilon} \tau \right)}{\Gamma \vdash_{\text{ops}} s : \{\mathsf{op}_1 \to f_1, ..., \mathsf{op}_n \to f_n\} : \tau \mid l \mid \varepsilon}\ \text{T-E-Ops-Shallow}$$

$$\frac{\Gamma \vdash_{\text{ops}} m : h : \tau \mid l \mid \varepsilon}{\Gamma \vdash_{\text{val}} \texttt{handler}^{\varepsilon}_{m}(h) : \left(\mathsf{unit} \to^{\langle l \mid \varepsilon \rangle} \tau\right) \to^{\varepsilon} \tau}\ \text{T-E-Handler}$$

$$\frac{\Gamma \vdash_{\text{ops}} m : h : \tau \mid l \mid \varepsilon \qquad \Gamma \vdash e : \tau \mid \langle l \mid \varepsilon \rangle}{\Gamma \vdash \texttt{handle}^{\varepsilon}_{m}(h, e) : \tau \mid \varepsilon}\ \text{T-E-Handle}$$

Figure 10: Type Rules for `Effects`

Lastly the semantic model of Effects is presented in Listing 2. A handler can be installed to a function as a shorthand, where it applies the function with a unit argument, which is ignored. The mode determines how its resumption behaves: a deep handler (when $m = d$) re-installs $\texttt{handle}_d(h, ...)$ around the captured continuation E, while a shallow handler (when $m = s$) resumes directly with the captured evaluation context and does not reinstall itself. In shallow handler, the continuation k might invoke the effect of the current handler so it must be denoted with the extra effect l where $\texttt{op} \to f \in h$ and $\texttt{op} \in \Sigma(l)$.

> In [18] and the Koka language, of which we based our Effects language, the authors explicitly use deep handlers and ignore semantics for shallow handlers. We provide shallow handler semantic because the compilation [from Contracts to Effects] benefits from having shallow handlers.

$$(\Lambda\alpha^{k}.v)[\tau] \quad \rightarrow \quad v[\alpha := \tau] \qquad \text{Step-E-TApp}$$

$$\texttt{handler}^{\varepsilon}_{m}(h)\ v \quad \rightarrow \quad \texttt{handle}^{\varepsilon}_{m}(h, v\ ()) \qquad \text{Step-E-Handler}$$

$$\texttt{handle}^{\varepsilon}_{m}(h, v) \quad \rightarrow \quad v \qquad \text{Step-E-Return}$$

$$\frac{\texttt{op} \notin \texttt{bop}(E) \wedge (\texttt{op} \rightarrow f) \in h \qquad \texttt{op} : \forall\alpha.\tau_1 \rightarrow \tau_2 \in \Sigma(l) \qquad k = \lambda^{\varepsilon} x : \tau_2[\alpha := \tau].\texttt{handle}^{\varepsilon}_{d}(h, E[x])}{\texttt{handle}^{\varepsilon}_{d}(h, E[\texttt{perform}\ \texttt{op}\ \tau\ v]) \quad \rightarrow \quad f[\tau]\ v\ k} \text{ Step-E-Perform-Deep}$$

$$\frac{\texttt{op} \notin \texttt{bop}(E) \wedge (\texttt{op} \rightarrow f) \in h \qquad \texttt{op} : \forall\alpha.\tau_1 \rightarrow \tau_2 \in \Sigma(l) \qquad k = \lambda^{\langle l,\ \varepsilon\rangle} x : \tau_2[\alpha := \tau].E[x]}{\texttt{handle}^{\varepsilon}_{s}(h, E[\texttt{perform}\ \texttt{op}\ \tau\ v]) \quad \rightarrow \quad f[\tau]\ v\ k} \text{ Step-E-Perform-Shallow}$$

$$\frac{E \neq \square}{E\left[\texttt{error}_{p}\right] \Longrightarrow \texttt{error}_{p}} \text{ Step-E-Error}$$

$$\frac{e_1 \rightarrow e_2}{E[e_1] \Longrightarrow E[e_2]} \text{ Step-E} \qquad \frac{e_1 \dashrightarrow e_2}{E[e_1] \Longrightarrow E[e_2]} \text{ Step-E-Base}$$

Helper functions

$$\begin{array}{lcl}
\texttt{bop}(\square) & = & \varnothing \\
\texttt{bop}(E\ e) & = & \texttt{bop}(E) \\
\vdots & \vdots & \vdots \\
\texttt{bop}(\texttt{handle}^{\varepsilon}_{m}(h, E)) & = & \texttt{bop}(E) \cup \{\texttt{op} \mid (\texttt{op} \rightarrow f) \in h\}
\end{array}$$

Listing 2: Reduction Rules for `Effects`

CHAPTER 4

Compiler

Following the compiler of [6], we can compile all contracts recursively by padding a new lambda for functional contracts, unfolding until flat contract. Through observation, instead of compiling flat contracts into if $p\ v$ then v else blame, this phrase is pushed into the handler and an effect is performed. Immediately, we can see an easy compilation strategy.

$\texttt{handler}_s(\{\mathscr{C}^1 \to \lambda_.\lambda k.\ \text{if } p\ v \text{ then } k\ v \text{ else blame}\})\ (\lambda_.(\texttt{perform}\ \mathscr{C}^1\ \mathsf{unit}))$

There are two main questions to this approach that when answered, it shows why this is undesired. First, do we need a different effect for each different contract? Second, where should we put the handler? The first question is relatively no, if we push the whole if/else clause into the handler. Which we can simplify by using tuples, or similar techniques to encode the predicate, value, and even blame label into the first argument of handler. Then it becomes:

$$\texttt{handler}_s(\{\mathscr{C} \to \lambda\langle l, \langle p, v\rangle\rangle.\lambda k.\ \text{if } p\ v \text{ then } k\ v \text{ else } \texttt{error}_l\})\ (\lambda_.(\texttt{perform}\ \mathscr{C}\ \langle l, \langle p, v\rangle\rangle))$$

Where we write $\lambda\langle l, \langle p, v\rangle\rangle$ as a short hand to deconstruct the tuples, which can be done through tuple projection. After this construction, all contract checking can be unified under a single effect $\mathscr{C}$. The second question is rather hard to answer. Obviously it can be placed directly where the effect is performed. We call this a *local compilation* (of contracts into effects). Then we experiment whether we can pull the handler far away from its local position. Without changing the handler, we try installing it at the top most place of the

program. For functional contracts, this handler works. However, dependent contracts will get stuck with this handler, as shown in the example of Figure 11.

$$h_{\text{naive}} = \{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.\ \text{if } e\ v \text{ then } k\ v \text{ else } \texttt{error}_l\}$$

where $\lambda\langle l, \langle e, v\rangle\rangle$ deconstructs the value into tuple pattern

$$\begin{array}{ll} & \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1) \to_i \lambda y.\ \mathsf{flat}(e_2), f)\ 42 \\ & \qquad \text{where } e_1 = \lambda x.\ \mathsf{true},\ e_2 = \lambda x.x > y, \text{ and } f = \lambda x.x + x \\ \rightarrowtail & (\lambda x_1.\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3, f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)\rangle\rangle)\ 42 \\ & \qquad \text{where } e_3 = e_2[y := \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle] \end{array}$$

$$\begin{array}{ll} & \texttt{handler}_d(h_{\text{naive}})\ \lambda_.(\lambda x_1.\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3, f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)\rangle\rangle)\ 42) \\ \Longrightarrow & \texttt{handle}_d(h_{\text{naive}}, (\lambda x_1.\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3, f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, x_1\rangle\rangle)\rangle\rangle\ 42)) \\ \Longrightarrow & \texttt{handle}_d(h_{\text{naive}}, \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3', f\ (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, 42\rangle\rangle)\rangle\rangle) \\ & \qquad \text{where } e_3' = e_2[y := \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, 42\rangle\rangle] \\ \Longrightarrow^* & \texttt{handle}_d(h_{\text{naive}}, \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3', f\ 42\rangle\rangle) \\ \Longrightarrow^* & \texttt{handle}_d(h_{\text{naive}}, \texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle k, \langle e_3', 84\rangle\rangle) \\ \Longrightarrow^* & \text{if } e_3'\ 84 \text{ then } (\lambda x.\texttt{handle}_d(h_{\text{naive}}, x))\ 84 \text{ else } \texttt{error}_k \\ \Longrightarrow^* & \text{if } 84 > (\texttt{perform}\ \mathscr{C}\ \mathsf{num}\ \langle l, \langle e_1, 42\rangle\rangle) \text{ then } (\lambda x.\texttt{handle}_d(h_{\text{naive}}, x))\ 84 \text{ else } \texttt{error}_k \\ & \textit{stuck} \end{array}$$

Figure 11: Naive handler stuck

The reason why the above handler cannot work with dependent contracts is because during the handling of the contract, it may call into another contract. But because it is currently handling the contract, there is no handler to handle the nested contract effect. One may argue that if each contract is given a different effect and stack the handler together, it might solve this problem, but then we need to work out the order of the handler[1]. By abusing the recursive function μ, we can construct a more elegant wrapper that solves dependent contracts effects. Through our experiement, we have two versions of the wrapper, one that

[1]Handler captures context, and if the outside handler is invoked first, the inside handler is now inside the continuation not outside, and it cannot handle effects called in condition code.

uses deep handler, and one that uses shallow handler. All three compiler versions, local, deep, shallow, preserve the semantics.

Careful readers may aware that the handler is pulled to the outside of the [compiled] program, and may also wonder if we can push it further inside the program (except local compilation). A quick answer is "No". A good answer is "It depends". The problem arises when we try to push the handler deeper is when an effect is stored inside a lambda. Consider the program in Figure 12, the program is an application between g and the argument. The argument runs some contract checking before returning a function f. By the analysis, if g does not call f, the handler can be pushed to near $\mathsf{mon}_{j'}^{k',l'}(e', v')$. However, because f is invoked, we need to place the handler outside the whole program. This case is a simple case to demonstrate the placement problem. Readers can consider cases where variables are bounded where each of them may or may not have a contract. Therefore, a recursive/ homomorphic compiler is not suitable to analyze the absent or presence of such effect.

$$f = \lambda x.\mathsf{mon}_{j}^{k,l}(e, x)$$

$$g = \lambda f.f\ 1$$

$$\text{main} = g\left((\lambda y.f)\left(\mathsf{mon}_{j'}^{k',l'}(e', v')\right)\right)$$

Figure 12: Handler placement analysis case

4.1 Formal Model

Our compiler is homomorphic over expression e. Because it compiles from typed Contracts to typed `Effects`, the compiler is defined through type-directed compilation. Although we have three different compiler models, two of them (Shallow and Deep) has the same compilation except for the wrapper. Therefore, we only provide two different compilers. The interesting contract compilation rules are provided in Figure 13.

- $\Gamma \vdash e : \tau \rightarrowtail^{l} e'$ for local compilation

- $\Gamma \vdash e : \tau \rightarrowtail e'$ for non-local compilation

$$\frac{\Gamma \vdash e_1 : \tau \to \mathsf{bool} \rightarrowtail e_3 \qquad \Gamma \vdash e_2 : \tau \rightarrowtail e_4}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) : \tau \rightarrowtail \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_3, e_4 \rangle\rangle}\ \text{Compile-Flat}$$

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrowtail e_2 \qquad \Gamma \vdash \lambda f : \tau_1 \to \tau_2 . \lambda x : \tau_1 . \mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_3}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail e_3\ e_2}\ \text{Compile-Func}$$

$$\frac{\begin{array}{c} f, x \notin \mathrm{dom}(\Gamma) \cup \{y\} \cup \mathrm{fv}(\kappa_1, \kappa_2) \qquad \Gamma \vdash e_1 : \tau_1 \to \tau_2 \rightarrowtail e_2 \\ e_w = \lambda f : \tau_1 \to \tau_2 . \lambda x : \tau_1 . \mathsf{mon}_j^{k,l}\left(\kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right], f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\ \Gamma \vdash e_w : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail e_3 \end{array}}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail e_3\ e_2}\ \text{Compile-Dep}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \mathsf{bool} \rightarrowtail^l e_3 \qquad \Gamma \vdash e_2 : \tau \rightarrowtail^l e_4}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) : \tau \rightarrowtail^l \mathtt{handler}_s(\mathtt{h}_{\mathscr{C}})\ \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_3, e_4 \rangle\rangle}\ \text{Local-Compile-Flat}$$

Figure 13: Compiler Rules

4.2 Contracts Hijacking (and Option Contracts)

When using non-local compilers, because the handler are moved outside, users can provide a different handler scope inside the program. This handler can alternate the contract execution policy. This allows us to provide optional contracts, where the handler continue without executing the contract predicate, or executing the contract predicate but ignore the result on purpose.

Consider the following example:

$$f = \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1) \to \mathsf{flat}(e_2), \lambda x.x)$$

$$\text{main} = (\lambda f.(\text{ignore } (f\ 1)) + (f\ 2))\ f$$

Where ignore is compiled into a wrapper handler that does not check $\{\mathscr{C} \to \Lambda\alpha.\lambda x.\lambda k.k\ x\}$. During the execution of $f\ 1$, it is executed inside this handler context and all contract checking are ignored. In [3], the author describe the option contract that attaches to the value, and are explicitly executed. Our model for optional contracts are different, but should share the same semantics. Contracts are executed by default, but can be "turned off". Its model are closer to *Toggle-able Contracts*. To turn the contract on again, another handler can be injected to execute the checks. Although it brings a way to provide option contracts, this model modifies the environment rather than the contract value, which violates how contracts should work. The example belows show where our modeling might disagree with the option contracts model.

$$f_1 \text{ is option contract protected}$$

$$f_2 \text{ is contract protected}$$

$$\text{choose} = \lambda a.\lambda b.\text{ randomly choose a or b}$$

$$\text{main} = (\lambda f.f\ ())\ \text{choose}(f_1, f_2)$$

Because the main program does not know which function to run, our hijacking handler cannot segment the environment between invocation of f_1 and f_2. While option contract on f_1 stops contract execution on f_1 if $\text{choose}(f_1, f_2) \twoheadrightarrow^* f_1$.

4.3 States (and Trace Contracts)

Local compiler cannot implement contract states, since it is a one-shot handler, the contract cannot reuse anything. However, when we use the outside wrapper of shallow and deep compiler versions, we observe that it can encode states natively. In Section 3.1, we demonstrated that states are encoded seemlessly by providing the initial state and the handler form is $\lambda x.\lambda k.\lambda s.e$ where x, k, s are perform value, continuation, and state respectively. Applying the same idea, we extend the handler with two new effects, $\mathscr{G}, \mathscr{M}$, representing get state and set state respecively.

We demonstrate how state(s) can be encoded effortlessly in shallow compiler version. Figure 14 shows the effect handler adapted to receive and handle a state variable. While executing the condition $p\,v$, it can invoke effect $\mathscr{G}, \mathscr{M}$ to obtain the current state, or set the current state. Since the state is provided as an extra argument, to obtain the result of the execution the wrapper must be a function that accepts the state(s).

$$H = \mu H.\texttt{handler}_s(h_s)$$

$$h_{s(\mathscr{C})} = \lambda\langle l, \langle p, v\rangle\rangle.\lambda k.\lambda s.H\ (\lambda_.\ \text{if}\ p\,v\ \text{then}\ k\,v\ \text{else}\ \texttt{error}_l)\ s$$

$$h_{s(\mathscr{G})} = \lambda u.\lambda k.\lambda s.H\ \lambda_.k\ s\ s$$

$$h_{s(\mathscr{M})} = \lambda s'.\lambda k.\lambda s.H\ \lambda_.k\ s'\ s'.$$

$$\mathsf{wrap}_s = \lambda f.H\ (\lambda_.(\lambda r.\lambda s.r)\ f)\ s$$

Figure 14: Shallow check handler with contract-controlled state

The current encoding of contract state is limited to a single value. It is easy to expand into multiple state variables. Since the handler always accepts three arguments (type,

value, continuation), the rest can be functions. The body of the handler is changed into $\lambda s_1 .... \lambda s_i . e$ form, where e is the main handler code. And for each state, we provide a pair of effects $(\mathscr{G}^i, \mathscr{M}^i)$ that returns and updates the corresponding s_i. Wrapper must be updated to ignore states s_i and return the result only. The wrapper must also provide all initial states to ensure execution.

$$H = \mu H.\texttt{handler}_s(h_s)$$

$$h_{s(\mathscr{C})} = \lambda\langle l, \langle p, v\rangle\rangle.\lambda k.\lambda s_1.\lambda s_2.H\ (\lambda_.\ \text{if } p\ v \text{ then } k\ v \text{ else } \texttt{error}_l)\ s_1\ s_2$$

$$h_{s(\mathscr{G}^1)} = \lambda u.\lambda k.\lambda s_1.\lambda s_2.H\ \lambda_.k\ s_1\ s_1$$

$$h_{s(\mathscr{M}^1)} = \lambda s_1'.\lambda k.\lambda s_1.\lambda s_2.H\ \lambda_.k\ s_1'\ s_1'.$$

$$h_{s(\mathscr{G}^2)} = \lambda u.\lambda k.\lambda s_1.\lambda s_2.H\ \lambda_.k\ s_2\ s_2$$

$$h_{s(\mathscr{M}^2)} = \lambda s_2'.\lambda k.\lambda s_1.\lambda s_2.H\ \lambda_.k\ s_2'\ s_2'.$$

$$\mathsf{wrap}_s = \lambda f.H\ (\lambda_.(\lambda r.\lambda s_1.\lambda s_2.r)\ f)\ s_1\ s_2$$

Figure 15: Shallow check handler with contract-controlled multiple states

A slightly different encoding is to use tuples $\langle s_1, \langle s_2, \langle s_3, ...\rangle\rangle\rangle$, which is only available for languages with tuples support. If we have multiple tuple spaces and index-able projection π_i, the handler can be written easier without a series of nested left/right injections.

Users want to use this must define the state in their program. Assume an extension to contract to provide states in Figure 16, $\mathsf{cstate}(e_1, \lambda x.e_2)$ where e_1 is the initial value of the state, and $\lambda s.e_2$ is the expression where the state will be bounded into s and appears in e_2'. While typing e_2, we provide $s : \mathsf{cstate}(\tau)$ in the context so that it can only be used under contract. There are only two expressions that can use these contract state variables, $\mathsf{cget}, \mathsf{cset}$. Under the dynamic settings, there is a storage that watches the contract states (similarly to mutable boxes).

$$
\begin{array}{rcl}
\tau & ::= & \ldots \\
 & | & \mathsf{cstate}(\tau) \\
 & & \\
e & ::= & \ldots \\
 & | & \mathsf{cstate}(e, \lambda x.e) \\
 & | & \mathsf{cget}(x) \\
 & | & \mathsf{cset}(x, e)
\end{array}
$$

Figure 16: Extend Contracts with contract states

The compiler should for each $\mathsf{cstate}(e_1, \lambda s.e_2)$ define a new effect, use e_1 as the initial state value and for every occurance of s in e_2, modifies $\mathsf{cget}, \mathsf{cset}$ accordingly to effect calling. This compiler is not yet complete, since there are cases where cstate are enclosed inside a function, so each time that function is run, the state is initialized fresh.

The setup provides a possible compilation for Trace Contracts, introduced in [12]. In trace contracts model, the collector (state) is defined as a list and it is being updated automatically. If we rewrite the handler to $\{\mathscr{C} \to \Lambda\alpha.\lambda x.\lambda k.\lambda s.(\lambda_.e)\ (\texttt{perform}\ \mathscr{M}\ x)\}$, we obtain a closer ressemblance to trace contracts semantics.

4.4 Connection to Effectful Contracts

Our compiler model shows that basic contract monitors can be compiled into an effect, and maintain the semantics with a crafted handler. Effectful contracts, [13], explicitly defines contracts for effects, for which we have not defined compilation rules. Following the constructions in [13], let $\mathsf{mon}_j^{k,l}(\kappa_1 \rhd \kappa_2, e)$ be a main effect contract. That if e executes an effect with value v_1, then $\mathsf{mon}_j^{k,l}(\kappa_1, v)$ is passed onto the effect handler, if the effect continues with value v_2, it returns $\mathsf{mon}_j^{l,k}(\kappa_2, v_2)$. However, because this construction can be nested, we need to collect multiple contracts on v_1, v_2. This semantic suggests a compilation that takes e and find out all $\texttt{perform eff}$ expressions, and perform a wrap of the input value, and output value. When all effect contracts are rewritten into basic contracts, we can perform compilation rules in Figure 13 to convert them into effects.

$$\mathsf{mon}_j^{k,l}(\kappa_1 \rhd \kappa_2, \texttt{perform eff}\ v) \rightarrowtail \mathsf{mon}_j^{l,k}\Big(\kappa_2, \texttt{perform eff}\ \mathsf{mon}_j^{k,l}(\kappa_1, v)\Big)$$

The effectful contracts model also specify the contract-effect contracts, or contracts for effects invoked inside a contract. However, this requires a handler semantics for contracts' effects. `Effects` only supports handler semantic(s) for the whole program. For this reason, we ignore this contract-effect contracts construction.

CHAPTER 5

Implementation

Our formal models are implemented in Redex and in OCaml. While the Redex models are closer to the formal compiler models, the OCaml implementation is more practical.

5.1 Redex models

We model three languages: Base, Contracts, `Effects`, and two compilers one for local, and non-local. To model the compiler, we create a Redex union language of Contracts and `Effects`, and define a judgement `(compile C:Γ C:x C:τ E:x)` where `C:` belongs to Contracts and `E:` belongs to `Effects`, and the input output model is `(compile I I O O)`. The interesting compilation rules are displayed in Listing 3. Listing 4 shows the local version compiler where all rules are kept except for the flat compilation rule accompanied by a handler.

For non-local compiler, their wrappers must be created accordingly to Listing 5 (using shallow handlers) or to Listing 6 (using deep handlers). We use `genericeff` as a common effect list consisting of the single contract checking effect identifier for type checking.

```racket
[(compile C:Γ C:e_1 (C:τ -> bool) E:e_1)
 (compile C:Γ C:e_2 C:τ E:e_2)
 ------------------------------------------------------- "C-Mon-Flat"
 (compile C:Γ (mon C:k C:l C:j (flat C:e_1) C:e_2) C:τ
          ((perform genericeff 𝒞 C:τ) (t E:e_1 E:e_2)))]

[(compile C:Γ C:e (C:τ_1 -> C:τ_2) E:e_compiled)
 (where (C:x C:x_1) ,(variables-not-in (term (C:Γ C:e)) '(x x_1)))
 (compile C:Γ
          (λ C:x_1 (C:τ_1 -> C:τ_2) (λ C:x C:τ_1
              (mon C:k C:l C:j C:κ_2 (C:x_1 (mon C:l C:k C:j C:κ_1 C:x)))))
          ((C:τ_1 -> C:τ_2) -> (C:τ_1 -> C:τ_2))
          E:e_4)
 ------------------------------------------------------------------------- "C-Mon-Func"
 (compile C:Γ
          (mon C:k C:l C:j (C:κ_1 -> C:κ_2) C:e)
          (C:τ_1 -> C:τ_2)
          (E:e_4 E:e_compiled))]

[(compile C:Γ C:e (C:τ_1 -> C:τ_2) E:e_compiled)
 (where (C:x C:x_1)
       ,(variables-not-in (term (C:Γ C:e C:κ_1 C:κ_2 C:x_arg)) '(x x_1)))
 (compile C:Γ
          (λ C:x_1 (C:τ_1 -> C:τ_2) (λ C:x C:τ_1 ((λ C:x_arg C:τ_1
                 (mon C:k C:l C:j C:κ_2
                      (C:x_1 (mon C:l C:k C:j C:κ_1 C:x))))
               (mon C:l C:j C:j C:κ_1 C:x))))
          ((C:τ_1 -> C:τ_2) -> (C:τ_1 -> C:τ_2))
          E:e_4)
 ------------------------------------------------------------------------- "C-Mon-Dep"
 (compile C:Γ
          (mon C:k C:l C:j (C:κ_1 ->i C:x_arg C:κ_2) C:e)
          (C:τ_1 -> C:τ_2)
          (E:e_4 E:e_compiled))]
```

Listing 3: Non-local compiler

```racket
[(compile C:Γ C:e_1 (C:τ -> bool) E:e_1)
 (compile C:Γ C:e_2 C:τ E:e_2)
 ------------------------------------------------------ "C-Mon-Flat"
 (compile C:Γ (mon C:k C:l C:j (flat C:e_1) C:e_2) C:τ
          ((s-handler (𝒞)
             ((𝒞
               (Λ β (λ (𝒞) x β (λ (𝒞) k β
                 (if ((injl x) (injr x))
                     (k (injr x))
                     (error 99991))))))))
           (λ (𝒞) ignored unit
             ((perform (𝒞) 𝒞 C:τ) (t E:e_1 E:e_2)))))]
```

Listing 4: Local compiler

```racket
(define-term h-check
  ((effcheck
     (Λ α (λ genericeff x α (λ genericeff k α
       ((w α) (λ genericeff _ unit
         (if ((injl x) (injr x))
             (k (injr x))
             (error 99991)))))))))

(define-term wrap
  (μ w unit (Λ α (λ genericeff f unit ((s-handler genericeff h-check) f)))))
```

Listing 5: Shallow handler wrapper for non-local compiler

```racket
(define-term h-check
  ((effcheck
     (Λ α (λ genericeff x α (λ genericeff k α
       (if ((w bool) (λ genericeff _ unit ((injl x) (injr x))))
           (k (injr x))
           (error 99991)))))))))

(define-term wrap
  (μ w unit (Λ α (λ genericeff f unit ((d-handler genericeff h-check) f)))))
```

Listing 6: Deep handler wrapper for non-local compiler

5.2 OCaml implementation

We have chosen OCaml as a base language to implement our contract system. The choice was purely because of two things. First, OCaml 5 has support for effect handlers

built-in. Second, OCaml has a macro system that we can rely on to modify the program. On top of these two reasons is the author's familiarity with the language.

5.2.1 OCaml Attributes and Rewritters

OCaml does not have a clean "macro system". In fact, OCaml compilation chain allows users to introduce a rewritter defined as a source to source transformation. A rewritter can be any program that takes the whole file source, and returns a (modified) file source. Because there is no restriction, rewritters are free to modify, add, delete source code. However, useful rewritters will try to edit programs with intention. For this purpose, ppxlib, a common library is provided for writting rewritters, allowing them to parse and modify source code in Abstract Syntax Tree (AST) objects. OCaml syntax also support rewritters by providing a way to annotate source code places in AST through the use of *attributes*. Attributes are enclosed in square brackets, tagged with an identifier and carry "code-like" payloads. Readers familiar with OCaml might have used attributes already, as one of the most commonly used attributes is `[@@deriving show]` to allow structs to be print-able.

5.2.2 OCaml Contracts as Attributes

Using attributes, we define contracts and our rewritter acts as the compiler as defined. Wrappers can be added manually or automatically with an attribute denoting the wrapper placement. The modified source code might use functions, objects that was not defined previously, therefore we expect users to include our library where it contains all extra declarations for compilation and running.

In Table 1 is the list of some contract attributes syntax. Users attach these attributes to guard a value, function or variable with some contract `c`, which we discuss later on how to write different contract types. To install the wrapper, users only need to attach `%contract_main` to the expression that has contract checking code.

Attributes	Meaning
`let[@contract c] x = e`	Place contract `c` on value/function `x`
`[%contract_main] e`	Install the contract wrapper on `e`
`let%contract_main f = e`	Install the contract wrapper on `f`
`[%mon c ~pos:p ~neg:n ~loc:j] e`	Place contract `c` on `e` with explicit labels

Table 1: Attributes for Contracts

Our rewritter supports three main types of contracts, flat, functional, and dependent. Firstly, a flat contract is a function that takes a value and returns boolean. Secondly, since contracts can be nested far inside by the use of functional and dpendent contracts, the structured syntax is recursive: if `c_1,c_2` is a contract, then `c_1 @-> c_2` and `dep c_1 (fun x -> c_2)` are contracts. During the writing of contracts, for brewvity, users can use identifiers to a function (must be already in-scope) instead of writing the contract code in full.

Because OCaml is a curry-by-default language, we take that same approach to introduce contracts for multi-parameters functions. For functional contracts, the syntax is clear and precise. For dependent contract, the syntax creates a nested structure that is unnatural to the human eyes. We provide a different syntax for dependent contracts that is inspired by Racket's contract macro, `[@contract dep_args [(arg_1, c_1); ...] (fun arg_1 arg_2 ... -> e)]`. The new syntax is purely for human users, it will be expanded into the nested structure accordingly during the rewriting process before compiling into contract checking code. Below is the example syntax for functional and dependent contracts annotation.

```ocaml
let [@contract c_1 @-> c_2 @-> c_3]
  f x y = e
let [@contract dep c_1 (fun x -> (dep c_2 (fun y -> c_3)))]
  g x y = e
let [@contract dep_args [(x,c_1); (y,c_2)] (fun x y -> c_3)]
  h x y = e
```

The macro unwraps functional and dependent contracts with lambdas until flat contract and simply convert it to an effect invocation. Accordingly to our formal model, there is

only one effect, defined as in Listing 7. Labels are automatically added, and during the lambdas unrolling, labels are swapped in the correct manner.

```ocaml
type label = string
type _ Effect.t +=
  | Check : { pos : label; loc : label; pred : 'a -> bool; value : 'a } -> 'a Effect.t
```

5.2.3 States encoding contracts

Our OCaml model also implement contract states, which we briefly introduced in Section 4.3. For this to work, we provide attributes to declare contract states, `%%contract_states`; and a wrapper attribute with states initialization, `%contract_main_states`. Declaring contract states are a series of let binding expressions, with their names, types and initial value. For each state declaration, the rewritter automatically generate two new effects, `%contract_get_*` and `%contract_set_*`. When writing the contracts, users can use these attributes to obtain or modify the states. Due to the limitation of OCaml rewritter, these states are only available when the whole program is in a single file if using `%contract_main_states`. Otherwise, users can manually write the wrapper to provide all possible states.

Listing 8 illustrates the simple example where a state is used to record inputs to the same function. Because states are controlled using effects, and the macro system does not decouple contract effects from main program effects, users can call these effects. Technically, our rewritter can try to limit the usage of these `%contract_get_*,%contract_set_*` attributes to contract checking code if we can identify contract checking code. However, since contracts can be an identifier of some function, we find that distinction undesirable.

```ocaml
[%%contract_states
  let previous : int option = None]
let nondecreasing input =
  match [%contract_get_previous] with
  | Some previous when input < previous -> false
  | _ ->
      [%contract_set_previous (Some input)];
      true
let[@contract nondecreasing @-> Contract.any] process input = input * 2
let () = [%contract_main_states]
    (List.iter
       (fun input ->
         try Printf.printf "%d -> %d\n" input (process input)
         with Contract.Blame _ -> Printf.printf "%d -> rejected\n" input)
       [ 1; 3; 3; 2; 5 ])
```

5.2.4 Boxes and Vectors contracts

Our OCaml implementation provides special types of boxes and vectors that acts similarly to Racket's chaperone[2]. Our boxes and vectors are encoded as structs with methods. They should not be called directly, but through the provided functions. For example, a box is created using `Box.create`, and its content can be obtained by calling `Box.get`. Through this encoding, we obtain an opaque box/vector with methods as functions. Contracts can now be added directly through these functions. We provide attributes `[@contract box c]` and `[@contract vector c]` that installs the contract `c` to boxes/vectors get/set methods. The rewritter will create a new struct object and install the contracts into the methods. This approach breaks the equality of boxes/vectors but replicates the opaque implementation of chaperone.

5.2.5 Mismatch with the formal models

Although OCaml provides a similar model to `Effects` which we can adapt to add contracts ($\mathsf{mon}_j^{k,l}(\kappa, e)$) and compile them away using the rules as defined in Figure 13, there is a slight difference between the formal models and the OCaml rewritter. One of the more obvious and important mismatch is dependent contracts input. In the formal models, we

[2]Opaque proxies around objects with support for contracts

build up the new contract $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e)$ for codomain as $\kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$ where x is created fresh during lambda padding. In our model, if y appears multiple times in κ_2 then the same contract runs that many times. In a language where there is no mutable boxes, such as `Effects`, running the same code multiple times has no difference from running it once. However, OCaml has mutable boxes, and to be careful, we adjusted the implementation to run the contract once, unless it's a function.

CHAPTER 6

Conclusion

6.1 Contract checking as Effects is Possible

Our formal models have proven that contracts checking can be compiled into a effects, and with an appropriate handler wrapper installed to the whole compiled program, the semantics are preserved. From the results obtained, we can conclude two facts, (1) contracts semantics can be emulated using effects and effect handlers, and (2) contracts do not bring extra expressiveness to effect handlers. The first fact is a direct result from the simulation proof(s) given by compilation of local, shallow, and deep handlers. The second fact is from the local compilation. While contracts do not add more expressiveness [to effects and effect handlers], the local compilation model is not useful. We have suggested modeling for the option contracts, and trace contracts under the shallow wrapper non-local compilation. Therefore, the expressiveness argument is both correct and incorrect.

While the compilation preserves semantics, it is not considered the "correct" contract behavior. Contracts provide guards around a value, when that value is used the contract is triggered. The compilations provide the similar behavior, but breaks when it models option contracts. Option contracts wrap around a value, while our method overrides the environment to disable the check instead of wrapping the check around the value or function. A simple distinction is when there are two values protected under option contracts, but only one is executed. Option contract semantics can control which one is executed because it attaches to the contract. Our method must create two different scope

for these values at their executing place. This scope creation is hard when we consider both values as functions, and used in a complex, intertwined manner.

6.2 Shallow or Deep handlers

We introduced three different compilation strategies, two of which use shallow handlers, and one uses deep handlers. It is widely agreed that the use of deep handlers is more useful, because effects might be invoked multiple times. Therefore, having a single handler that automatically re-installs itself after each invocation aligns with how programmers view effects. We analyzed the local and non-local compilation strategies and see which handler type should we used.

In the local compiler, the shallow handler is the better choice. By analyzing the perform site, $\texttt{handle}_m(h, \texttt{perform h}_{\mathscr{C}}\ v)$ captures the context $E = \square$. Therefore, the continuation is $k_1 = \lambda x.\texttt{handle}_{\mathrm{d}}(h, x)$ for deep handler, and $k_2 = \lambda x.x$ for shallow handler. The continuation is invoked with v yields $k_1\ v \Longrightarrow^* v$ and $k_2\ v \Longrightarrow^* v$. The handler disappear immediately, although deep handler may take an extra step for discarding handler with a value. Performance wise, shallow handler is more optimal. To be precise, for each contract executed, the deep handler version takes an extra step. As a result, the reduction step count increments the more contract the program executes.

In the non-local compiler, we actually have three different wrappers instead of introduced two. The other unmentioned wrapper is similarly to the shallow wrapper it wraps the whole if else clause but using the wrapper of deep non-local compiler. Let call them by compiler I (shallow wrapper), II (deep wrapper as introduced), III (deep wrapper as previously described). Tracing their execution on nested checks, dependent contracts or flat contract with contract code, III eventually adds more nested handle frames and produces the most reduction steps. Both I and II preserve strictly a single handle frame on the

"top", II's continuation has a handle re-installed but is dormant. Because I is a shallow handler, it does not need extra steps to remove the handler on value continuation and reduces some reduction steps.

We proved that regardless of local or non-local compiler, shallow and deep handlers provide the same equivalence to Contracts semantics with some [minimal] overhead when using deep handlers. However, as previously stated, deep handlers are more practical and implemented widely. Users should implement our compilation model using shallow handlers, and use deep handlers only if the language only supports deep handlers, for example the Koka language.

6.3 Effectful Contracts

Our formal models can model simple contracts into effects, but it cannot assert any certainty whether it can also model effectful contracts. Effectful contracts requires contracts for two types of effects, main program effects, and contract effects. The effectful contract model manipulates the contracts by installing contract dynamically, which can be a hurdle to model those contracts into effects. Although a simple observation that effectful contracts basically install checks at effect's entry and effect's exit (continuation invocation), we are unsure if such compilation is possible.

6.4 Practicality

The OCaml implementation has proven practical of the formal model, by using attributes to mark contracts, and let the rewritter to resemble the [non-local] compiler model. The practicality is proven through building a Current-Environment-Storage-Continuation (CESK) machine with some important functionalities protected with contracts. With the correct contract usage, we eliminate writing code for malformed cases. Although in the

context of OCaml, it does not provide much usefulness compared to pattern matching where every cases are considered.

6.5 Comparison to Racket

Racket has chaperones, an opaque proxy over protected value. This proxy maintains the value equality checker, but on usage, runs the contract attached to the value. Our OCaml implementation cannot provide the opaque proxy because we wrap values under [new] lambdas. This is expected, since chaperone are built-in under the language runtime of Racket, and OCaml has no equivalent construction. Regardless of this, we have modeled contracts for boxes and vectors by providing a wrapper struct where methods are functional and contracts are installed onto these functions.

Racket contracts have no state management built-in. Users want state contracts should either use mutable boxes or the `trace-contract` library. Our OCaml implementation encodes states without extra effort by using handler's state encoding trick through lambdas.

The above facts do not show that our implementation is superior to Racket's contract system, nor that it is inferior. The two models have their limitations. However, an interesting question arises. What if our implementation supports chaperone, or if Racket supports effects. We think that utilizing the chaperone models, and adopting effects into Racket might provide a better model. Chaperone maintains the object equality, and effects compilation provides contract states without hassle.

Bibliography

[1] Andrej Bauer and Matija Pretnar. 2014. An Effect System for Algebraic Effects and Handlers. *Log. Methods Comput. Sci.* 10, 4 (2014). https://doi.org/10.2168/LMCS-10(4:9)2014

[2] Christos Dimoulas and Matthias Felleisen. 2011. On contract satisfaction in a higher-order world. *ACM Trans. Program. Lang. Syst.* 33, 5 (2011), 16:1–16:29. https://doi.org/10.1145/2039346.2039348

[3] Christos Dimoulas, Robert Bruce Findler, and Matthias Felleisen. 2013. Option contracts. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2013, part of SPLASH 2013, Indianapolis, IN, USA, October 26-31, 2013*, 2013. ACM, 475–494. https://doi.org/10.1145/2509136.2509548

[4] Christos Dimoulas, Sam Tobin-Hochstadt, and Matthias Felleisen. 2012. Complete Monitors for Behavioral Contracts. In *Programming Languages and Systems - 21st European Symposium on Programming, ESOP 2012, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2012, Tallinn, Estonia, March 24 - April 1, 2012. Proceedings (Lecture Notes in Computer Science)*, 2012. Springer, 214–233. https://doi.org/10.1007/978-3-642-28869-2_11

[5] Tim Disney, Cormac Flanagan, and Jay McCarthy. 2011. Temporal higher-order contracts. *SIGPLAN Not.* 46, 9 (September 2011), 176–188. https://doi.org/10.1145/2034574.2034800

[6] Robert Bruce Findler and Matthias Felleisen. 2013. ICFP 2002: Contracts for higher-order functions. *ACM SIGPLAN Notices* 48, 4S (2013), 34–45. https://doi.org/10.1145/2502508.2502521

[7] Oleg Kiselyov and KC Sivaramakrishnan. 2018. Eff directly in OCaml. *arXiv preprint arXiv:1812.11664* (2018).

[8] Daan Leijen. 2016. *Algebraic Effects for Functional Programming.* Retrieved from https://www.microsoft.com/en-us/research/publication/algebraic-effects-for-functional-programming/

[9] Daan Leijen. 2017. Type directed compilation of row-typed algebraic effects. *SIGPLAN Not.* 52, 1 (January 2017), 486–499. https://doi.org/10.1145/3093333.3009872

[10] Paul Blain Levy. 2013. Call-by-push-value. Doctoral dissertation.

[11] Bertrand Meyer. 1992. The eiffel programming language. *See http://www. eiffel. com* (1992).

[12] Cameron Moy and Matthias Felleisen. 2023. Trace contracts. *J. Funct. Program.* 33, (2023). https://doi.org/10.1017/S0956796823000096

[13] Cameron Moy, Christos Dimoulas, and Matthias Felleisen. 2024. Effectful Software Contracts. *Proc. ACM Program. Lang.* 8, POPL (2024), 2639–2666. https://doi.org/10.1145/3632930

[14] Gordon D. Plotkin and John Power. 2003. Algebraic Operations and Generic Effects. *Appl. Categorical Struct.* 11, 1 (2003), 69–94. https://doi.org/10.1023/A:1023064908962

[15] Matija Pretnar. 2010. Logic and handling of algebraic effects. Doctoral dissertation. Retrieved from https://hdl.handle.net/1842/4611

[16] Matija Pretnar. 2015. An Introduction to Algebraic Effects and Handlers. Invited tutorial paper. In *The 31st Conference on the Mathematical Foundations of Programming Semantics, MFPS 2015, Nijmegen, The Netherlands, June 22-25, 2015 (Electronic Notes in Theoretical Computer Science)*, 2015. Elsevier, 19–35. https://doi.org/10.1016/J.ENTCS.2015.12.003

[17] K. C. Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, and Anil Madhavapeddy. 2021. Retrofitting Effect Handlers onto OCaml. *CoRR* abs/2104.00250, (2021). Retrieved from https://arxiv.org/abs/2104.00250

[18] Ningning Xie, Jonathan Immanuel Brachthäuser, Daniel Hillerström, Philipp Schuster, and Daan Leijen. 2020. Effect handlers, evidently. *Proc. ACM Program. Lang.* 4, ICFP (2020), 99:1–99:29. https://doi.org/10.1145/3408981

[19] Dana N Xu. 2014. Dynamic Contract Checking for OCaml.

CHAPTER A

Base Safety Theorem

We prove type safety of Base. The language has no mutable cells, so the store never changes and a step is just $e \longrightarrow e'$.

Each binary operator $o \in \{+, -, \wedge, \vee\}$ has a signature $o : \tau_a \times \tau_b \to \tau_o$ (used by T-Op), with $+, - : \mathsf{num} \times \mathsf{num} \to \mathsf{num}$ and $\wedge, \vee : \mathsf{bool} \times \mathsf{bool} \to \mathsf{bool}$.

A.1 Type Safety

Theorem A.1. (Base Type Safety)

1. If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$
2. If $\cdot \vdash e : \tau$ then e is a value or $e \longrightarrow e'$ for some e'.

Proof. By Lemma A.2 and Lemma A.3. □

Lemma A.2. (Base Preservation) If $\Gamma \vdash e_1 : \tau$ and $e_1 \longrightarrow e_2$ then $\Gamma \vdash e_2 : \tau$.

Proof of Lemma A.2.

$e_1 = E[e_1'], e_2 = E[e_2'], e_1' \dashrightarrow e_2'$	`Step`
$\Gamma \vdash e_1' : \tau_1$	Lemma A.5
$\Gamma \vdash e_2' : \tau_1$	Lemma A.6
$\Gamma \vdash E[e_2'] : \tau$	Lemma A.5

□

Lemma A.3. (Base Progress) If $\cdot \vdash e_1 : \tau$ then e_1 is a value $e_1 \longrightarrow e_2$ for some e_2.

Proof of Lemma A.3. By induction on the derivation of $\cdot \vdash e_1 : \tau$.

- Case T-Var. Impossible, the context is empty.
- Case T-App, $e_1 = e_3\, e_4$. By the I.H. e_3 is a value or steps, and then e_4 is a value or steps. If one of them steps then it steps, else if both are values:

$\cdot \vdash e_3 : \tau_2 \to \tau$	`T-App`
$e_3 = \lambda x : \tau_2 . e_5$	Lemma A.4

$e_3\ e_4 \longrightarrow e_5[x := e_4]$ S-App

- Other cases are trivial: the expression is a value, or it takes a step by the matching rule of $\dashrightarrow$ (using Lemma A.4 for the shape of values).

□

Lemma A.4. (Canonical forms) If $\Gamma \vdash v : \tau$ then:
- If $\tau = \mathsf{num}$ then $v = n$.
- If $\tau = \mathsf{bool}$ then $v = b$.
- If $\tau = \mathsf{unit}$ then $v = \mathsf{unit}$.
- If $\tau = \tau_1 \to \tau_2$ then $v = \lambda x : \tau_1.e$.
- If $\tau = \langle \tau_1, \tau_2 \rangle$ then $v = \langle v_1, v_2 \rangle$ with $\Gamma \vdash v_1 : \tau_1$ and $\Gamma \vdash v_2 : \tau_2$.

Proof. By type rules T-Num, T-Bool, T-Unit, T-Lambda, T-Tuple. □

Lemma A.5. (Decomposition) If $\Gamma \vdash E[e] : \tau$ then there is τ_1 such that $\Gamma \vdash e : \tau_1$, and for every e' with $\Gamma \vdash e' : \tau_1$ we have $\Gamma \vdash E[e'] : \tau$.

Proof. By induction on E.

- Case $E = \square$. Take $\tau_1 = \tau$. The premise is $\Gamma \vdash e : \tau$, and the conclusion follows immediately.
- Case $E = E_0\ e_0$.

  $\Gamma \vdash E_0[e]\ e_0 : \tau$ Given
  $\Gamma \vdash E_0[e] : \tau_2 \to \tau$ T-App
  $\Gamma \vdash e_0 : \tau_2$ T-App
  $\Gamma \vdash e : \tau_1$ I.H.
  $\Gamma \vdash E_0[e'] : \tau_2 \to \tau$ I.H.
  $\Gamma \vdash E_0[e']\ e_0 : \tau$ T-App

- Other cases are trivial: invert the typing rule of the frame, apply the I.H. to E_0 under the same Γ, and apply the same typing rule again.

□

Lemma A.6. (Redex preservation) If $\Gamma \vdash e_1 : \tau$ and $e_1 \dashrightarrow e_2$ then $\Gamma \vdash e_2 : \tau$.

Proof. By cases on $e_1 \dashrightarrow e_2$.

- Case S-App, $(\lambda x : \tau_1.e)\ v \dashrightarrow e[x := v]$.

  $\Gamma \vdash \lambda x : \tau_1.e : \tau_1 \to \tau$ T-App
  $\Gamma \vdash v : \tau_1$ T-App
  $\Gamma, x : \tau_1 \vdash e : \tau$ T-Lambda
  $\Gamma \vdash e[x := v] : \tau$ Lemma A.7

- Case S-Fix, $\mu x : \tau . e \dashrightarrow e[x := \mu x : \tau . e]$.

  $\Gamma, x : \tau \vdash e : \tau$ — `T-Fix`
  $\Gamma \vdash \mu x : \tau . e : \tau$ — Given
  $\Gamma \vdash e[x := \mu x : \tau . e] : \tau$ — Lemma A.7

- Other cases are trivial.

□

Lemma A.7. (Substitution) If $\Gamma, x : \tau_1 \vdash e : \tau$ and $\Gamma \vdash e_1 : \tau_1$ then $\Gamma \vdash e[x := e_1] : \tau$.

Proof. By induction on the derivation of $\Gamma, x : \tau_1 \vdash e : \tau$.

- Case T-Var with $e = x$. Then $\Gamma, x : \tau_1 \vdash x : \tau_1$ and $\tau = \tau_1$.

  $\Gamma \vdash e_1 : \tau_1$ — Given
  $x[x := e_1] = e_1$ — Substitution
  $\Gamma \vdash x[x := e_1] : \tau$

- Case T-Var with $e = y \neq x$. Then $\Gamma, x : \tau_1 \vdash y : \tau$

  $y : \tau \in (\Gamma, x : \tau_1)$ — `T-Var`
  $y : \tau \in \Gamma$ — $y \neq x$
  $\Gamma \vdash y[x := e_1] : \tau$ — `T-Var`, $y[x := e_1] = y$

- Case T-Lambda with $e = \lambda y : \tau_2 . e_2$, $y \neq x$ and $y \notin \text{fv}(e_1)$.

  $\Gamma, x : \tau_1, y : \tau_2 \vdash e_2 : \tau_3$ — `T-Lambda`
  $\Gamma, y : \tau_2, x : \tau_1 \vdash e_2 : \tau_3$ — Unordered Context
  $\Gamma, y : \tau_2 \vdash e_2[x := e_1] : \tau_3$ — I.H.
  $\Gamma \vdash \lambda y : \tau_2 . e_2[x := e_1] : \tau_2 \to \tau_3$ — `T-Lambda`

- Case T-Fix with $e = \mu y : \tau . e_2$. Same as T-Lambda.

- Case T-App with $e = e_2\ e_3$.

  $\Gamma, x : \tau_1 \vdash e_2 : \tau_2 \to \tau$ — `T-App`
  $\Gamma, x : \tau_1 \vdash e_3 : \tau_2$ — `T-App`
  $\Gamma \vdash e_2[x := e_1] : \tau_2 \to \tau$ — I.H.
  $\Gamma \vdash e_3[x := e_1] : \tau_2$ — I.H.
  $\Gamma \vdash (e_2\ e_3)[x := e_1] : \tau$ — `T-App`

- Other cases are trivial: the expression has no variables, or the I.H. applies to every premise and the same rule is used again.

□

CHAPTER B

Contracts Safety Theorems

Theorem B.1. (Contracts Type Safety)

1. If $\cdot \vdash e : \tau$ and $e \twoheadrightarrow e'$ then $\cdot \vdash e' : \tau$.
2. If $\cdot \vdash e : \tau$ then e is a value or $e = \mathsf{blame}_j^k$ or $e \twoheadrightarrow e'$ for some e'.

Proof. By Lemma B.2 and Lemma B.3. □

Lemma B.2. (Contracts Preservation) If $\Gamma \vdash e_1 : \tau$ and $e_1 \twoheadrightarrow e_2$ then $\Gamma \vdash e_2 : \tau$.

Proof of Lemma B.2. By cases on the step.

- Case Step-C, $E[e_1'] \twoheadrightarrow E[e_2']$ with $e_1' \dashrightarrow e_2'$.

  $\Gamma \vdash e_1' : \tau_1$ — Lemma B.7
  $\Gamma \vdash e_2' : \tau_1$ — Lemma B.8
  $\Gamma \vdash E[e_2'] : \tau$ — Lemma B.7

- Case Step-C-Blame, $E\left[\mathsf{blame}_j^k\right] \twoheadrightarrow \mathsf{blame}_j^k$.

  $\Gamma \vdash \mathsf{blame}_j^k : \tau$ — `Type-C-Blame`

□

Lemma B.3. (Contracts Progress)

If $\Gamma \vdash e_1 : \tau$ then e_1 is a value, or $e_1 = \mathsf{blame}_j^k$, or there is e_2 with $e_1 \twoheadrightarrow e_2$.

Proof of Lemma B.3. By induction on the derivation of $\Gamma \vdash e_1 : \tau$.

During induction, we encounter case $e' = \mathsf{blame}_{j'}^{k'}$. This makes $e = E\left[\mathsf{blame}_{j'}^{k'}\right]$ and would take a step. In the following proof, we left this case (blame) out.

- Cases of Base.

  Similarly to Lemma A.3, except for T-App. Function type by Lemma B.4, introduces two new cases:

  - $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)$. Steps by Guard-Func.
  - $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)$. Steps by Guard-Dep.

- Case Type-C-Blame. $e_1 = \mathsf{blame}_j^k$.
- Case Type-C-Guard. e_1 is a value.

- Case Type-C-Check, $e_1 = \mathsf{check}_j^k(e, v)$.

  $\Gamma \vdash e : \mathsf{bool}$ — Type-C-Check
  $e = \mathsf{true} \mid \mathsf{false}$ — Lemma B.4
  $e_1 \twoheadrightarrow e_1'$ — Check-True or Check-False

- Case Type-C-Mon, $e_1 = \mathsf{mon}_j^{k,l}(\kappa, v)$

  $\Gamma \vdash \kappa : \mathsf{con}(\tau)$ — Type-C-Mon

  Subcase on type of conract:

  - Type-C-Flat. Steps by Mon-Flat.
  - Type-C-Fun or Type-C-DepFun. Steps by Mon-Guard.

□

Lemma B.4. (Canonical forms) If $\Gamma \vdash v : \tau$ then either:

- If $\tau = \mathsf{bool}$ then $v = \mathsf{true}$ or $v = \mathsf{false}$
- If $\tau = \tau_1 \to \tau_2$ then $v = \lambda x : \tau_1.e$ or $v = \mathsf{guard}_j^{k,l}(\kappa, v')$ where $\kappa = \kappa_1 \to \kappa_2$ or $\kappa = \kappa_1 \to_i \lambda y.\kappa_2$

Proof. Trivial by type inversion. □

Lemma B.5. (Substitution Expression)

If $\Gamma \vdash e_1 : \tau_1$ and $\Gamma, x : \tau_1 \vdash e : \tau$ then $\Gamma \vdash e[x := e_1] : \tau$.

Proof. By induction on $\Gamma, x : \tau_1 \vdash e : \tau$. The cases of Base are those of Lemma A.7.

- Case $\Gamma, x : \tau_1 \vdash \mathsf{mon}_j^{k,l}(\kappa, e_2) : \tau$

  $\Gamma, x : \tau_1 \vdash \kappa : \mathsf{con}(\tau)$ — `Type-C-Mon`
  $\Gamma, x : \tau_1 \vdash e_2 : \tau$ — `Type-C-Mon`
  $\Gamma \vdash \kappa[x := e_1] : \mathsf{con}(\tau)$ — Lemma B.6
  $\Gamma \vdash e[x := e_1] : \tau$ — I.H.
  $\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa, e)[x := e_1] : \tau$ — `Type-C-Mon`

- Cases Type-C-Check and Type-C-Guard.

  Similarly to the above, using I.H..

- Case $\Gamma, x : \tau_1 \vdash \mathsf{blame}_j^k : \tau$

  $\mathsf{blame}_j^k[x := e_1] = \mathsf{blame}_j^k$ — Substitution
  $\Gamma \vdash \mathsf{blame}_j^k[x := e_1] : \tau$

□

Lemma B.6. (Substitution Contract)

If $\Gamma \vdash \kappa : \tau_1$ and $\Gamma, x : \tau_1 \vdash \kappa : \mathsf{con}(\tau)$ then $\Gamma \vdash \kappa[x := e_1] : \mathsf{con}(\tau)$.

Proof.

- Case $\Gamma, x : \tau_1 \vdash \mathsf{flat}(e) : \mathsf{con}(\tau)$

  $\Gamma, x : \tau_1 \vdash e : \tau \to \mathsf{bool}$ — `Type-C-Flat`
  $\Gamma \vdash e[x := e_1] : \tau \to \mathsf{bool}$ — Lemma B.5
  $\Gamma \vdash \mathsf{flat}(e)[x := e_1] : \mathsf{con}(\tau)$ — `Type-C-Flat`

- Case Type-C-Fun and Type-C-Tuple.

  Similarly to the above.

- Case $\Gamma, x : \tau_1 \vdash \kappa_1 \to_i \lambda y.\kappa_2 : \mathsf{con}(\tau_2 \to \tau_3)$, $y \neq x$ and $y \notin \mathrm{fv}(e_1)$.

  $\Gamma, x : \tau_1 \vdash \kappa_1 : \mathsf{con}(\tau_2)$ — `Type-C-DepFun`
  $\Gamma, x : \tau_1, y : \tau_2 \vdash \kappa_2 : \mathsf{con}(\tau_3)$ — `Type-C-DepFun`
  $\Gamma \vdash \kappa_1[x := e_1] : \mathsf{con}(\tau_2)$ — I.H.
  $\Gamma, y : \tau_2 \vdash e_1 : \tau_1$ — Weakening context
  $\Gamma, y : \tau_2 \vdash \kappa_2[x := e_1] : \mathsf{con}(\tau_3)$ — I.H.
  $\Gamma \vdash \kappa[x := e_1] : \mathsf{con}(\tau_2 \to \tau_3)$ — `Type-C-DepFun`

□

Lemma B.7. (Decomposition) If $\Gamma \vdash E[e] : \tau$ then there is τ_1 such that $\Gamma \vdash e : \tau_1$, and for every e' with $\Gamma \vdash e' : \tau_1$ we have $\Gamma \vdash E[e'] : \tau$.

Proof. By induction on E. The frames of Base are as in Lemma A.5. Two frames are new.

- Case $E = \mathsf{mon}^{k,l}_j(\kappa, E_0)$.

  $\Gamma \vdash \mathsf{mon}^{k,l}_j(\kappa, E_0[e]) : \tau$ — Given
  $\Gamma \vdash \kappa : \mathsf{con}(\tau)$ — `Type-C-Mon`
  $\Gamma \vdash E_0[e] : \tau$ — `Type-C-Mon`
  $\Gamma \vdash e : \tau_1$ — I.H.
  $\Gamma \vdash E_0[e'] : \tau$ — I.H., for $\Gamma \vdash e' : \tau_1$
  $\Gamma \vdash \mathsf{mon}^{k,l}_j(\kappa, E_0[e']) : \tau$ — `Type-C-Mon`

- Case $E = \mathsf{check}^k_j(E_0, v)$.

  $\Gamma \vdash \mathsf{check}^k_j(E_0[e], v) : \tau$ — Given
  $\Gamma \vdash E_0[e] : \mathsf{bool}$ — `Type-C-Check`
  $\Gamma \vdash v : \tau$ — `Type-C-Check`
  $\Gamma \vdash e : \tau_1$ — I.H.
  $\Gamma \vdash E_0[e'] : \mathsf{bool}$ — I.H., for $\Gamma \vdash e' : \tau_1$

$\Gamma \vdash \mathsf{check}_j^k(E_0[e'], v) : \tau$ Type-C-Check

□

Lemma B.8. (Redex preservation) If $\Gamma \vdash e_1 : \tau$ and $e_1 \dashrightarrow e_2$ then $\Gamma \vdash e_2 : \tau$.

Proof. By cases on $e_1 \dashrightarrow e_2$. The reductions of Base are as in Lemma A.6, using Lemma B.5 for S-App and S-Fix.

- Case $\mathsf{check}_j^k(\mathsf{true}, v) \dashrightarrow v$.

  $\Gamma \vdash v : \tau$ Type-C-Check

- Case $\mathsf{check}_j^k(\mathsf{false}, v) \dashrightarrow \mathsf{blame}_j^k$.

  $\Gamma \vdash \mathsf{blame}_j^k : \tau$ Type-C-Blame

- Case $\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}_j^k(e\ v, v)$.

  $\Gamma \vdash \mathsf{flat}(e) : \mathsf{con}(\tau)$ Type-C-Mon
  $\Gamma \vdash v : \tau$ Type-C-Mon
  $\Gamma \vdash e : \tau \to \mathsf{bool}$ Type-C-Flat
  $\Gamma \vdash e\ v : \mathsf{bool}$ T-App
  $\Gamma \vdash \mathsf{check}_j^k(e\ v, v) : \tau$ Type-C-Check

- Case $\mathsf{mon}_j^{k,l}(\kappa, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa, v)$ with κ a function contract.

  $\Gamma \vdash \kappa : \mathsf{con}(\tau)$ Type-C-Mon
  $\Gamma \vdash v : \tau$ Type-C-Mon
  $\Gamma \vdash \mathsf{guard}_j^{k,l}(\kappa, v) : \tau$ Type-C-Guard

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$.

  $\Gamma \vdash v_2 : \tau_1$ T-App
  $\Gamma \vdash \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1) : \tau_1 \to \tau$ T-App
  $\Gamma \vdash \kappa_1 \to \kappa_2 : \mathsf{con}(\tau_1 \to \tau)$ Type-C-Guard
  $\Gamma \vdash v_1 : \tau_1 \to \tau$ Type-C-Guard
  $\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1)$ Type-C-Fun
  $\Gamma \vdash \kappa_2 : \mathsf{con}(\tau)$ Type-C-Fun
  $\Gamma \vdash \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau_1$ Type-C-Mon
  $\Gamma \vdash v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau$ T-App
  $\Gamma \vdash \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) : \tau$ Type-C-Mon

- Case Guard-Dep, $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$ with $\kappa_3 = \kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$.

  $\Gamma \vdash v_2 : \tau_1$ T-App

$\Gamma \vdash v_1 : \tau_1 \to \tau$ Type-C-Guard

$\Gamma \vdash \kappa_1 \to_i \lambda x.\kappa_2 : \mathsf{con}(\tau_1 \to \tau)$ Type-C-Guard

$\Gamma \vdash \kappa_1 : \mathsf{con}(\tau_1)$ Type-C-DepFun

$\Gamma, x : \tau_1 \vdash \kappa_2 : \mathsf{con}(\tau)$ Type-C-DepFun

$\Gamma \vdash \mathsf{mon}_j^{l,j}(\kappa_1, v_2) : \tau_1$ Type-C-Mon

$\Gamma \vdash \kappa_3 : \mathsf{con}(\tau)$ Lemma B.6

$\Gamma \vdash \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau_1$ Type-C-Mon

$\Gamma \vdash v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2) : \tau$ T-App

$\Gamma \vdash \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) : \tau$ Type-C-Mon

□

CHAPTER C

Contracts Erasure Theorem

Define erasure compiler E as a homomorphic function over expressions of Contracts with a single transformation:

$\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa, e) : \tau \rightarrowtail e$

C.1 Relation

Define Contracts F as contexts without $\mathsf{check}_j^k(E, v)$.

Define these relations:

- $R_e \subseteq$ Contracts $e \times$ Base e

  If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We define $\tilde{e}$ as shorthand for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

- $R_E \subseteq$ Contracts $F \times$ Base E

  If $(E, E') \in R_E$, we write $E \sim_{R_E} E'$ or $E \sim E'$. We define $\tilde{E}$ as shorthand for any context $\tilde{E}$ satisfying $E \sim \tilde{E}$.

- $R_v \subseteq$ Contracts $v \times$ Base v

  If $(v, v') \in R_v$, we write $v \sim_{R_v} v'$ or $v \sim v'$. We define $\tilde{v}$ as shorthand for any variable $\tilde{v}$ satisfying $v \sim \tilde{v}$.

We define context F that is contexts without $\mathsf{check}_j^k(E, v)$.

$\boxed{\sim \subseteq \text{Contracts } F \times \text{Base } E}$

Define $\tilde{F}$ such that $F \sim \tilde{F}$.

$$\begin{aligned} \square &\sim \square \\ F\ e &\sim \tilde{F}\ \tilde{e} \\ v\ F &\sim \tilde{v}\ \tilde{F} \\ \mathsf{mon}_j^{k,l}(\kappa, F) &\sim \tilde{F} \end{aligned}$$

$\sim \subseteq$ Contracts $e \times$ Base e

$$
\begin{array}{rcl}
v & \sim & \tilde{v} \\
e_1\ e_2 & \sim & \tilde{e}_1\ \tilde{e}_2 \\
\mathsf{mon}_j^{k,l}(\kappa, e_2) & \sim & \tilde{e}_2 \\
\mathsf{check}_j^{k}(e_1, v) & \sim & \tilde{v}
\end{array}
$$

$\sim \subseteq$ Contracts $v \times$ Base v

$$
\begin{array}{rcl}
n & \sim & n \\
b & \sim & b \\
x & \sim & x \\
\lambda x.e & \sim & \lambda x.\tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa, v) & \sim & \tilde{v}
\end{array}
$$

Lemma C.1. (Relation is closed under check-free contexts)

If $e \sim \tilde{e}$ and $F \sim \tilde{F}$, then $F[e] \sim \tilde{F}[\tilde{e}]$.

Proof.

By induction on $F \sim \tilde{F}$.

- Case $\square \sim \square$

$e \sim \tilde{e}$	Given
$F[e] = e$	Context Plug in
$\tilde{F}[\tilde{e}] = \tilde{e}$	Context Plug in
$F[e] \sim \tilde{F}[\tilde{e}]$	

- Case $F_1\ e_2 \sim \tilde{F}_1\ e_2$

$e \sim \tilde{e}$	Given
$F_1[e] \sim \tilde{F}_1[\tilde{e}]$	I.H.
$F[e] = F_1[e]\ e_2$	Context Plug in
$\tilde{F}[\tilde{e}] = \tilde{F}_1[\tilde{e}]\ \tilde{e}_2$	Context Plug in
$F[e] \sim \tilde{F}[\tilde{e}]$	definition

- Similarly for other cases

□

Lemma C.2. (Relation Decomposition Through Contexts)

If $e_1 \sim e_1'$, $e_1 = E[e_2]$, and e_2 is a redex, then there are $\tilde{E}$ and $\tilde{e}_2$ such that $E \sim \tilde{E}$ and $e_2 \sim \tilde{e}_2$ and $e_1' = \tilde{E}[\tilde{e}_2]$.

Proof. By induction on the check-free context E.

- Case $E = \square$

$e_1 = e_2$	Context Plug in
$\tilde{E} = \square$	Choose
$\tilde{e}_2 = e_1'$	Choose
$E \sim \tilde{E}$	Definition
$e_2 \sim \tilde{e}_2$	Given
$e_1' = \tilde{E}[\tilde{e}_2]$	Context Plug in

- Case $E = E_0\ e_0$

$e_1 = E_0[e_2]\ e_0 \sim e_1'$	Given
$E_0[e_2] \sim e_0'$	Relation inversion
$e_0 \sim \tilde{e}_0$	Relation inversion
$e_1' = e_0'\ \tilde{e}_0$	Relation inversion
$E_0 \sim \tilde{E}_0$	I.H.
$e_2 \sim \tilde{e}_2$	I.H.
$e_0' = \tilde{E}_0[\tilde{e}_2]$	I.H.
$\tilde{E} = \tilde{E}_0\ \tilde{e}_0$	Choose
$E \sim \tilde{E}$	Definition
$e_1' = \tilde{E}[\tilde{e}_2]$	Context Plug in

- Case $E = v\ E_0$

$e_1 = v\ E_0[e_2] \sim e_1'$	Given
$v \sim \tilde{v}$	Relation inversion
$E_0[e_2] \sim e_0'$	Relation inversion
$e_1' = \tilde{v}\ e_0'$	Relation inversion
$E_0 \sim \tilde{E}_0$	I.H.
$e_2 \sim \tilde{e}_2$	I.H.
$e_0' = \tilde{E}_0[\tilde{e}_2]$	I.H.
$\tilde{E} = \tilde{v}\ \tilde{E}_0$	Choose
$E \sim \tilde{E}$	Definition
$e_1' = \tilde{E}[\tilde{e}_2]$	Context Plug in

- Case $E = \mathsf{mon}_j^{k,l}(\kappa, E_0)$

$e_1 = \mathsf{mon}_j^{k,l}(\kappa, E_0[e_2]) \sim e_1'$	Given
$E_0[e_2] \sim e_1'$	Relation inversion
$E_0 \sim \tilde{E}_0$	I.H.
$e_2 \sim \tilde{e}_2$	I.H.
$e_1' = \tilde{E}_0[\tilde{e}_2]$	I.H.
$\tilde{E} = \tilde{E}_0$	Choose
$E \sim \tilde{E}$	Definition
$e_1' = \tilde{E}[\tilde{e}_2]$	Context Plug in

- Other check-free context cases follow similarly.

□

Lemma C.3. (Relation is closed under substitution)

If $e \sim \tilde{e}$ and $v \sim \tilde{v}$, then $e[x := v] \sim \tilde{e}[x := \tilde{v}]$ for every x.

Proof.

By I.H. on $e \sim \tilde{e}$.

- Case $x \sim x$

$v \sim \tilde{v}$	Given
$e[x := v] = v$	Replacement
$\tilde{e}[x := \tilde{v}] = \tilde{v}$	Replacement
$e[x := v] \sim \tilde{e}[x := \tilde{v}]$	By $v \sim \tilde{v}$

- Case $y \sim y$, where $y \neq x$

$v \sim \tilde{v}$	Given
$e[x := v] = y$	Replacement
$\tilde{e}[x := \tilde{v}] = y$	Replacement
$e[x := v] \sim \tilde{e}[x := \tilde{v}]$	By $y \sim y$

- Case $e_1\ e_2 \sim \tilde{e}_1\ \tilde{e}_2$

$e_1[x := v] \sim \tilde{e}_1[x := \tilde{v}]$	I.H.
$e_2[x := v] \sim \tilde{e}_2[x := \tilde{v}]$	I.H.
$e[x := v] = e_1[x := v]\ e_2[x := v]$	Replacement
$\tilde{e}[x := v] = \tilde{e}_1[x := \tilde{v}]\ \tilde{e}_2[x := \tilde{v}]$	Replacement
$e[x := v] \sim \tilde{e}[x := \tilde{v}]$	Definition

- Similarly with other cases. Lambda case avoid same variable by replacement $e[x := v]$ to free variables of e.

□

Lemma C.4. (Evaluation-context decomposition)

Forall context E, either:

- $E = F$ or
- $E = F\left[\mathsf{check}^k_j(E', v)\right]$ (get the first check)

Proof. By induction on E.

- Case $\square$

  $F = \square$ Choose
  $E = F$

- Case $E = \mathsf{check}^k_j(E_1, v)$

  $F = \square$ Choose
  $E' = E_1$ Choose
  $E = F\left[\mathsf{check}^k_j(E_1, v)\right]$

- Case $E_1\ e$

  I.H. on E_1:

  - Case $E_1 = F_1$

    $E = F_1\ e$
    $F = F_1\ e$ Choose
    $E = F$

  - Case $E_1 = F_1\left[\mathsf{check}^k_j(E_1', v)\right]$

    $E = F_1\left[\mathsf{check}^k_j(E_1', v)\right]\ v$
    $F = F_1\ v$ Choose
    $E' = E_1'$ Choose
    $E = F\left[\mathsf{check}^k_j(E_1', v)\right]$

- Other cases follows similarly

□

We define the predicate that will be used for simulation. The beauty of the predicate $\mathrm{ER}(e, e')$ is that $e = E\left[\mathsf{blame}^k_j\right]$ returns false, so we can skip this case in simulation proof.

$$\mathrm{ER}(e, e') = \begin{cases} \text{true if } e{\sim}e' \text{ and } e{\neq}F\left[\mathsf{check}^k_j(e_0, v)\right] \\ \text{true if } e{=}F\left[\mathsf{check}^k_j(e_0, v)\right] \text{ and } e'{=}\tilde{F}[\tilde{v}] \\ \text{false otherwise} \end{cases}$$

Lemma C.5. (Erasure satisfies the state predicate)

If $e' = \mathrm{erase}(e)$, then $\mathrm{ER}(e, e')$.

Proof. I.H on $e' = \mathrm{erase}(e)$.

Because the compiler doesn't compile check (runtime expression), we never get the second clause of EQ. Therefore, $e \neq F\left[\mathsf{check}_j^k(e_0, v)\right]$, and all child nodes of e also not this form.

- Case $e' = \mathrm{erase}\left(\mathsf{mon}_j^{k,l}(\kappa, e_1)\right) = \mathrm{erase}(e_1) = e_1'$

  $\mathrm{ER}(e_1, e_1')$ — I.H.
  $e_1 \sim e_1'$ — First clause EQ
  $\mathsf{mon}_j^{k,l}(\kappa, e_1) \sim \tilde{e}_1$ — Definition
  $e_1 \sim \tilde{e}_1$ — Definition
  $\tilde{e}_1 = e_1'$ — Choose
  $\mathsf{mon}_j^{k,l}(\kappa, e_1) \sim e_1'$
  $\mathrm{ER}(e, e')$ — First clause EQ

- Case $e' = \mathrm{erase}(e_1\ e_2) = \mathrm{erase}(e_1)\ \mathrm{erase}(e_2) = e_1'\ e_2'$

  $\mathrm{ER}(e_1, e_1')$ — I.H.
  $\mathrm{ER}(e_2, e_2')$ — I.H.
  $e_1 \sim e_1'$ — First clause EQ
  $e_2 \sim e_2'$ — First clause EQ
  $e_1\ e_2 \sim \tilde{e}_1\ \tilde{e}_2$ — Definition
  $\mathrm{ER}(e, e')$ — First clause EQ

- Other cases follow similarly to the above case.

□

C.2 Main Theorem Proof

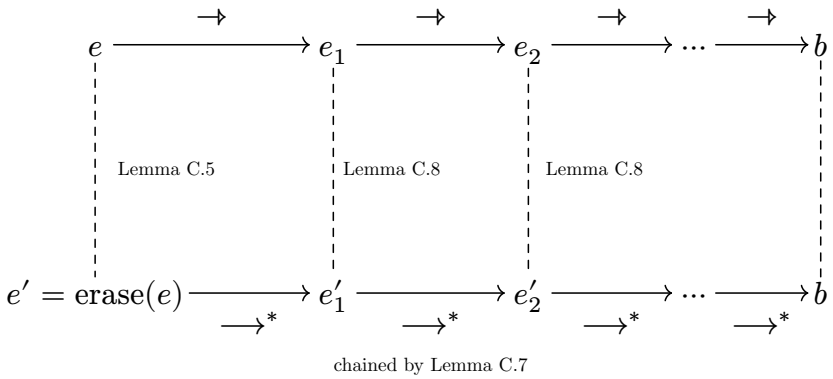


Figure 17: Structure of the contract erasure proof. Dashed vertical lines denote ER.

Theorem C.6. (Contract Erasure Theorem)

If $e' = \text{erase}(e)$ and $e \twoheadrightarrow^* b$, then $e' \longrightarrow^* b$.

Proof.

$\text{ER}(e, e')$	Lemma C.5
$e' \longrightarrow^* b$	Lemma C.7

□

Lemma C.7. (Erasure Multistep Simulation)

If $\text{ER}(e, e')$ and $e \twoheadrightarrow^* b$ then $e' \longrightarrow^* b$.

$\{(e, e') \mid e \twoheadrightarrow^* b \text{ and } (e, e') \in \text{EQ}\}$. This active filter prevents $e \twoheadrightarrow^* \mathsf{blame}_j^k$ as well as anything not b. Mainly we worry about $E\left[\mathsf{blame}_j^k\right] \to \mathsf{blame}_j^k$ and $E\left[\mathsf{check}_j^k(\mathsf{false}, v)\right] \to E\left[\mathsf{blame}_j^k\right]$. It's a strong claim here that $e \twoheadrightarrow^* b$ so we can ignore these states. But the single step simulation must be reworded correctly.

Now if we check the argument for EQ, we can actually filter out $E\left[\mathsf{blame}_j^k\right]$ because it doesn't fit two clauses. So we only care about the check expression.

Proof.

By induction on $e \twoheadrightarrow^* b$.

- Case $b \twoheadrightarrow^* b$

$\text{ER}(e, e')$	Given
$e \sim e'$	First clause EQ
$e' = b$	Relation inversion $b \sim e'$
$e' \longrightarrow^* b$	Reflexivity

- Case $e \to e_1$ and $e_1 \twoheadrightarrow^* b$

  By cases on $e \to e_1$.

  - Case $e = E\left[\mathsf{blame}_j^k\right] \to \mathsf{blame}_j^k = e_1$ for some $E \neq \square, k, j$

$e_1 \twoheadrightarrow^* \mathsf{blame}_j^k$	Reflexivity
$e_1 \twoheadrightarrow^* b$	Given
$\bot$	Contradiction

  - Case $e = E[e_2] \to E[e_3] = e_1$ where $e_2 \dashrightarrow e_3$ is a redex, and $e_2 \neq \mathsf{blame}_j^k$.

$\text{ER}(e, e')$	Given
$e \to e_1$	Given
$e \neq E\left[\mathsf{blame}_j^k\right]$	Case assumption

– Case $e_2 = \mathsf{check}^k_j(\mathsf{false}, v)$, and $e_3 = \mathsf{blame}^k_j$

$e_1 \twoheadrightarrow \mathsf{blame}^k_j$	Step-C
$e_1 \twoheadrightarrow^* \mathsf{blame}^k_j$	
$e_1 \twoheadrightarrow^* b$	Given
$\bot$	Contradiction

– Other cases, $e_2 \neq \mathsf{check}^k_j(\mathsf{false}, v)$, and $e_3 \neq \mathsf{blame}^k_j$

Relation is not local so we run simulation on $e \twoheadrightarrow e_1$.

$e_1 \neq E\left[\mathsf{blame}^k_j\right]$	
$e' \longrightarrow^* e'_1$	Lemma C.8
$\mathrm{ER}(e_1, e'_1)$	Lemma C.8
$e'_1 \longrightarrow^* b$	I.H.
$e' \longrightarrow^* b$	Transitive

□

Lemma C.8. (Erasure Step Simulation)

If $\mathrm{ER}(e_1, e'_1)$, $e_1 \twoheadrightarrow e_2$, and $e_2 \neq E_2\left[\mathsf{blame}^k_j\right]$, then there is e'_2 such that $e'_1 \longrightarrow^* e'_2$ and $\mathrm{ER}(e_2, e'_2)$.

Proof.

By case analysis of $\mathrm{ER}(e_1, e'_1)$:

- Case $e_1 = F\left[\mathsf{check}^{k_1}_{j_1}(e_{1a}, v)\right]$ and $e'_1 = \tilde{F}[\tilde{v}]$
  - Case $e_{1a} = \mathsf{true}$

$e_1 \twoheadrightarrow F[v]$	Step-C
$e_2 = F[v]$	
$e_2 \sim \tilde{F}[\tilde{v}]$	Definition
$e'_1 \longrightarrow^* \tilde{F}[\tilde{v}]$	Reflexivity
$e'_2 = \tilde{F}[\tilde{v}]$	Choose

  - Case $e_{1a} = \mathsf{false}$
  - Case $e_{1a} \dashrightarrow e_{1b}$

$e_1 \twoheadrightarrow F\left[\mathsf{check}^{k_1}_{j_1}(e_{1b}, v)\right]$	Step-C
$e_2 = F\left[\mathsf{check}^{k_1}_{j_1}(e_{1b}, v)\right]$	
$e_2 \sim \tilde{F}[\tilde{v}]$	Definition
$e'_1 \longrightarrow^* \tilde{F}[\tilde{v}]$	Reflexivity
$e'_2 = \tilde{F}[\tilde{v}]$	Choose

- Case $e_1 \sim e_1'$ and $e_1 \neq F\left[\mathsf{check}_{j_1}^{k_1}(e_{1a}, v)\right]$

  Casing on $e_1 \to e_2$:

  ▸ Case $E\left[\mathsf{blame}_{j_2}^{k_2}\right] \to \mathsf{blame}_{j_2}^{k_2}$ contradiction.

  ▸ Case $E[e_3] \to E[e_4]$ where $e_3 \dashrightarrow e_4$ is a redex.

  By Lemma C.4,

  – Case $E = F\left[\mathsf{check}_{j_3}^{k_3}(E_3, v_3)\right]$

  $e_1 = F\left[\mathsf{check}_{j_3}^{k_3}(E_3[e_3], v_3)\right]$ contradiction.

  – Case $E = F$ with F a non-check context. Without confusion, we use E below:

  For all cases, we have $E \sim \tilde{E}, e_3 \sim \tilde{e}_3, e_4 \sim \tilde{e}_4$ by Lemma C.2.

  By cases on the redex reduction $e_3 \dashrightarrow e_4$:

  - Case $(\lambda x.e_5)\ v_5 \dashrightarrow e_5[x := v_5]$

$e_3 \sim (\lambda x.\tilde{e}_5)\ \tilde{v}_5$	Definition
$\tilde{e}_3 = (\lambda x.\tilde{e}_5)\ \tilde{v}_5$	$e_3 \sim \tilde{e}_3$
$(\lambda x.\tilde{e}_5)\ \tilde{v}_5 \dashrightarrow \tilde{e}_5[x := \tilde{v}_5]$	`S-App`
$e_4 \sim \tilde{e}_5[x := \tilde{v}_5]$	Lemma C.3
$\tilde{e}_4 = \tilde{e}_5[x := \tilde{v}_5]$	Lemma C.3
$E[e_3] \sim \tilde{E}[\tilde{e}_3]$	Lemma C.1
$e_1' = \tilde{E}[\tilde{e}_3] \longrightarrow^* \tilde{E}[\tilde{e}_4] = e_2'$	Step and Closure
$e_2 = E[e_4] \sim \tilde{E}[\tilde{e}_4] = e_2'$	Lemma C.1

  - Case $\mu x : \tau.e_3 \dashrightarrow e_3[x := \mu x : \tau.e_3]$

  Similar to the preceding case.

  For below cases, we left out obvious rules about relation.

  - Case $\mathsf{check}_j^k(\mathsf{true}, v) \dashrightarrow v$

$\tilde{e}_3 = \tilde{v}$	Definition
$e_4 \sim \tilde{v}$	Definition
$e_1' \longrightarrow^* \tilde{E}[\tilde{v}] = e_2'$	Reflexivity
$e_2 \sim e_2'$	Lemma C.1

  - Case $\mathsf{check}_j^k(\mathsf{false}, v) \dashrightarrow \mathsf{blame}_j^k$

$e_2 = E\left[\mathsf{blame}_j^k\right]$	`C-Step`
$e_2 \neq E_2\left[\mathsf{blame}_{j_2}^{k_2}\right]$	Given
$\bot$	Contradiction

- Case $\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), v_3) \dashrightarrow \mathsf{check}_j^k(e_3\ v_3, v_3)$

  $\tilde{e}_3 = \tilde{v}_3$ Definition
  $e_4 \sim \tilde{v}_3$ Definition
  $e_1' \longrightarrow^* \tilde{E}[\tilde{v}_3] = e_2'$ Reflexivity
  $e_2 \sim e_2'$ Lemma C.1

- Case $\mathsf{mon}_j^{k,l}(\kappa, v_3) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa, v_3)$

  $\tilde{e}_3 = \tilde{v}_3$ Definition
  $e_4 \sim \tilde{v}_3$ Definition
  $e_1' \longrightarrow^* \tilde{E}[\tilde{v}_3] = e_2'$ Reflexivity
  $e_2 \sim e_2'$ Lemma C.1

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$

  $\tilde{e}_3 = \tilde{v}_1\ \tilde{v}_2$ Definition
  $\tilde{e}_4 \sim \tilde{v}_1\ \tilde{v}_2$ Definition
  $e_1' \longrightarrow^* \tilde{E}[\tilde{v}_1\ \tilde{v}_2] = e_2'$ Reflexivity
  $e_2 \sim e_2'$ Lemma C.1

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$, where $\kappa_3 = \kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$

  Similar to the preceding case.

- Other cases (Base) follows similarly.

□

CHAPTER D

Effects Safety Theorem

Follow the proofs of System F^{ε} in [18], our language extends a new type blab, and $\texttt{error}_p$ for migrating labels of Contracts to Effects. We also extend with shallow handler which is very similar to deep handler. We also add recursive function μ. These are the extra details that we must extend the proofs.

First we redefine some "operations" and rules that change when when adding the above.

- $\lceil E \rceil^l$ extracts all labels l from a context in reverse order:

  $\lceil F_0 \cdot \texttt{handle}_m(h_1, \cdot) \cdot F_1 \cdot ... \cdot \texttt{handle}_m(h_n, \cdot) \cdot F_n \rceil^l = \langle l_n, ..., l_1 \rangle$ iff $h_i : \Sigma(l_i)$

  Adapted to include both shallow and deep handle.

- Evaluation context typing

  Adapted to include both shallow and deep handle.

$$\frac{\Gamma \vdash_{\text{ops}} m : h : \sigma \mid l \mid \varepsilon \qquad \Gamma \vdash_{\text{ec}} E : \sigma_1 \to \sigma \mid \langle l, \varepsilon \rangle}{\Gamma \vdash_{\text{ec}} \texttt{handle}^{\varepsilon}_m(h, E) : \sigma_1 \to \sigma \mid \varepsilon} \text{CHANDLE}$$

D.1 Evaluation Context Typing Lemmas

Lemma D.1. (Evaluation Context Typing)

If $\cdot \vdash_{\text{ec}} E : \tau_1 \to \tau \mid \varepsilon$ and $\cdot \vdash e : \tau_1 \mid \langle \lceil E \rceil^l, \varepsilon \rangle$ then $\cdot \vdash E[e] : \tau \mid \varepsilon$

Lemma D.2. (Effect corresponds to the evaluation context)

If $\cdot \vdash E[e] : \tau \mid \varepsilon$ then there exists τ_1 such that $\cdot \vdash_{\text{ec}} E : \tau_1 \to \tau \mid \varepsilon$ and $\cdot \vdash e : \tau_1 \mid \langle \lceil E \rceil^l, \varepsilon \rangle$

Lemma D.3. (Well typed operations are handled)

If $\cdot \vdash E[\texttt{perform op } \overline{\tau}\, v] : \tau \mid \langle\rangle$ then E has the form $E_1 \cdot \texttt{handle}^{\varepsilon}_m(h, \cdot) \cdot E_2$ with $\texttt{op} \notin \texttt{bop}(E_2)$ and $\texttt{op} \to f \in h$.

Lemma D.4. (Effects types are meaningful)

If $\cdot \vdash E[\texttt{perform op}\ \overline{\tau}\ v] : \tau \mid \varepsilon$ with $\texttt{op} \notin \texttt{bop}(E)$, then $\texttt{op} \in \Sigma(l)$ and $l \in \varepsilon$, i.e. effect types cannot be discarded without a handler.

D.2 Type Safety Theorem

Theorem D.5. (`Effects` Progress)

If $\cdot \vdash e_1 : \tau \mid \langle\rangle$ then e_1 is a value or $e_1 = \texttt{error}_p$ or $e_1 \Longrightarrow e_2$.

Proof. By Lemma D.6, e_1 is a value, or $e_1 = \texttt{error}_p$, or $e_1 \Longrightarrow e_2$, or $e_1 = E[\texttt{perform op}\ \overline{\tau}\ v]$ with $\texttt{op} \notin \texttt{bop}(E)$ and $\texttt{op} \in \Sigma(l)$. The last case is impossible:

$\cdot \vdash E[\texttt{perform op}\ \overline{\tau}\ v] : \tau \mid \langle\rangle$	Given
$l \in \langle\rangle$	Lemma D.4
Contradiction	$\langle\rangle$ is empty

□

Lemma D.6. (`Effects` Progress with effects)

If $\cdot \vdash e_1 : \tau \mid \varepsilon$ then either

- e_1 is a value or
- $e_1 = \texttt{error}_p$ or
- $e_1 \Longrightarrow e_2$ or
- $e_1 = E[\texttt{perform op}\ \overline{\tau}\ v]$ where $\texttt{op} : \forall \overline{\alpha}.\tau_1 \to \tau_2 \in \Sigma(l)$, and $\texttt{op} \notin \texttt{bop}(E)$.

Proof. By induction on $\cdot \vdash e_1 : \tau \mid \varepsilon$

- Case $e_1 = v$, trivially true
- Case $e_1 = \texttt{error}_p$, trivially true
- Case $e_1 = e_3\ e_4$

$\cdot \vdash e_3\ e_4 : \tau \mid \varepsilon$	Given
$\cdot \vdash e_3 : \tau_1 \to^{\varepsilon} \tau \mid \varepsilon$	Given
$\cdot \vdash e_4 : \tau_1 \mid \varepsilon$	Given

  By I.H. on e_3:

  - Case $e_3 \Longrightarrow e_5$

    $$e_1 = e_3\ e_4 \Longrightarrow e_5\ e_4 = e_2$$

  - Case $e_3 = \texttt{error}_p$

    $$e_1 = \texttt{error}_p\ e_4 \Longrightarrow \texttt{error}_p = e_2$$

- Case $e_3 = E_0[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ v]$ where $\texttt{op} : \forall \overline{\alpha}.\tau_1 \to \tau_2 \in \Sigma(l)$, and $\texttt{op} \notin \texttt{bop}(E)$

  $e_1 = E_0[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ v]\ e_4 = E_1[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ v]$.

- Case e_3 is a value

  By I.H. on e_4.

  - Cases $e_4 \Longrightarrow e_5$, $e_4 = \texttt{error}_p$ makes e_1 steps.
  - Case e_4 is a value, by canonical forms of e_3, only some cases are valid:
    - Case $e_3 = \lambda x.e_6$. Then $e_1 = (\lambda x.e_6)\ e_4 \Longrightarrow e_6[x := e_4] = e_2$
    - Case $e_3 = \texttt{perform}\ \texttt{op}\ \overline{\tau}$. Then $e_1 = \square[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ e_4]$
    - Case $e_3 = \texttt{handler}^{\varepsilon}_m(h)$. Then $e_1 = \texttt{handler}^{\varepsilon}_m(h)\ e_4 \Longrightarrow \texttt{handle}^{\varepsilon}_m(h, e_4\ ()) = e_2$

- Case $e_1 = e_3\ [\tau_1]$

  $\cdot \vdash e_3\ [\tau_1] : \tau_2[\alpha := \tau_1] \mid \varepsilon$ Given

  $\cdot \vdash e_3 : \forall \alpha.\tau_2 \mid \varepsilon$

  By I.H. on e_3:

  - Case $e_3 \Longrightarrow e_5$

    $e_1 = e_3\ [\tau_1] \Longrightarrow e_5\ [\tau_1] = e_2$

  - Case $e_3 = \texttt{error}_p$

    $e_1 = \texttt{error}_p\ [\tau_1] \Longrightarrow \texttt{error}_p = e_2$

  - Case $e_3 = E_0[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ v]$ where $\texttt{op} : \forall \overline{\alpha}.\tau_1 \to \tau_2 \in \Sigma(l)$, and $\texttt{op} \notin \texttt{bop}(E)$

    $e_1 = E_0[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ v]\ [\tau_1] = E_1[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ v]$.

  - Case e_3 is a value

    By canonical form of value, $e_3 = \Lambda \alpha.e_4$

    $e_1 = (\Lambda \alpha.e_4)\ [\tau_1] \Longrightarrow e_4[\alpha := \tau_1] = e_2$

- Case $e_1 = \texttt{handle}^{\varepsilon}_m(h, e_3)$

  $\cdot \vdash \texttt{handle}^{\varepsilon}_m(h, e_3) : \tau \mid \varepsilon$ Given

  $\cdot \vdash e_3 : \tau \mid \langle l, \varepsilon \rangle$ Given

  By I.H. on e_3:

  - Case $e_3 \Longrightarrow e_4$

    $e_1 = \texttt{handle}^{\varepsilon}_m(h, e_3) \Longrightarrow \texttt{handle}^{\varepsilon}_m(h, e_4) = e_2$

  - Case $e_3 = \texttt{error}_p$

    $e_1 = \texttt{handle}^{\varepsilon}_m(h, \texttt{error}_p) \Longrightarrow \texttt{error}_p = e_2$

- Case $e_3 = E_0[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ v]$ where $\texttt{op} : \forall\overline{\alpha}.\tau_1 \to \tau_2 \in \Sigma(l)$, and $\texttt{op} \notin \texttt{bop}(E)$

  $e_1 = \texttt{handle}^{\varepsilon}_{m}(h, E_0[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ v])$.

  - Case $\texttt{op} \to f \in h$
    - Case $m = s$ shallow handler

      $e_1 \Longrightarrow f\ \overline{\tau}\ v\ (\lambda x.E_0[x]) = e_2$
    - Case $m = d$ deep handler

      $e_1 \Longrightarrow f\ \overline{\tau}\ v\ (\lambda x.\texttt{handle}^{\varepsilon}_{d}(h, E_0[x])) = e_2$
  - Case $\texttt{op} \to f \notin h$

    $e_1 = E_1[\texttt{perform}\ \texttt{op}\ \overline{\tau}\ v]$.
- Case e_3 is a value

  $e_1 = \texttt{handle}^{\varepsilon}_{m}(h, e_3) \Longrightarrow e_3 = e_2$
- Other cases follow similarly.

□

Theorem D.7. (`Effects` Preservation)

If $\cdot \vdash e_1 : \tau \mid \langle\rangle$ and $e_1 \Longrightarrow e_2$ then $\cdot \vdash e_2 : \tau \mid \langle\rangle$.

Proof. By cases on the step.

- Case $e_1 = E[e_3] \Longrightarrow E[e_4] = e_2$ and $e_3 \rightarrowtail e_4$.

$\cdot \vdash E[e_3] : \tau \mid \langle\rangle$	Given
$\cdot \vdash_{\text{ec}} E : \tau_1 \to \tau \mid \langle\rangle$	Lemma D.2
$\cdot \vdash e_3 : \tau_1 \mid \langle\lceil E\rceil^{l}, \varepsilon\rangle$	Lemma D.2
$\cdot \vdash e_4 : \tau_1 \mid \langle\lceil E\rceil^{l}, \varepsilon\rangle$	Lemma D.8
$\cdot \vdash E[e_4] : \tau \mid \langle\rangle$	Lemma D.1

- Case Step-E-Error, $E[\texttt{error}_p] \Longrightarrow \texttt{error}_p$.

$\cdot \vdash_{\text{val}} \texttt{error}_p : \tau$	T-E-Error
$\cdot \vdash \texttt{error}_p : \tau \mid \varepsilon$	T-E-Val

□

Lemma D.8. (Redex preservation)

If $\cdot \vdash e_1 : \tau \mid \varepsilon$ and $e_1 \rightarrowtail e_2$ then $\cdot \vdash e_2 : \tau \mid \varepsilon$.

Proof. By cases on $e_1 \rightarrowtail e_2$.

- Case $(\lambda^{\varepsilon} x : \tau_1.e)\ v \rightarrowtail e[x := v]$.

$\cdot \vdash \lambda^{\varepsilon} x : \tau_1 . e : \tau_1 \to^{\varepsilon} \tau \mid \varepsilon$ T-E-App
$\cdot \vdash v : \tau_1 \mid \varepsilon$ T-E-App
$x : \tau_1 \vdash e : \tau \mid \varepsilon$ T-E-Val, T-E-Abs
$\cdot \vdash_{\text{val}} v : \tau_1$ T-E-Val
$\cdot \vdash e[x := v] : \tau \mid \varepsilon$ Lemma D.9

- Case $(\Lambda \alpha^k . v)[\tau_0] \rightarrow v[\alpha := \tau_0]$

$\cdot \vdash (\Lambda \alpha^k . v)[\tau_0] : \tau_1[\alpha := \tau_0] \mid \varepsilon$ Given
$\cdot \vdash \Lambda \alpha^k . v : \forall \alpha^k . \tau_1 \mid \varepsilon$ T-E-Tapp
$\vdash_{\text{wf}} \tau_0 : k$ T-E-Tapp
$\cdot \vdash_{\text{val}} v : \tau_1$ T-E-Val, T-E-TAbs
$\cdot \vdash_{\text{val}} v[\alpha := \tau_0] : \tau_1[\alpha := \tau_0]$ Lemma D.10
$\cdot \vdash v[\alpha := \tau_0] : \tau \mid \varepsilon$ T-E-Val

- Case $\mu x : \tau . e \rightarrow e[x := \mu x : \tau . e]$.

TODO: Fix this, redex/value problem. Cannot substitute.

$\cdot \vdash \mu x : \tau . e : \tau \mid \varepsilon$ Given

- Case $\texttt{handler}^{\varepsilon}_{m}(h)\ v \rightarrow \texttt{handle}^{\varepsilon}_{m}(h, v\ \mathsf{unit})$.

$\cdot \vdash \texttt{handler}^{\varepsilon}_{m}(h)\ v : \tau \mid \varepsilon$ Given
$\cdot \vdash \texttt{handler}^{\varepsilon}_{m}(h) : \tau_1 \to^{\varepsilon} \tau \mid \varepsilon$ T-E-App
$\cdot \vdash v : \tau_1 \mid \varepsilon$ T-E-App
$\cdot \vdash_{\text{val}} \texttt{handler}^{\varepsilon}_{m}(h) : \tau_1 \to^{\varepsilon} \tau$ T-E-Val
$\cdot \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon$ T-E-Handler
$\tau_1 = \mathsf{unit} \to^{\langle l \mid \varepsilon \rangle} \tau$ T-E-Handler
$\cdot \vdash v : \mathsf{unit} \to^{\langle l \mid \varepsilon \rangle} \tau \mid \langle l \mid \varepsilon \rangle$ Lemma D.11
$\cdot \vdash v\ \mathsf{unit} : \tau \mid \langle l \mid \varepsilon \rangle$ T-Unit, T-E-App
$\cdot \vdash \texttt{handle}^{\varepsilon}_{m}(h, v\ \mathsf{unit}) : \tau \mid \varepsilon$ T-E-Handle

- Case $\texttt{handle}^{\varepsilon}_{m}(h, v) \rightarrow v$.

$\cdot \vdash v : \tau \mid \langle l \mid \varepsilon \rangle$ T-E-Handle
$\cdot \vdash_{\text{val}} v : \tau$ T-E-Val
$\cdot \vdash v : \tau \mid \varepsilon$ Lemma D.11

- Case $\texttt{handle}^{\varepsilon}_{d}(h, E[\texttt{perform}\ \mathsf{op}\ \tau_0\ v]) \rightarrow f[\tau_0]\ v\ k$, where $\mathsf{op} \notin \texttt{bop}(E)$, $(\mathsf{op} \to f) \in h$, $\mathsf{op} : \forall \alpha . \tau_1 \to \tau_2 \in \Sigma(l)$ and $k = \lambda^{\varepsilon} x : \tau_2[\alpha := \tau_0] . \texttt{handle}^{\varepsilon}_{d}(h, E[x])$.

$\cdot \vdash \texttt{handle}^{\varepsilon}_{d}(h, E[\texttt{perform}\ \mathsf{op}\ \tau_0\ v]) : \tau \mid \varepsilon$ Given
$\cdot \vdash_{\text{ops}} h : \tau \mid l \mid \varepsilon$ T-E-Handle
$\cdot \vdash f : \forall \alpha . \tau_1 \to^{\varepsilon} ((\tau_2 \to^{\varepsilon} \tau) \to^{\varepsilon} \tau) \mid \varepsilon$ T-E-Ops

$\cdot \vdash_{\text{val}} f : \forall\alpha.\tau_1 \to^{\varepsilon} ((\tau_2 \to^{\varepsilon} \tau) \to^{\varepsilon} \tau)$ T-E-Val
$\cdot \vdash f[\tau_0] : \tau_1[\alpha := \tau_0] \to^{\varepsilon} ((\tau_2[\alpha := \tau_0] \to^{\varepsilon} \tau) \to^{\varepsilon} \tau) \mid \varepsilon$ T-E-Val, T-E-Tapp
$\cdot \vdash \texttt{perform}\ \texttt{op}\ \tau_0\ v : \tau_2[\alpha := \tau_0] \mid \langle \lceil \texttt{handle}^{\varepsilon}_{d}(h, E) \rceil^{l}, \varepsilon \rangle$ Lemma D.2
$\cdot \vdash_{\text{ec}} \texttt{handle}^{\varepsilon}_{d}(h, E) : \tau_2[\alpha := \tau_0] \to \tau \mid \varepsilon$ Lemma D.2
$\cdot \vdash v : \tau_1[\alpha := \tau_0] \mid \varepsilon$ T-E-App, Lemma D.11
$\cdot \vdash f[\tau_0]\ v : (\tau_2[\alpha := \tau_0] \to^{\varepsilon} \tau) \to^{\varepsilon} \tau \mid \varepsilon$ T-E-Val, T-E-Tapp

build up $k \downarrow$

$x : \tau_2[\alpha := \tau_0] \vdash_{\text{val}} x : \tau_2[\alpha := \tau_0]$ T-E-Var
$x : \tau_2[\alpha := \tau_0] \vdash x : \tau_2[\alpha := \tau_0] \mid \varepsilon$ T-E-Val
$x : \tau_2[\alpha := \tau_0] \vdash_{\text{ec}} \texttt{handle}^{\varepsilon}_{d}(h, E) : \tau_2[\alpha := \tau_0] \to \tau \mid \varepsilon$ weakening
$x : \tau_2[\alpha := \tau_0] \vdash \texttt{handle}^{\varepsilon}_{d}(h, E[x]) : \tau \mid \varepsilon$ Lemma D.1
$\cdot \vdash_{\text{val}} \lambda^{\varepsilon} x : \tau_2[\alpha := \tau_0].\texttt{handle}^{\varepsilon}_{d}(h, E[x]) : \tau_2[\alpha := \tau_0] \to^{\varepsilon} \tau$ T-E-Abs
$\cdot \vdash \lambda^{\varepsilon} x : \tau_2[\alpha := \tau_0].\texttt{handle}^{\varepsilon}_{d}(h, E[x]) : \tau_2[\alpha := \tau_0] \to^{\varepsilon} \tau \mid \varepsilon$ T-E-Val
$\cdot \vdash k : \tau_2[\alpha := \tau_0] \to^{\varepsilon} \tau \mid \varepsilon$ T-E-Abs, T-E-Val
$\cdot \vdash f[\tau_0]\ v\ k : \tau \mid \varepsilon$ T-E-App

- Case $\texttt{handle}^{\varepsilon}_{s}(h, E[\texttt{perform}\ \texttt{op}\ \tau_0\ v]) \rightarrow f[\tau_0]\ v\ k$, where $\texttt{op} \notin \texttt{bop}(E)$, $(\texttt{op} \to f) \in h$, $\texttt{op} : \forall\alpha.\tau_1 \to \tau_2 \in \Sigma(l)$ and $k = \lambda^{\langle l,\, \varepsilon\rangle} x : \tau_2[\alpha := \tau_0].E[x]$, with x fresh.

$\cdot \vdash \texttt{handle}^{\varepsilon}_{s}(h, E[\texttt{perform}\ \texttt{op}\ \tau_0\ v]) : \tau \mid \varepsilon$ Given
$\cdot \vdash_{\text{ops}} s : h : \tau \mid l \mid \varepsilon$ T-E-Handle
$\cdot \vdash E[\texttt{perform}\ \texttt{op}\ \tau_0\ v] : \tau \mid \langle l, \varepsilon \rangle$ T-E-Handle
$\cdot \vdash_{\text{val}} f : \forall\alpha.\tau_1 \to^{\varepsilon} (\tau_2 \to^{\langle l,\, \varepsilon\rangle} \tau) \to^{\varepsilon} \tau$ T-E-Ops-Shallow
$\cdot \vdash f[\tau_0] : \tau_1[\alpha := \tau_0] \to^{\varepsilon} (\tau_2[\alpha := \tau_0] \to^{\langle l,\, \varepsilon\rangle} \tau) \to^{\varepsilon} \tau \mid \varepsilon$ T-E-Val, T-E-Tapp
$\cdot \vdash \texttt{perform}\ \texttt{op}\ \tau_0\ v : \tau_2[\alpha := \tau_0] \mid \langle \lceil E \rceil^{l}, \langle l, \varepsilon \rangle \rangle$ Lemma D.2
$\cdot \vdash_{\text{ec}} E : \tau_2[\alpha := \tau_0] \to \tau \mid \langle l, \varepsilon \rangle$ Lemma D.2
$\cdot \vdash v : \tau_1[\alpha := \tau_0] \mid \varepsilon$ T-E-App, Lemma D.11
$\cdot \vdash f[\tau_0]\ v : (\tau_2[\alpha := \tau_0] \to^{\langle l,\, \varepsilon\rangle} \tau) \to^{\varepsilon} \tau \mid \varepsilon$ T-E-App
$x : \tau_2[\alpha := \tau_0] \vdash_{\text{val}} x : \tau_2[\alpha := \tau_0]$ T-E-Var
$x : \tau_2[\alpha := \tau_0] \vdash x : \tau_2[\alpha := \tau_0] \mid \langle \lceil E \rceil^{l}, \langle l, \varepsilon \rangle \rangle$ T-E-Val
$x : \tau_2[\alpha := \tau_0] \vdash_{\text{ec}} E : \tau_2[\alpha := \tau_0] \to \tau \mid \langle l, \varepsilon \rangle$ Weakening
$x : \tau_2[\alpha := \tau_0] \vdash E[x] : \tau \mid \langle l, \varepsilon \rangle$ Lemma D.1
$\cdot \vdash_{\text{val}} k : \tau_2[\alpha := \tau_0] \to^{\langle l,\, \varepsilon\rangle} \tau$ T-E-Abs
$\cdot \vdash k : \tau_2[\alpha := \tau_0] \to^{\langle l,\, \varepsilon\rangle} \tau \mid \varepsilon$ T-E-Val
$\cdot \vdash f[\tau_0]\ v\ k : \tau \mid \varepsilon$ T-E-App

- Cases S-If-True, S-If-False, S-Inj-Left, S-Inj-Right, S-Op, S-Zero. As in Lemma A.6, with every judgment at effect ε.

□

Lemma D.9. (Variable Substitution)

If $\Gamma_1, x : \tau_1, \Gamma_2 \vdash e : \tau \mid \varepsilon$ and $\Gamma_1, \Gamma_2 \vdash_{\text{val}} v : \tau_1$ then $\Gamma_1, \Gamma_2 \vdash e[x := v] : \tau \mid \varepsilon$

Proof. □

Lemma D.10. (Type Variable Substitution)

If $\Gamma \vdash e : \tau \mid \varepsilon$ and $\vdash_{\text{wf}} \tau_1 : k$ then $\Gamma[\alpha^k := \tau_1] \vdash e[\alpha^k := \tau_1] : \tau[\alpha^k := \tau_1] \mid \varepsilon$

Proof. □

Lemma D.11. (Values take any effect)

If $\Gamma \vdash_{\text{val}} v : \tau$ then $\Gamma \vdash v : \tau \mid \varepsilon$

CHAPTER E

Contracts to Effects Compiler Typesoundness

E.1 Overview

The environment that will be used to type check target programs has the following Σ signature.

$$\Sigma(\mathtt{h}_{\mathscr{C}}) = \left\{\mathscr{C} : \forall\alpha.\langle \mathtt{blab}, \langle \alpha \rightarrow^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \alpha\rangle\rangle \rightarrow \alpha\right\}$$

$\cdot^* : \mathsf{Contracts}\ \tau \rightarrow \mathtt{Effects}\ \tau$

$$\begin{aligned}
\mathsf{num}^* &= \mathsf{num} \\
\mathsf{bool}^* &= \mathsf{bool} \\
\mathsf{unit}^* &= \mathsf{unit} \\
\langle \tau_1, \tau_2 \rangle^* &= \langle \tau_1^*, \tau_2^* \rangle \\
(\tau_1 \rightarrow \tau_2)^* &= \tau_1^* \rightarrow^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau_2^* \\
(x_1 : \tau_1, ..., x_n : \tau_n)^* &= x_1 : \tau_1^*, ..., x_n : \tau_n^*
\end{aligned}$$

E.2 Compiler Type Preservation

Theorem E.1. (Compiler Type Preservation)

If $\Gamma \vdash e : \tau \rightarrowtail^l e'$ then $\Gamma^* \vdash e' : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$.

Proof.

By induction on structure of $\Gamma \vdash e : \tau \rightarrowtail^l e'$.

- Case $e = x$, $e = n$, $e = b$, $e = \mathsf{unit}$

  $e' = e$ — Given

  $\Gamma^* \vdash_{\mathrm{val}} e' : \tau^*$ — T-E-Var, T-Num, T-Bool, T-Unit

  $\Gamma^* \vdash e' : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ — T-E-Val

- Case

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \rightarrowtail^l e''}{\Gamma \vdash \lambda x : \tau_1.e : \tau_1 \to \tau_2 \rightarrowtail^l \lambda x : \tau_1^*.e''}$$

$\Gamma^*, x : \tau_1^* \vdash e'' : \tau_2^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^* \vdash_{\text{val}} \lambda x : \tau_1^*.e'' : \tau_1^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau_2^*$ **T-E-Abs**
$\Gamma^* \vdash_{\text{val}} \lambda x : \tau_1^*.e'' : (\tau_1 \to \tau_2)^*$ Type compilation $\cdot^*$
$\Gamma^* \vdash \lambda x : \tau_1^*.e'' : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ **T-E-Val**

- Case

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \rightarrowtail^l e_1' \qquad \Gamma \vdash e_2 : \tau_1 \rightarrowtail^l e_2'}{\Gamma \vdash e_1\ e_2 : \tau_2 \rightarrowtail^l e_1'\ e_2'}$$

$\Gamma^* \vdash e_1' : \tau_1^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau_2^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^* \vdash e_2' : \tau_1^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^* \vdash e_1'\ e_2' : \tau_2^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ **T-E-App**

The effect on the arrow and the effect of the premises are both $\langle \mathtt{h}_{\mathscr{C}} \rangle$.

- Case $\mu x : \tau.e$, $\langle e_1, e_2 \rangle$, $\mathsf{inj.l}(e)$, $\mathsf{inj.r}(e)$, if e_1 then e_2 else e_3, $o(e_1, ..., e_n)$

$\Gamma^* \vdash e_i' : \tau_i^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ for every sub-expression I.H.
$\Gamma^* \vdash e' : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ lifted rule of Base

For $\mu x : \tau.e$, the binder is $x : \tau^*$, and the I.H. is used under $\Gamma^*, x : \tau^*$.

- Case

$$\frac{\Gamma \vdash e_3 : \tau \to \mathsf{bool} \rightarrowtail^l e_5 \qquad \Gamma \vdash e_4 : \tau \rightarrowtail^l e_6}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), e_4) : \tau \rightarrowtail^l \mathtt{perform}\ \mathscr{C}\ \tau^*\ \langle k, \langle e_5, e_6 \rangle \rangle}$$

$\Gamma^* \vdash e_5 : \tau^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool} \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^* \vdash e_6 : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^* \vdash \mathtt{perform}\ \mathscr{C}\ \tau^*\ \langle k, \langle e_5, e_6 \rangle \rangle : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ Lemma E.2

This is the only rule that introduces $\mathtt{h}_{\mathscr{C}}$.

- Case

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \rightarrowtail^l e_2 \qquad \Gamma \vdash \lambda f : \tau_1 \to \tau_2.\lambda x : \tau_1.\mathsf{mon}_j^{k,l}\big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^l e_3}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail^l e_3\ e_2}$$

$\Gamma^* \vdash e_2 : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^* \vdash e_3 : ((\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2))^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^* \vdash e_3 : (\tau_1 \to \tau_2)^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ def. of $\cdot^*$

$\Gamma^* \vdash e_3\ e_2 : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-App

- Case

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \rightarrowtail^l e_2 \qquad \Gamma \vdash e_w : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^l e_3}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail^l e_3\ e_2}$$

where $e_w = \lambda f : \tau_1 \to \tau_2.\lambda x : \tau_1.\mathsf{mon}_j^{k,l}\Big(\kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\Big], f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)$

$\Gamma^* \vdash e_2 : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^* \vdash e_3 : (\tau_1 \to \tau_2)^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H., def. of $\cdot^*$
$\Gamma^* \vdash e_3\ e_2 : (\tau_1 \to \tau_2)^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-App

The dependency y is replaced in the source by the indy monitor $\mathsf{mon}_j^{l,j}(\kappa_1, x) : \tau_1$, so the I.H. on e_w already covers κ_2 in the context $\Gamma, f : \tau_1 \to \tau_2, x : \tau_1$.

- Case

$$\frac{\begin{array}{c} x \notin \mathrm{dom}(\Gamma) \qquad \Gamma, x : \langle \tau_1, \tau_2 \rangle \vdash \mathsf{mon}_j^{k,l}(\kappa_1, \mathsf{inj.l}(x)) : \tau_1 \rightarrowtail^l e_1 \\ \Gamma, x : \langle \tau_1, \tau_2 \rangle \vdash \mathsf{mon}_j^{k,l}(\kappa_2, \mathsf{inj.r}(x)) : \tau_2 \rightarrowtail^l e_2 \qquad \Gamma \vdash e : \langle \tau_1, \tau_2 \rangle \rightarrowtail^l e_3 \end{array}}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\langle \kappa_1, \kappa_2 \rangle, e) : \langle \tau_1, \tau_2 \rangle \rightarrowtail^l (\lambda x : \langle \tau_1, \tau_2 \rangle^*.\langle e_1, e_2 \rangle)\ e_3}$$

$\Gamma^*, x : \langle \tau_1, \tau_2 \rangle^* \vdash e_1 : \tau_1^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^*, x : \langle \tau_1, \tau_2 \rangle^* \vdash e_2 : \tau_2^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^*, x : \langle \tau_1, \tau_2 \rangle^* \vdash \langle e_1, e_2 \rangle : \langle \tau_1, \tau_2 \rangle^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-Tuple
$\Gamma^* \vdash_{\mathrm{val}} \lambda x : \langle \tau_1, \tau_2 \rangle^*.\langle e_1, e_2 \rangle : \langle \tau_1, \tau_2 \rangle^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \langle \tau_1, \tau_2 \rangle^*$ T-E-Abs
$\Gamma^* \vdash \lambda x : \langle \tau_1, \tau_2 \rangle^*.\langle e_1, e_2 \rangle : \langle \tau_1, \tau_2 \rangle^* \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \langle \tau_1, \tau_2 \rangle^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-Val
$\Gamma^* \vdash e_3 : \langle \tau_1, \tau_2 \rangle^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ I.H.
$\Gamma^* \vdash (\lambda x.\langle e_1, e_2 \rangle)\ e_3 : \langle \tau_1, \tau_2 \rangle^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-App

□

Lemma E.2. (Perform Type)

If $\Gamma \vdash e_1 : \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool} \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ and $\Gamma \vdash e_2 : \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ then

$\Gamma \vdash \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_1, e_2 \rangle \rangle : \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$.

Proof.

$\Gamma \vdash_{\mathrm{val}} \mathtt{perform}\ \mathscr{C}\ \tau : \langle \mathtt{blab}, \langle \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \tau \rangle \rangle \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau$ T-E-Perform, $\alpha := \tau, \varepsilon := \langle\rangle$
$\Gamma \vdash \mathtt{perform}\ \mathscr{C}\ \tau : \langle \mathtt{blab}, \langle \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \tau \rangle \rangle \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-Val
$\Gamma \vdash k : \mathtt{blab} \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-E-Label
$\Gamma \vdash e_1 : \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool} \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ Given
$\Gamma \vdash e_2 : \tau \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ Given
$\Gamma \vdash \langle e_1, e_2 \rangle : \langle \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \tau \rangle \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-Tuple
$\Gamma \vdash \langle k, \langle e_1, e_2 \rangle \rangle : \langle \mathtt{blab}, \langle \tau \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \tau \rangle \rangle \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ T-Tuple

$\Gamma \vdash \texttt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_1, e_2\rangle\rangle : \tau \mid \langle \mathtt{h}_{\mathscr{C}}\rangle$ T-E-App

□

E.3 Wrapped Compiled Program Typing (Deep Handler)

The deep handler wrapper from the compiler equivalence proof removes $\mathtt{h}_{\mathscr{C}}$ from the effect row. We rewrite the wrapper as follows:

$$\mathsf{wrap}_d = \mu w : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \alpha\big) \to^{\langle\rangle} \alpha.\Lambda\alpha.\lambda f : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \alpha.\mathtt{handler}_d(h_w)\ f$$

$$h_w = \big\{\mathscr{C} \to \Lambda\beta.\lambda\langle l, \langle e, v\rangle\rangle.\lambda k : \beta \to^{\langle\rangle} \alpha.\ \text{if } w[\mathsf{bool}]\ (\lambda_ : \mathsf{unit}.e\ v)\ \text{then } k\ v\ \text{else } \mathtt{error}_l\big\}$$

Note that h_w signature matches $\Sigma(\mathtt{h}_{\mathscr{C}})$ if we replace α with β.

Theorem E.3. (Compiler Effect Safety with wrapper deep handler)

If $\cdot \vdash e : \tau \rightarrowtail^{l} e'$ then $\cdot \vdash \mathsf{wrap}_d[\tau^*]\ (\lambda_ : \mathsf{unit}.e') : \tau^* \mid \langle\rangle$.

Proof.

$\cdot \vdash e' : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}}\rangle$ Theorem E.1
$\cdot \vdash_{\mathrm{val}} \lambda_ : \mathsf{unit}.e' : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \tau^*$ T-E-Abs
$\cdot \vdash \mathsf{wrap}_d[\tau^*] : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \tau^*\big) \to^{\langle\rangle} \tau^* \mid \langle\rangle$ Lemma E.4, T-E-Tapp
$\cdot \vdash \mathsf{wrap}_d[\tau^*]\ (\lambda_ : \mathsf{unit}.e') : \tau^* \mid \langle\rangle$ T-E-App

□

Lemma E.4. (Type of wrap deep handler)

$\cdot \vdash_{\mathrm{val}} \mathsf{wrap}_d : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \alpha\big) \to^{\langle\rangle} \alpha$

Proof.

Assume contexts $\Gamma_w = w : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \alpha\big) \to^{\langle\rangle} \alpha.$

And $\Delta = \Gamma_w, l : \mathtt{blab}, e : \beta \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \mathsf{bool}, v : \beta, k : \beta \to^{\langle\rangle} \alpha.$

$\Delta \vdash e\ v : \mathsf{bool} \mid \langle \mathtt{h}_{\mathscr{C}}\rangle$ T-E-App
$\Delta \vdash_{\mathrm{val}} \lambda_ : \mathsf{unit}.e\ v : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \mathsf{bool}$ T-E-Abs
$\Delta \vdash w[\mathsf{bool}] : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \mathsf{bool}\big) \to^{\langle\rangle} \mathsf{bool} \mid \langle\rangle$ T-E-Tapp
$\Delta \vdash w[\mathsf{bool}]\ (\lambda_ : \mathsf{unit}.e\ v) : \mathsf{bool} \mid \langle\rangle$ T-E-App
$\Delta \vdash k\ v : \alpha \mid \langle\rangle$ T-E-App
$\Delta \vdash \mathtt{error}_l : \alpha \mid \langle\rangle$ T-E-Error
$\Delta \vdash \text{if } w[\mathsf{bool}]\ (\lambda_ : \mathsf{unit}.e\ v)\ \text{then } k\ v\ \text{else } \mathtt{error}_l : \alpha \mid \langle\rangle$ T-If

$\Gamma_w \vdash_{\text{val}} h_{w(\mathscr{C})} : \forall\beta.\langle \texttt{blab}, \langle \beta \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \beta \rangle\rangle \to^{\langle\rangle} \big((\beta \to^{\langle\rangle} \alpha) \to^{\langle\rangle} \alpha\big)$ T-E-Abs, T-E-TAbs

$\Gamma_w \vdash_{\text{ops}} h_w : \alpha \mid \langle \mathtt{h}_{\mathscr{C}} \mid \langle\rangle\rangle$ T-E-Ops

$\Gamma_w \vdash_{\text{val}} \texttt{handler}_d(h_w) : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$ T-E-Handler

$\Gamma_w, f : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha \vdash \texttt{handler}_d(h_w)\ f : \alpha \mid \langle\rangle$ T-E-App

$\Gamma_w \vdash_{\text{val}} \lambda f : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha.\texttt{handler}_d(h_w)\ f : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$ T-E-Abs

$\Gamma_w \vdash_{\text{val}} \Lambda\alpha.\lambda f.\texttt{handler}_d(h_w)\ f : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$ T-E-TAbs

$\cdot \vdash_{\text{val}} \mathsf{wrap}_d : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$ T-Fix

□

E.4 Wrapped Compiled Program Typing (Shallow Handler)

$$\mathsf{wrap} = \Lambda\alpha.\ \lambda f : \mathsf{unit} \to \alpha.\ \mathsf{H}\ f$$

$$\mathsf{H} = \mu h.\texttt{handler}_s(\{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.h\ (\lambda_.\ \text{if } e\ v \text{ then } k\ v \text{ else } \texttt{error}_l)\})$$

Note that h_w signature matches $\Sigma(\mathtt{h}_{\mathscr{C}})$ if we replace α with β.

Theorem E.5. (Compiler Effect Safety with wrapper shallow handler)

If $\cdot \vdash e : \tau \rightarrowtail^{l} e'$ then $\cdot \vdash \mathsf{wrap}_s[\tau^*]\ (\lambda_ : \mathsf{unit}.e') : \tau^* \mid \langle\rangle$.

Proof.

$\cdot \vdash e' : \tau^* \mid \langle \mathtt{h}_{\mathscr{C}} \rangle$ Theorem E.1

$\cdot \vdash_{\text{val}} \lambda_ : \mathsf{unit}.e' : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau^*$ T-E-Abs

$\cdot \vdash \mathsf{wrap}_s[\tau^*] : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \tau^*\big) \to^{\langle\rangle} \tau^* \mid \langle\rangle$ Lemma E.6, T-E-Tapp

$\cdot \vdash \mathsf{wrap}_s[\tau^*]\ (\lambda_ : \mathsf{unit}.e') : \tau^* \mid \langle\rangle$ T-E-App

□

Lemma E.6. (Type of wrap shallow handler)

$\cdot \vdash_{\text{val}} \mathtt{wrap}_s : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$

Proof.

Assume $\Gamma_w = h : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$.

$\Gamma_w \vdash_{\text{val}} \mathsf{H} : \forall\beta.\langle \texttt{blab}, \langle \beta \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \mathsf{bool}, \beta \rangle\rangle \to^{\langle\rangle} \big((\beta \to^{\langle\rangle} \alpha) \to^{\langle\rangle} \alpha\big)$ T-E-Abs, T-E-TAbs

$\Gamma_w \vdash_{\text{ops}} \mathsf{H} : \alpha \mid \langle \mathtt{h}_{\mathscr{C}} \mid \langle\rangle\rangle$ T-E-Ops

$\Gamma_w \vdash_{\text{val}} \texttt{handler}_s(\mathsf{H}) : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$ T-E-Handler

$\Gamma_w \vdash_{\text{val}} \lambda f : \mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha.\texttt{handler}_s(\mathsf{H})\ f : \big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$ T-E-Abs

$\cdot \vdash_{\text{val}} \Lambda\alpha.\lambda f.\texttt{handler}_s(\mathsf{H})\ f : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$ T-E-TAbs

$\cdot \vdash_{\text{val}} \mathtt{wrap}_s : \forall\alpha.\big(\mathsf{unit} \to^{\langle \mathtt{h}_{\mathscr{C}} \rangle} \alpha\big) \to^{\langle\rangle} \alpha$ T-Fix

□

CHAPTER F

Contracts to Effects Compiler Correctness (Deep Handler)

F.1 Overview

Proof for a compiler from Contracts to Effects using the deep handler, where the continuation automatically reinstalls the handler.

$$\begin{array}{lcl}
\mathtt{h}_{\mathscr{C}} : \mathtt{lab} & = & \{\mathscr{C} \to \forall\alpha.\langle \mathtt{blab}, \langle \alpha \to \mathsf{bool}, \alpha\rangle\rangle \to \alpha\} \\
\mathsf{wrap} & = & \mu w : \forall\alpha.\big(\big(() \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \alpha\big) \to \alpha\big).\Lambda\alpha.\ \lambda f : \big(() \to^{\langle \mathtt{h}_{\mathscr{C}}\rangle} \alpha\big).\mathtt{handler}_d(h)\ f \\
h & = & \{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.\ \text{if } w[\mathsf{bool}]\ (\lambda_.e\ v) \text{ then } k\ v \text{ else } \mathtt{error}_l\} \\
 & & \quad \text{where } \lambda\langle l, \langle e, v\rangle\rangle \text{ deconstructs the argument as tuple pattern}
\end{array}$$

F.2 Relations

Before we define relations that will be used in the simulation proof below. We define:

- Context E^* is any context without $\mathsf{check}^k_j(e, v)$ in Contracts.
- Context $E^+ = \square \mid E^+\left[E^*\left[\mathsf{check}^k_j(\square, v)\right]\right]$

$\boxed{R_v \subseteq \text{Contracts } v \times \mathtt{Effects}\ v}$

Abuse of notation: If $(v, v') \in R_v$, we write $v \sim_{R_v} v'$ or $v \sim v'$. We write $\tilde{v}$ for any value $\tilde{v}$ satisfying $v \sim \tilde{v}$.

$$\begin{array}{rcl}
x & \sim & x \\
b & \sim & b \\
N & \sim & N
\end{array}$$

$$
\begin{array}{rcl}
\lambda x : \tau . e & \sim & \lambda x : \tau^* . \tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x : \tau . \mathsf{mon}_j^{k,l}\Big(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x : \tau . \mathsf{mon}_j^{k,l}\Big(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
 & & \kappa_3 = \kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\Big]
\end{array}
$$

$R_e \subseteq \mathsf{Contracts}\ e \times \mathtt{Effects}\ e$

Abuse of notation: If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We write $\tilde{e}$ for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

$$
\begin{array}{rcl}
v & \sim & \tilde{v} \\
e_1\ e_2 & \sim & \tilde{e}_1\ \tilde{e}_2 \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) & \sim & \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2 \rangle \rangle \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\Big(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
 & & \kappa_3 = \kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\Big] \\
\mathsf{blame}_j^k & \sim & \mathtt{error}_k
\end{array}
$$

$R_{E^*} \subseteq \mathsf{Contracts}\ E^* \times \mathtt{Effects}\ E^*$

Abuse of notation: If $(E^*, E'^*) \in R_{E^*}$, we write $E^* \sim_{R_{E^*}} E'^*$ or $E^* \sim E'^*$. We write $\tilde{E}^*$ for any expression $\tilde{E}^*$ satisfying $E^* \sim \tilde{E}^*$.

$$
\begin{array}{rcl}
\square & \sim & \square \\
E^*\ e & \sim & \tilde{E}^*\ \tilde{e} \\
v\ E^* & \sim & \tilde{v}\ \tilde{E}^* \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), E^*) & \sim & \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{E}^* \rangle \rangle \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E^*) & \sim & \tilde{e}_1\ \tilde{E}^* \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)
\end{array}
$$

$$\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, E^*) \quad \sim \quad \tilde{e}_1\ \tilde{E}^* \qquad e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\big(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big) \qquad \kappa_3 = \kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\big]$$

$\boxed{R_{E^+} \subseteq \mathsf{Contracts}\ E^+ \times \mathtt{Effects}\ E^+}$

Abuse of notation: If $(E^+, E'^+) \in R_{E^+}$, we write $E^+ \sim_{R_{E^+}} E'^+$ or $E^+ \sim E'^+$. We write $\tilde{E}^+$ for any expression $\tilde{E}^+$ satisfying $E^+ \sim \tilde{E}^+$.

$$\Box \quad \sim \quad \Box$$

$$E^+\big[E^*\big[\mathsf{check}_j^k(\Box, v)\big]\big] \quad \sim \quad \tilde{E}^+\big[\text{if } \Box \text{ then } \big(\lambda x.\mathtt{handle}_d\big(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]\big)\big)\ \tilde{v} \text{ else } \mathtt{error}_k\big]$$

Lemma F.1. (Relation is closed) If $e \sim e'$ and $E^* \sim E'^*$ then $E^*[e] \sim E'^*[e']$.

Proof. By induction on the structure of $E^* \sim E'^*$.

- Case $\Box \sim \Box$

  $E^*[e] = e$ — Context Plug in
  $E'^*[e'] = e'$ — Context Plug in
  $e \sim e'$ — Given
  $E^*[e] \sim E'^*[e']$

- Case $E_1^*\ e_1 \sim \tilde{E}_1^*\ \tilde{e}_1$

  $E_1^* \sim \tilde{E}_1^*$ — Given
  $E_1^*[e] \sim \tilde{E}_1^*[e']$ — I.H.
  $E_1^*[e]\ e_1 \sim \tilde{E}_1^*[e']\ \tilde{e}_1$ — Relation
  $E^*[e] \sim E'^*[e']$

- Case $\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), E_1^*) \sim \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{E}_1^* \rangle\rangle$

  $E_1^* \sim \tilde{E}_1^*$ — Given
  $E_1^*[e] \sim \tilde{E}_1^*[e']$ — I.H.
  $\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), E_1^*[e]) \sim \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{E}_1^*[e'] \rangle\rangle$ — Relation
  $E^*[e] \sim E'^*[e']$

- Case $\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, \tilde{E}_1) \sim \tilde{e}_1\ \tilde{E}_1^*$, with $e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big)$

  $e_1 \sim \tilde{e}_1$ — Given
  $E_1^* \sim \tilde{E}_1^*$ — Given
  $E_1^*[e] \sim \tilde{E}_1^*[e']$ — I.H.

$\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E_1^*[e]) \sim \tilde{e}_1\ \tilde{E}_1^*[e']$ Relation
$E^*[e] \sim E'^*[e']$

- Case $\mathsf{mon}_j^{k,l}\big(\kappa_1 \to_i \lambda y.\kappa_2, \tilde{E}_1\big) \sim \tilde{e}_1\ \tilde{E}_1^*$, with $e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)$ and $\kappa_3 = \kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\Big]$

  Similarly to the above case

- Other cases follow similarly.

□

Lemma F.2. (Relation decomposition through check-free contexts)

If $E^*[e] \sim e'$, then there exist E'^* and e'_0 such that $E^* \sim E'^*$ and $e \sim e'_0$ and $e' = E'^*[e'_0]$.

Proof. By induction on the check-free context E^*.

- Case $E^* = \square$

  $E^*[e] = e \sim e'$ Given
  $E'^* = \square$ Choose
  $e'_0 = e'$ Choose
  $E^* \sim E'^*$ Definition
  $e \sim e'_0$ Given
  $e' = E'^*[e'_0]$ Context Plug in

- Case $E^* = E_0^*\ e_1$

  $E_0^*[e]\ e_1 \sim e'$ Given
  $E_0^*[e] \sim e'_0$ Relation inversion
  $e_1 \sim e'_1$ Relation inversion
  $e' = e'_0\ e'_1$ Relation inversion
  $E_0^* \sim E_0'^*$ I.H.
  $e \sim e'_2$ I.H.
  $e'_0 = E_0'^*[e'_2]$ I.H.
  $E'^* = E_0'^*\ e'_1$ Choose
  $E^* \sim E'^*$ Definition
  $e' = E'^*[e'_2]$ Context Plug in

- Case $E^* = v\ E_0^*$

  $v\ E_0^*[e] \sim e'$ Given
  $v \sim v'$ Relation inversion
  $E_0^*[e] \sim e'_0$ Relation inversion

$e' = v'\ e'_0$ — Relation inversion
$E^*_0 \sim E'^*_0$ — I.H.
$e \sim e'_1$ — I.H.
$e'_0 = E'^*_0[e'_1]$ — I.H.
$E'^* = v'\ E'^*_0$ — Choose
$E^* \sim E'^*$ — Definition
$e' = E'^*[e'_1]$ — Context Plug in

- Case $E^* = \mathsf{mon}^{k,l}_j(\mathsf{flat}(e_0), E^*_0)$

$\mathsf{mon}^{k,l}_j(\mathsf{flat}(e_0), E^*_0[e]) \sim e'$ — Given
$e_0 \sim e'_0$ — Relation inversion
$E^*_0[e] \sim e'_1$ — Relation inversion
$e' = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle e'_0, e'_1\rangle\rangle$ — Relation inversion
$E^*_0 \sim E'^*_0$ — I.H.
$e \sim e'_2$ — I.H.
$e'_1 = E'^*_0[e'_2]$ — I.H.
$E'^* = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle e'_0, E'^*_0\rangle\rangle$ — Choose
$E^* \sim E'^*$ — Definition
$e' = E'^*[e'_2]$ — Context Plug in

- Case $E^* = \mathsf{mon}^{k,l}_j(\kappa, E^*_0)$

$\mathsf{mon}^{k,l}_j(\kappa, E^*_0[e]) \sim e'$ — Given
$E^*_0[e] \sim e'_0$ — Relation inversion
$e' = e'_c\ e'_0$ — Relation inversion
$E^*_0 \sim E'^*_0$ — I.H.
$e \sim e'_1$ — I.H.
$e'_0 = E'^*_0[e'_1]$ — I.H.
$E'^* = e'_c\ E'^*_0$ — Choose
$E^* \sim E'^*$ — Definition
$e' = E'^*[e'_1]$ — Context Plug in

- Other cases follow similarly.

□

Lemma F.3. (Relation is closed under substitution)

If $e \sim e'$ and $v \sim v'$ then $e[x := v] \sim e'[x := v']$.

Proof. By induction on the derivation of $e \sim e'$.

- Case $x \sim x$

$x[x := v] = v$ Replacement
$x[x := v'] = v'$ Replacement
$v \sim v'$ Given
$x[x := v] \sim x[x := v']$

- Case $z \sim z$, where $z \neq x$

$z[x := v] = z$ Replacement
$z[x := v'] = z$ Replacement
$z[x := v] \sim z[x := v']$ Definition

- Case $\lambda z.e_1 \sim \lambda z.e_1'$

$e_1[x := v] \sim e_1'[x := v']$ I.H.
$\lambda z.e_1[x := v] \sim \lambda z.e_1'[x := v']$ Definition
$(\lambda z.e_1)[x := v] \sim (\lambda z.e_1')[x := v']$ Replacement

- Case $e_1\ e_2 \sim e_1'\ e_2'$

$e_1[x := v] \sim e_1'[x := v']$ I.H.
$e_2[x := v] \sim e_2'[x := v']$ I.H.
$e_1\ e_2[x := v] = e_1[x := v]\ e_2[x := v]$ Replacement
$e_1'\ e_2'[x := v'] = e_1'[x := v']\ e_2'[x := v']$ Replacement
$e_1\ e_2[x := v] \sim e_1'\ e_2'[x := v']$ Definition

- Case $\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) \sim \texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2 \rangle \rangle$

$e_1[x := v] \sim \tilde{e}_1[x := v']$ I.H.
$e_2[x := v] \sim \tilde{e}_2[x := v']$ I.H.
$\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2)[x := v] \sim \texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2 \rangle \rangle[x := v']$ Definition

- Case $\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1) \sim \tilde{e}_2\ \tilde{e}_1$, with $e_2 = \lambda f.\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, y)\right)$

$e_1 \sim \tilde{e}_1$ Given
$e_2 \sim \tilde{e}_2$ Given
$e_1[x := v] \sim \tilde{e}_1[x := v']$ I.H.
$e_2[x := v] \sim \tilde{e}_2[x := v']$ I.H.
$\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1)[x := v] = \mathsf{mon}_j^{k,l}(\kappa_1[x := v] \to \kappa_2[x := v], e_1[x := v])$
$e_2[x := v] = \lambda f.\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2[x := v], f\ \mathsf{mon}_j^{l,k}(\kappa_1[x := v], y)\right)$
$\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1)[x := v] \sim \tilde{e}_2[x := v']\ \tilde{e}_1[x := v']$ Relation
$\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1)[x := v] \sim (\tilde{e}_2\ \tilde{e}_1)[x := v']$

Choose variables, $f, y \neq x$ and $f, y \notin \mathrm{fv}(v, v')$ by alpha-renaming.

- Case $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, \tilde{e}_1) \sim \tilde{e}_2\ \tilde{e}_1$, with $e_2 = \lambda f.\lambda z.\mathsf{mon}_j^{k,l}\left(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, z)\right)$ and $\kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, z)\right]$ and $y \neq x$

  $e_1 \sim \tilde{e}_1$ — Given
  $e_2 \sim \tilde{e}_2$ — Given
  $e_1[x := v] \sim \tilde{e}_1[x := v']$ — I.H.
  $e_2[x := v] \sim \tilde{e}_2[x := v']$ — I.H.
  $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_1)[x := v] =$ — Substitution

  $\mathsf{mon}_j^{k,l}(\kappa_1[x := v] \to_i \lambda y.\kappa_2[x := v], e_1[x := v])$

  $e_2[x := v] = \lambda f.\lambda z.\mathsf{mon}_j^{k,l}\left(\kappa_3[x := v], f\ \mathsf{mon}_j^{l,k}(\kappa_1[x := v], z)\right)$
  $\kappa_3[x := v] = \left(\kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, z)\right]\right)[x := v]$
  $\kappa_3[x := v] = (\kappa_2[x := v])\left[y := \mathsf{mon}_j^{l,j}(\kappa_1[x := v], z)\right]$ — Substitution?
  $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_1)[x := v] \sim \tilde{e}_2[x := v']\ \tilde{e}_1[x := v']$ — Relation
  $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_1)[x := v] \sim (\tilde{e}_2\ \tilde{e}_1)[x := v']$ — Definition

  Choose variables, $f, z \neq x$ and $f, y, z \notin \mathrm{fv}(v, v')$ by alpha-renaming.

- Case $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, \tilde{e}_1) \sim \tilde{e}_2\ \tilde{e}_1$, with $e_2 = \lambda f.\lambda z.\mathsf{mon}_j^{k,l}\left(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, z)\right)$ and $\kappa_3 = \kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1, z)\right]$

  $e_1 \sim \tilde{e}_1$ — Given
  $e_2 \sim \tilde{e}_2$ — Given
  $e_1[x := v] \sim \tilde{e}_1[x := v']$ — I.H.
  $e_2[x := v] \sim \tilde{e}_2[x := v']$ — I.H.
  $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, e_1)[x := v] =$ — Substitution

  $\mathsf{mon}_j^{k,l}(\kappa_1[x := v] \to_i \lambda x.\kappa_2, e_1[x := v])$

  $e_2[x := v] = \lambda f.\lambda z.\mathsf{mon}_j^{k,l}\left(\kappa_3[x := v], f\ \mathsf{mon}_j^{l,k}(\kappa_1[x := v], y)\right)$
  $\kappa_3[x := v] = \left(\kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1, z)\right]\right)[x := v]$
  $\kappa_3[x := v] = \left(\kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1[x := v], z)\right]\right)$
  $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, e_1)[x := v] \sim \tilde{e}_2[x := v']\ \tilde{e}_1[x := v']$ — Relation
  $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, e_1)[x := v] \sim (\tilde{e}_2\ \tilde{e}_1)[x := v']$ — Definition

  Choose variables, $f, z \neq x$ and $f, y, z \notin \mathrm{fv}(v, v')$ by alpha-renaming.

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1) \sim \tilde{e}_1$, with $e_1 = \lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, y)\right)$

  $e_1 \sim \tilde{e}_1$ — Given
  $e_1[x := v] \sim \tilde{e}_1[x := v']$ — I.H.
  $e_1[x := v] = \lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2[x := v], v_1[x := v]\ \mathsf{mon}_j^{l,k}(\kappa_1[x := v], y)\right)$
  $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)[x := v] = \mathsf{guard}_j^{k,l}(\kappa_1[x := v] \to \kappa_2[x := v], v_1[x := v])$
  $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)[x := v] \sim \tilde{e}_1[x := v']$ — Relation

Choose variables, $y \neq x$ and $y \notin \mathrm{fv}(v, v')$ by alpha-renaming.

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \sim \tilde{e}_1$, with $e_1 = \lambda z.\mathsf{mon}_j^{k,l}\left(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, z)\right)$ and $\kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, z)\right]$

  $e_1 \sim \tilde{e}_1$ Given
  $e_1[x := v] \sim \tilde{e}_1[x := v']$ I.H.
  $e_1[x := v] = \lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_3[x := v], v_1[x := v]\ \mathsf{mon}_j^{l,k}(\kappa_1[x := v], x)\right)$
  $\kappa_3[x := v] = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, z)\right][x := v]$
  $\kappa_3[x := v] = (\kappa_2[x := v])\left[y := \mathsf{mon}_j^{l,j}(\kappa_1[x := v], z)\right]$ Substitution?
  $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)[x := v] =$

  $\mathsf{guard}_j^{k,l}(\kappa_1[x := v] \to_i \lambda y.\kappa_2[x := v], v_1[x := v])$

  $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)[x := v] \sim \tilde{e}[x := v']$ Relation

  Choose variables, $z \neq x$ and $y, z \notin \mathrm{fv}(v, v')$ by alpha-renaming.

- Case $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v) \sim \tilde{e}_1$, with $e_1 = \lambda z.\mathsf{mon}_j^{k,l}\left(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$ and $\kappa_3 = \kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$

  $e_1 \sim \tilde{e}_1$ Given
  $e_1[x := v] \sim \tilde{e}_1[x := v']$ I.H.
  $e_1[x := v] = \lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_3[x := v], v_1[x := v]\ \mathsf{mon}_j^{l,k}(\kappa_1[x := v], x)\right)$
  $\kappa_3[x := v] = \kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1, z)\right][x := v]$
  $\kappa_3[x := v] = \kappa_2\left[x := \mathsf{mon}_j^{l,j}(\kappa_1[x := v], z)\right]$
  $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)[x := v] =$

  $\mathsf{guard}_j^{k,l}(\kappa_1[x := v] \to_i \lambda x.\kappa_2, v_1[x := v])$

  $\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda x.\kappa_2, v_1)[x := v] \sim \tilde{e}[x := v']$ Relation

  Choose variables, $z \neq x$ and $z \notin \mathrm{fv}(v, v')$ by alpha-renaming.

- Other cases follows similarly.

□

Lemma F.4. (Breaking down Context E)

For all contexts E, uniquely decompose $E = E^+[E^*]$.

Uniqueness observation: Recall that E^* has no check contexts.

- If E does not have check, then it is $E^* = E$, and $E^+ = \square$.
- Else there is only a single decomposition to get out E^* without check contexts

Proof. By induction on E.

- Case $E = \square$

Choose $E^+ = E^* = \square$

- Case $E = E_1\ e$

  $E_1 = E_1^+[E_1^*]$ I.H.
  $E = E_1^+[E_1^*]\ e$
  $E = (E_1^+\ e)[E_1^*]$

  - Case $E_1^+ = \square$

    $E = E_1^*\ e$
    $E^+ = \square$ Choose
    $E^* = E_1^*\ e$ Choose

  - Case $E_1^+ \neq \square$

    $E^+ = E_1^+\ e$ Lemma F.5 ($K = \square\ e$) and Choose
    $E^* = E_1^*$ Choose

- Case $E = \mathsf{check}_j^k(E_1, v)$

  $E_1 = E_1^+[E_1^*]$ I.H.
  $E = \mathsf{check}_j^k(E_1^+[E_1^*], v) = \mathsf{check}_j^k(E_1^+, v)[E_1^*]$

  - Case $E_1^+ = \square$

    $E = \mathsf{check}_j^k(\square, v)[E_1^*]$
    $E^+ = \mathsf{check}_j^k(\square, v), E^* = E_1^*$ Choose

  - Case $E_1^+ \neq \square$

    $E_2^+ = \mathsf{check}_j^k(E_1^+, v)$ Lemma F.5 ($K = \mathsf{check}_j^k(\square, v)$)
    $E = E_2^+[E^*]$

□

Lemma F.5.

If $E_1^+ \neq \square$ then there is a E_2^+, $E_2^+ = K[E_1^+]$.

Where $K = \square\ e \mid v\ \square \mid \mathsf{mon}_j^{k,l}(\kappa, \square) \mid \mathsf{check}_j^k(\square, v)$

Proof.

By induction on the structure of E_1^+.

- Case $\square$ invalid.
- Case $E_3^+\left[E^*\left[\mathsf{check}_j^k(\square, v)\right]\right]$

  - Case $E_3^+ = \square$

    – Case $K = \mathsf{check}_{j_1}^{k_1}(\square, v_1)$

$K = E_4^+$ Definition of E^+

$K[E_1^+] = E_4^+[E^*[\mathsf{check}_j^k(\square, v)]]$

$E_2^+ = E_4^+[E^*[\mathsf{check}_j^k(\square, v)]]$ Choose

– Case $K \neq \mathsf{check}_{j_1}^{k_1}(\square, v_1)$

$K = E_4^*$ Definition of E^*

$K[E_1^+] = E_4^*[E^*[\mathsf{check}_j^k(\square, v)]] = E_5^*[\mathsf{check}_j^k(\square, v)]$

$E_2^+ = \square[E_5^*[\mathsf{check}_j^k(\square, v)]]$ Choose

▸ Case $E_3^+ \neq \square$

$E_4^+ = K[E_3^+]$ I.H.

$K[E_1^+] = K[E_3^+[E^*[\mathsf{check}_j^k(\square, v)]]] =$

$K[E_3^+][E^*[\mathsf{check}_j^k(\square, v)]] = E_4^+[E^*[\mathsf{check}_j^k(\square, v)]]$

$E_2^+ = E_4^+[E^*[\mathsf{check}_j^k(\square, v)]]$ Choose

□

Lemma F.6. (Compiler is subset of relation)

If $\Gamma \vdash e_1 : \tau \rightarrowtail^l e_2$ then $e_1 \sim e_2^*$.

In particular, for closed programs, if $\cdot \vdash e_1 : \tau \rightarrowtail^l e_2$ then $e_1 \sim e_2^*$.

In this lemma, we use e^* as shorthand to denote e untyped.

Proof.

By induction on structure of $\Gamma \vdash e_1 : \tau \rightarrowtail^l e_2$.

- Case

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau \rightarrowtail^l x}$$

$e_2^* = x$ Definition

$x \sim x$ Relation

- Case

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \rightarrowtail^l e'}{\Gamma \vdash \lambda x : \tau_1.e : \tau_1 \to \tau_2 \rightarrowtail^l \lambda x : \tau_1.e'}$$

$e_2^* = \lambda x.e'^*$ Definition

$e \sim e'^*$ I.H. on the premise with context $\Gamma, x : \tau_1$

$\lambda x.e \sim \lambda x.e'^*$ Relation

- Case

$$\frac{\Gamma \vdash e_3 : \tau \to \mathsf{bool} \rightarrowtail^{l} e_5 \qquad \Gamma \vdash e_4 : \tau \rightarrowtail^{l} e_6}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), e_4) : \tau \rightarrowtail^{l} \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_5, e_6 \rangle\rangle}$$

$e_2^* = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle e_5^*, e_6^* \rangle\rangle$ Definition
$e_3 \sim e_5^*$ I.H.
$e_4 \sim e_6^*$ I.H.
$e_1 \sim e_2^*$ Relation

- Case

$$\frac{\Gamma \vdash e_3 : \tau_2 \rightarrowtail^{l} e_4}{\Gamma \vdash \lambda f : \tau_1 \to \tau_2 . \lambda x : \tau_1 . \mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^{l} e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_1) : \tau_1 \to \tau_2 \rightarrowtail^{l} e_5\ e_4}$$

$e_3 \sim e_4^*$ I.H.
$\lambda f : \tau_1 \to \tau_2 . \lambda x : \tau_1 . \mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \sim e_5^*$ I.H.
$e_1 \sim e_2^*$ Relation

- Case

$$\frac{\Gamma \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail^{l} e_4}{\Gamma \vdash \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\Big(\kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\big], f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^{l} e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail^{l} e_5\ e_4}$$

$e_3 \sim e_4^*$ I.H.
$\lambda f . \lambda x . \mathsf{mon}_j^{k,l}\Big(\kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\big], f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \sim e_5^*$ I.H.
$e_1 \sim e_2^*$ Relation

- Other cases are similar.

□

F.3 Compiler Correctness

Figure 18 shows the structure of the proof. The source program e compiles to e', and the target program runs e' under wrap. After the handler is installed, the target reaches a state e_0'' with $\mathrm{EQ}(e, e_0'')$ (Lemma F.8). From then on, every source step $e_i \to e_{i+1}$ is matched by zero or more target steps $e_i'' \Longrightarrow^* e_{i+1}''$ that keep $\mathrm{EQ}(e_{i+1}, e_{i+1}'')$ (Lemma F.10). Chaining the squares (Lemma F.9), the source result r and the target result $\tilde{r}$ are related: a value goes to its related value, and blame_j^k goes to $\mathtt{error}_k$.

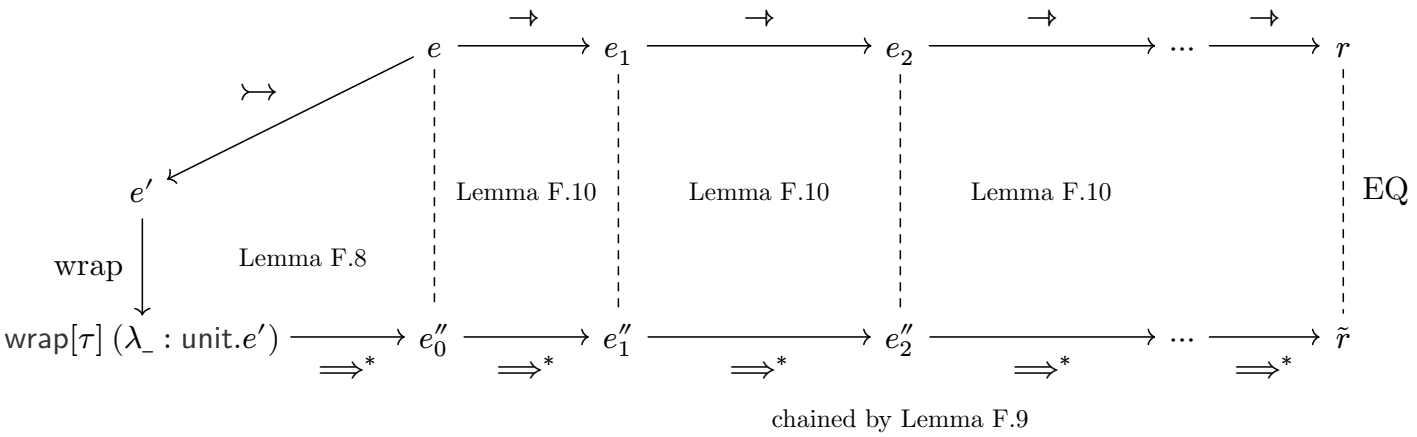


Figure 18: Structure of the compiler correctness proof. Dashed lines rel states by EQ.

Define a predicate between Contracts e and Untyped Effects e'.

$$\mathrm{EQ}(e, e') = \begin{cases} \text{true if } e = v \text{ and } e' = \tilde{v} \\ \text{true if } e' = \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{e}) \text{ and } e \neq E^*\left[\mathsf{check}^k_j(e'', v)\right] \\ \text{true if } e = E^+\left[E^*\left[\mathsf{check}^k_j(e_1, v)\right]\right] \text{ and} \\ \qquad e' = \tilde{E}^+\left[\text{if } e'_1 \text{ then } \left(\lambda x.\mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]\right)\right) \tilde{v} \text{ else } \mathtt{error}_j\right] \text{ and} \\ \qquad \mathrm{EQ}(e_1, e'_1) \\ \text{true if } e = E\left[\mathsf{blame}^k_j\right] \text{ and } e' = \mathtt{error}_k \\ \text{false otherwise} \end{cases}$$

Theorem F.7. (Compiler Correctness)

If $\cdot \vdash e : \tau \rightarrowtail^l e'$ and $e^* = \mathrm{untyped}(\mathsf{wrap}[\tau]\ (\lambda_ : \mathsf{unit}.e'))$ and $e \twoheadrightarrow^* b$ then $e^* \Longrightarrow^* b$.

Proof.

$e^* \Longrightarrow^* e'_1$	Lemma F.8
$\mathrm{EQ}(e, e'_1)$	Lemma F.8
$e'_1 \Longrightarrow^* b$	Lemma F.9
$e^* \Longrightarrow^* b$	Combining steps

□

Lemma F.8. (Wrapped program steps into an EQ state with source)

If $\cdot \vdash e : \tau \rightarrowtail^l e'$ and $e^*_1 = \mathsf{wrap}[\tau]\ (\lambda_ : \mathsf{unit}.e')$ then it exists e^*_2 such that $e^*_1 \Longrightarrow^* e^*_2$ and $\mathrm{EQ}(e, e^*_2)$.

Proof.

$e \sim e'^*$	Lemma F.6
$e^*_1 \Longrightarrow^* \mathtt{handler}_d(\mathtt{h}_{\mathscr{C}})\ \lambda_.e'^* \Longrightarrow^* \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, e'^*) = e^*_2$	

$\mathrm{EQ}(e, e_2^*)$ Definition

□

Lemma F.9. If $e \twoheadrightarrow^* r$ and $\mathrm{EQ}(e, e')$ then $e' \Longrightarrow^* \tilde{r}$.

Proof. By breaking down the $\twoheadrightarrow^*$, and induction on it.

- Case $v \twoheadrightarrow^* v$

  $e' = \tilde{r}$ Definition of $\mathrm{EQ}(e, e')$
  $e' \Longrightarrow^* \tilde{r}$ Reflexivity

- Case $\mathsf{blame}_j^k \twoheadrightarrow^* \mathsf{blame}_j^k$

  $r = \mathsf{blame}_j^k$ Case assumption
  $\tilde{r} = \mathtt{error}_j$ Relation
  $e' = \mathtt{error}_j = \tilde{r}$ Definition of $\mathrm{EQ}(e, e')$
  $e' \Longrightarrow^* \tilde{r}$ Reflexivity

- Case $e \twoheadrightarrow e_1$ and $e_1 \twoheadrightarrow^* r$

  $e' \Longrightarrow^* e_1'$ Lemma F.10
  $\mathrm{EQ}(e_1, e_1')$ Lemma F.10
  $e_1' \Longrightarrow^* \tilde{r}$ I.H.
  $e' \Longrightarrow^* \tilde{r}$ Chaining steps

□

Lemma F.10. If $e_1 \twoheadrightarrow e_2$ and $\mathrm{EQ}(e_1, e_1')$ then there exists e_2' such that $e_1' \Longrightarrow^* e_2'$ and $\mathrm{EQ}(e_2, e_2')$.

Proof of Lemma F.10. By case analysis on $e_1 \twoheadrightarrow e_2$ with respect to $\mathrm{EQ}(e_1, e_1')$.

- Case $E\left[\mathsf{blame}_j^k\right] \twoheadrightarrow \mathsf{blame}_j^k$

  $E = E^+[E^*]$ Lemma F.4

  ▸ Case $E^+ = \square$

  $E^*\left[\mathsf{blame}_j^k\right] \sim \tilde{E}^*[\mathtt{error}_k]$ Lemma F.2
  $\tilde{E}^*[\mathtt{error}_k] \Longrightarrow \mathtt{error}_k$
  $\mathsf{blame}_j^k \sim \mathtt{error}_k$ Definition
  $\mathrm{EQ}\left(\mathsf{blame}_j^k, \mathtt{error}_k\right)$ EQ Definition
  $e_2' = \mathtt{error}_k$ Choose

  ▸ Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

  $E\left[\mathsf{blame}_j^k\right] = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(\mathsf{blame}_j^k, v_1\right)\right]\right]$

$\mathsf{blame}^k_j \sim \mathtt{error}_k$ Definition
$e_1 \sim \tilde{E}_1^+\left[\text{if } \mathtt{error}_k \text{ then } \left(\lambda x.\mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}_1^*[x]\right)\right) \tilde{v} \text{ else } \mathtt{error}_{k_1}\right]$
$e_1' \Longrightarrow \mathtt{error}_k$
$\mathrm{EQ}\left(\mathsf{blame}^k_j, \mathtt{error}_k\right)$ EQ Definition
$e_2' = \mathtt{error}_k$ Choose

- Case $E[e_3] \to E[e_4]$ where $e_3 \dashrightarrow e_4$

  Overview of proof strategy: breaking down $E = E^+[E^*]$ using Lemma F.4 and case on E^+, then try to match $\mathrm{EQ}(e_1, e_1')$ cases. EQ case 2 holds if and only if $E^+ = \square$, and case 3 holds if and only if $E^+ \neq \square$ (other case) easily observed the form of e_1 in EQ. By step it should arrive at the next state that matches EQ predicate cases.

  During the reduction, we likely encounter $\mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[e]\right)$. By Lemma F.11, the handler invoked is obviously the current handler.

  - Case $e_3 = \mathsf{mon}^{k,l}_j(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}^k_j(e\ v, v) = e_4$

    Actually need $e \neq E'\left[\mathsf{check}^{k'}_{j'}(e', v')\right]$ to obtain $\tilde{e}$. But the proof goes through without stating this fact because we inverse from EQ predicate. When running, this e is never a check, because we don't evaluate inside flat contract.

    $E = E^+[E^*]$ Lemma F.4
    $e_1 = E^+[E^*[e_3]]$ Substitution

    – Case $E^+ = \square$

    $E^* \sim \tilde{E}^*$ Lemma F.2
    $e_3 \sim \tilde{e}_3$ Lemma F.2
    $\tilde{e}_1 = \tilde{E}^*[\tilde{e}_3]$ Lemma F.2
    $e_1 = E^*[e_3] \sim \tilde{E}^*[\mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle]$ Relation
    $e_1' = \mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle]\right)$ Case 2
    $e_1' \Longrightarrow^*$
    $\text{if } \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{e}\ \tilde{v}) \text{ then } \left(\lambda x.\mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]\right)\right) \tilde{v} \text{ else } \mathtt{error}_j$
    $e_2' =$ Choose
    $\text{if } \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{e}\ \tilde{v}) \text{ then } \left(\lambda x.\mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]\right)\right) \tilde{v} \text{ else } \mathtt{error}_j$
    $\mathrm{EQ}(e\ v, \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{e}\ \tilde{v}))$ Case 2
    $\mathrm{EQ}(e_2, e_2')$ Case 3

    – Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}^{k_1}_{j_1}(\square, v_1)\right]\right]$

    $e_1 = E_1^+\left[E_1^*\left[\mathsf{check}^{k_1}_{j_1}(E^*[e_3], v_1)\right]\right]$
    $e_3 \sim \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle = \tilde{e}_3$
    $\mathrm{EQ}\left(E^*[e_3], \mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_3]\right)\right)$ Case 2, by shape

$\mathrm{EQ}\big(e_1, \tilde{E}_1^+\big[\text{if } \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_3]) \text{ then } (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}_1^*[x]))\ \tilde{v}_1 \text{ else } \mathtt{error}_{j_1}\big]\big)$
$e_1' = \tilde{E}_1^+\big[\text{if } \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_3]) \text{ then } (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}_1^*[x]))\ \tilde{v}_1 \text{ else } \mathtt{error}_{j_1}\big]$
Identified e_1' based on the condition of EQ case 3
$e_1' \Longrightarrow^* \tilde{E}_2^+\big[\text{if } \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{e}\ \tilde{v}) \text{ then } (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]))\ \tilde{v} \text{ else } \mathtt{error}_j\big]$
$e_2' = \tilde{E}_2^+\big[\text{if } \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{e}\ \tilde{v}) \text{ then } (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]))\ \tilde{v} \text{ else } \mathtt{error}_j\big]$ Choose
where $\tilde{E}_2^+ = \tilde{E}_1^+\big[\text{if } \square \text{ then } (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}_1^*[x]))\ \tilde{v}_1 \text{ else } \mathtt{error}_{j_1}\big]$
$e_2 = E_1^+\big[E_1^*\big[\mathsf{check}_{j_1}^{k_1}\big(E^*\big[\mathsf{check}_j^k(e\ v, v)\big], v_1\big)\big]\big]$
$\mathrm{EQ}\big(E^*\big[\mathsf{check}_j^k(e\ v, v)\big], \text{if } \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{e}\ \tilde{v}) \text{ then } (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]))\ \tilde{v} \text{ else } \mathtt{error}_j\big)$
$\mathrm{EQ}(e_2, e_2')$ Case 3

(Case 2 is invalid, because of the shape of e_2)

- Case $e_3 = \mathsf{check}_j^k(\mathsf{true}, v) \dashrightarrow v = e_4$

$E = E^+[E^*]$ Lemma F.4
$e_1 = E^+[E^*[e_3]]$ Substitution

– Case $E^+ = \square$

Let $e_5 = (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]))\ \tilde{v}$.

$e_1' = \text{if } e_3' \text{ then } e_5 \text{ else } \mathtt{error}_j$ $\mathrm{EQ}(e_1, e_1')$, case 3
$\mathrm{EQ}(\mathsf{true}, e_3')$ Since true is a value
$e_3' = \mathsf{true}$ Definition of EQ
$e_1' \Longrightarrow^* \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{v}]) = e_2'$
$e_2 = E^*[v]$
$\mathrm{EQ}(e_2, e_2')$ Case 2

– Case $E^+ = E_1^+\big[E_1^*\big[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\big]\big]$

Let $e_5 = (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]))\ \tilde{v}$ and $e_6 = (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}_1^*[x]))\ \tilde{v}_1$.

$e_1' =$ $\mathrm{EQ}(e_1, e_1')$, case 3
$\tilde{E}_1^+\big[\text{if if } e_3' \text{ then } e_5 \text{ else } \mathtt{error}_j \text{ then } e_6 \text{ else } \mathtt{error}_{j_1}\big]$
$\mathrm{EQ}(\mathsf{true}, e_3')$ Since true is a value
$e_3' = \mathsf{true}$ Definition of EQ
$e_1' \Longrightarrow^* \tilde{E}_1^+\big[\text{if } \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{v}]) \text{ then } e_6 \text{ else } \mathtt{error}_{j_1}\big] =$
e_2'
$e_2 = E_1^+\big[E_1^*\big[\mathsf{check}_{j_1}^{k_1}(E^*[v], v_1)\big]\big]$
$\mathrm{EQ}\big(E^*[v], \mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{v}])\big)$ Case 2
$\mathrm{EQ}(e_2, e_2')$ Case 3

(Case 2 is invalid, because of the shape of e_2.)

- Case $e_3 = \mathsf{check}_j^k(\mathsf{false}, v) \dashrightarrow \mathsf{blame}_j^k = e_4$

  $E = E^+[E^*]$ — Lemma F.4

  $e_1 = E^+[E^*[e_3]]$ — Substitution

  – Case $E^+ = \square$

    $e_1 = E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]$

    $e_1' =$

  – Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

    $e_1 = E^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]\right]$

    $e_1' =$ — $\mathrm{EQ}(e_1, e_1')$

    $\tilde{E}^+\left[\text{if false then } \left(\lambda x.\mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[x]\right)\right)\tilde{v} \text{ else } \mathtt{error}_j\right]$

    $e_1' \Longrightarrow^* \tilde{E}^+\left[\mathtt{error}_j\right] = e_2'$ — Reduction

    $e_2 = E^+\left[E^*\left[\mathsf{blame}_j^k\right]\right]$ — Source reduction

    $\tilde{E}^+\left[\mathtt{error}_j\right] \Longrightarrow \mathtt{error}_j$ — **Step-E-Error**

    $\mathsf{blame}_j^k \sim \mathtt{error}_j$ — Relation

    $\mathrm{EQ}(e_2, e_2')$ — Definition

- Case $e_3 = \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) = e_4$

  $E = E^+[E^*]$ — Lemma F.4

  $e_1 = E^+[E^*[e_3]]$ — Substitution

  – Case $E^+ = \square$

    Let $e_5 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) = \lambda f.\lambda x.e_{5a}$

    $E^* \sim \tilde{E}^*$ — Lemma F.2

    $e_3 \sim \tilde{e}_3$ — Lemma F.2

    $\tilde{e}_1 = \tilde{E}^*[\tilde{e}_3]$ — Lemma F.2

    $e_1 = E^*[e_3] \sim \tilde{E}^*[(\lambda f.\lambda x.\tilde{e}_{5a})\ \tilde{v}]$ — Relation

    $e_1' = \mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[(\lambda f.\lambda x.\tilde{e}_{5a})\ \tilde{v}]\right)$ — $\mathrm{EQ}(e_1, e_1')$, case 2

    $e_1' \Longrightarrow^* \mathtt{handle}_d\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\lambda x.\tilde{e}_{5a}[f := \tilde{v}]]\right) = e_2'$ — Reduction

    $e_2 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right]$

    $e_6 = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$ — Let

    $e_6 = e_5[f := v]$ — Substitution

    $\tilde{e}_6 = \tilde{e}_5[f := \tilde{v}]$ — Lemma F.3

    $\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \sim \tilde{e}_6 = \tilde{e}_5[f := \tilde{v}]$ — Definition

    $E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right] \sim \tilde{E}^*[\tilde{e}_5[f := \tilde{v}]]$ — Lemma F.1

    $\mathrm{EQ}(e_2, e_2')$ — Case 2

  – Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

Let $e_5 = \lambda f.\lambda x.e_{5a}$, where $e_{5a} = \mathsf{mon}_j^{k,l}\big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big)$.

Let $\tilde{E}_2^+ = \tilde{E}_1^+\big[\text{if } \square \text{ then } (\lambda x.\mathtt{handle}_d(\mathtt{h}_{\mathscr{C}}, \tilde{E}_1^*[x]))\ \tilde{v}_1 \text{ else } \mathtt{error}_{j_1}\big]$

$e_1' = \tilde{E}_2^+\big[\mathtt{handle}_d\big(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_5\ \tilde{v}]\big)\big]$

$e_1' \Longrightarrow^* \tilde{E}_2^+\big[\mathtt{handle}_d\big(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\lambda x.\tilde{e}_{5a}[f := \tilde{v}]]\big)\big] = e_2'$

$e_2 = E_1^+\big[E_1^*\big[\mathsf{check}_{j_1}^{k_1}\big(E^*\big[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\big], v_1\big)\big]\big]$

$e_6 = \lambda x.\mathsf{mon}_j^{k,l}\big(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big)$ — Let

$e_6 = e_{5a}[f := v]$ — Substitution

$\tilde{e}_6 = \tilde{e}_{5a}[f := \tilde{v}]$ — Lemma F.3

$\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \sim \tilde{e}_6 = \tilde{e}_{5a}[f := \tilde{v}]$ — Definition

$E^*\big[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\big] \sim \tilde{E}^*[\tilde{e}_{5a}[f := \tilde{v}]]$ — Lemma F.1

$\mathrm{EQ}\big(E^*\big[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\big], \mathtt{handle}_d\big(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_{5a}[f := \tilde{v}]]\big)\big)$ — Case 1

$\mathrm{EQ}(e_2, e_2')$ — Case 3

- Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big) = e_4$

  $E = E^+[E^*]$ — Lemma F.4

  $e_1 = E^+[E^*[e_3]]$ — Substitution

  – Case $E^+ = \square$

  Let $e_5 = \lambda f.\lambda x.e_{5a}$, where $e_{5a} = \mathsf{mon}_j^{k,l}\big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big)$.

  $E^* \sim \tilde{E}^*$ — Lemma F.2

  $e_3 \sim \tilde{e}_3$ — Lemma F.2

  $\tilde{e}_1 = \tilde{E}^*[\tilde{e}_3]$ — Lemma F.2

  $\tilde{e}_3 = \tilde{e}_5\ \tilde{v}_2$ — Relation

  $e_1' = \mathtt{handle}_d\big(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_3]\big)$ — $\mathrm{EQ}(e_1, e_1')$, case 2

  $e_1' \Longrightarrow^* e_1' = e_2'$ — Reflexivity

  $\tilde{e}_4 = \tilde{e}_5\ \tilde{v}_2$ — Relation

  $E^*[e_4] \sim \tilde{E}^*[\tilde{e}_4]$ — Lemma F.1

  $\mathrm{EQ}(e_2, e_2')$ — Case 2

  – Case $E^+ = E_1^+\big[E_1^*\big[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\big]\big]$

  Same as the case $E^+ = \square$, inside $\tilde{E}^+$, by case 3.

- Case $e_3 = \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) = e_4$

  Similar to its functional variant.

- Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big) = e_4$ where $\kappa_3 = \kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\big]$

  Similar to its functional variant.

□

Lemma F.11. ($\tilde{E}^*$ does not have a handler)

If $E^* \sim \tilde{E}^*$, then $\mathscr{C} \notin \mathtt{bop}(\tilde{E}^*)$.

Proof.

By induction on the structure of $E^* \sim \tilde{E}^*$. Trivial, the related context has no handler.□

CHAPTER G

Contracts to Effects Compiler Correctness (Shallow Handler)

G.1 Overview

Proof for a compiler from Contracts to Effects using shallow handler, where the continuation does not reinstall the handler.

$$\mathtt{h}_{\mathscr{C}} : \mathtt{lab} \quad = \quad \{\mathscr{C} \to \forall\alpha.\langle \mathtt{blab}, \langle \alpha \to \mathsf{bool}, \alpha\rangle\rangle \to \alpha\}$$

$$\mathsf{H} \quad = \quad \mu h.\mathtt{handler}_s(\{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.h\ (\lambda_.\ \text{if } e\ v \text{ then } k\ v \text{ else } \mathtt{error}_l)\})$$

$$\mathsf{wrap} \quad = \quad \Lambda\alpha.\ \lambda f : \mathsf{unit} \to \alpha.\ \mathsf{H}\ f$$

where $\lambda\langle l, \langle e, v\rangle\rangle$ deconstructs the argument as tuple pattern

The handler clause re-installs H around the rest of the program by itself, instead of the continuation doing it. So the whole program always runs under exactly one $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \cdot)$ at the top, and the predicate of a check runs under the same handler as the program. We write $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e)$ for the program running under the handler of H.

Compared to the deep semantics, there are no states without handle, and there is no nested handle for predicates. This makes the relation much simpler.

G.2 Relation

Before we define relations that will be used in the simulation proof below. We define:

- Context E^* is any context without $\mathsf{check}^k_j(e, v)$ in Contracts.
- Context $E^+ = \Box \mid E^+\left[E^*\left[\mathsf{check}^k_j(\Box, v)\right]\right]$

$R_v \subseteq \mathsf{Contracts}\ v \times \mathtt{Effects}\ v$

Abuse of notation: If $(v, v') \in R_v$, we write $v \sim_{R_v} v'$ or $v \sim v'$. We write $\tilde{v}$ for any value $\tilde{v}$ satisfying $v \sim \tilde{v}$.

$$
\begin{array}{rcl}
x & \sim & x \\
b & \sim & b \\
N & \sim & N \\
\lambda x : \tau . e & \sim & \lambda x : \tau^* . \tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x : \tau . \mathsf{mon}_j^{k,l}\left(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x : \tau . \mathsf{mon}_j^{k,l}\left(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
 & & \kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]
\end{array}
$$

$R_e \subseteq \mathsf{Contracts}\ e \times \mathtt{Effects}\ e$

Abuse of notation: If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We write $\tilde{e}$ for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

$$
\begin{array}{rcl}
v & \sim & \tilde{v} \\
e_1\ e_2 & \sim & \tilde{e}_1\ \tilde{e}_2 \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) & \sim & \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2 \rangle \rangle \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\left(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
 & & \kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right] \\
\mathsf{blame}_j^k & \sim & \mathtt{error}_k
\end{array}
$$

$R_{E^*} \subseteq \mathsf{Contracts}\ E^* \times \mathtt{Effects}\ E^*$

Abuse of notation: If $(E^*, E'^*) \in R_{E^*}$, we write $E^* \sim_{R_{E^*}} E'^*$ or $E^* \sim E'^*$. We write $\tilde{E}^*$ for any expression $\tilde{E}^*$ satisfying $E^* \sim \tilde{E}^*$.

$$
\begin{array}{rcl}
\square & \sim & \square \\
E^*\ e & \sim & \tilde{E}^*\ \tilde{e} \\
v\ E^* & \sim & \tilde{v}\ \tilde{E}^* \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), E^*) & \sim & \texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{E}^* \rangle \rangle \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E^*) & \sim & \tilde{e}_1\ \tilde{E}^* \\
 & & e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, E^*) & \sim & \tilde{e}_1\ \tilde{E}^* \\
 & & e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
 & & \kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]
\end{array}
$$

$\boxed{R_{E^+} \subseteq \mathsf{Contracts}\ E^+ \times \texttt{Effects}\ E^+}$

Abuse of notation: If $(E^+, E'^+) \in R_{E^+}$, we write $E^+ \sim_{R_{E^+}} E'^+$ or $E^+ \sim E'^+$. We write $\tilde{E}^+$ for any expression $\tilde{E}^+$ satisfying $E^+ \sim \tilde{E}^+$.

Note: This relation is different from deep handler compiler proof.

$$
\begin{array}{rcl}
\square & \sim & \square \\
E^+\left[E^*\left[\mathsf{check}_j^k(\square, v)\right]\right] & \sim & \text{if } \square \text{ then } \left(\lambda x.\tilde{E}^+\left[\tilde{E}^*[x]\right]\right)\ \tilde{v} \text{ else } \texttt{error}_k
\end{array}
$$

Lemma G.1. (Relation is closed)

If $E^* \sim \tilde{E}^*$, $e \sim \tilde{e}$ and e has no check in evaluation position, then $E^*[e] \sim \tilde{E}^*[\tilde{e}]$.

Proof. By induction on E. Follows similarly to Lemma F.1 because the relations (R_e, R_E) is same.

□

Lemma G.2. (Relation is closed under substitution) If $e \sim \tilde{e}$ and $v \sim \tilde{v}$ then $e[x := v] \sim \tilde{e}[x := \tilde{v}]$.

Proof. By induction on $e \sim e'$. Follows Lemma F.3 because relations (R_v, R_e) are the same. □

Lemma G.3. (Compiler is subset of relation)

If $\Gamma \vdash e_1 : \tau \rightarrowtail^l e_2$ then $e_1 \sim e_2^*$.

Proof.

By induction on structure of $\Gamma \vdash e_1 : \tau \rightarrowtail^l e_2$.

- Case

$$\frac{\Gamma \vdash e_3 : \tau \to \mathsf{bool} \rightarrowtail^l e_5 \qquad \Gamma \vdash e_4 : \tau \rightarrowtail^l e_6}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), e_4) : \tau \rightarrowtail^l \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_5, e_6 \rangle\rangle}$$

$e_2^* = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle e_5^*, e_6^* \rangle\rangle$
$e_3 \sim e_5^*$ I.H.
$e_4 \sim e_6^*$ I.H.
$e_1 \sim e_2^*$

- Case

$$\frac{\Gamma \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail^l e_4 \qquad \Gamma \vdash \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^l e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail^l e_5\ e_4}$$

$e_2^* = e_5^*\ e_4^*$
$e_3 \sim e_4^*$ I.H.
$\lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \sim e_5^*$ I.H.
$e_1 \sim e_2^*$

- Case

$$\frac{\Gamma \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail^l e_4 \qquad \Gamma \vdash \lambda f.\lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^l e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail^l e_5\ e_4}$$

$e_2^* = e_5^*\ e_4^*$
$e_3 \sim e_4^*$ I.H.
$\lambda f.\lambda x.\left(\lambda y.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)\right)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) \sim e_5^*$ I.H.
$e_1 \sim e_2^*$

- Other cases follow similarly, by I.H.

□

Lemma G.4. (Relation decomposition through check-free contexts)

If $E^*[e] \sim e'$, then there exist E'^* and e'_0 such that $E^* \sim E'^*$ and $e \sim e'_0$ and $e' = E'^*[e'_0]$.

Proof.

By induction on the check-free context E^*. Follows Lemma F.2 because relations (R_E, R_e) are the same. □

Lemma G.5. (Breaking down Context E)

For all contexts E, uniquely decompose $E = E^+[E^*]$.

Uniqueness observation: Recall that E^* has no check contexts.

- If E does not have check, then it is $E^* = E$, and $E^+ = \square$.
- Else there is only a single decomposition to get out E^* without check contexts

Proof. By induction on the structure of E. Follows Lemma F.4 because the contexts E, E^+, E^* are the same. □

Lemma G.6.

If $E_1^+ \neq \square$ then there is a E_2^+, $E_2^+ = K[E_1^+]$.

Where $K = \square\, e \mid v\, \square \mid \mathsf{mon}_j^{k,l}(\kappa, \square) \mid \mathsf{check}_j^k(\square, v)$

Proof. By induction on the structure of E_1^+. Follows Lemma F.5 because the context E^+ is the same. □

G.3 Compiler Correctness

Figure 19 shows the structure of the proof. The source program e compiles to e', and the target program runs e' under wrap. After the handler is installed, the target reaches a state e''_0 with $\mathrm{EQ}(e, e''_0)$ (Lemma G.8). From then on, every source step $e_i \twoheadrightarrow e_{i+1}$ is matched by zero or more target steps $e''_i \Longrightarrow^* e''_{i+1}$ that keep $\mathrm{EQ}(e_{i+1}, e''_{i+1})$ (Lemma G.10). Chaining the squares (Lemma G.9), the source result r and the target result $\tilde{r}$ are related: a value goes to its related value, and blame_j^k goes to $\mathtt{error}_k$.

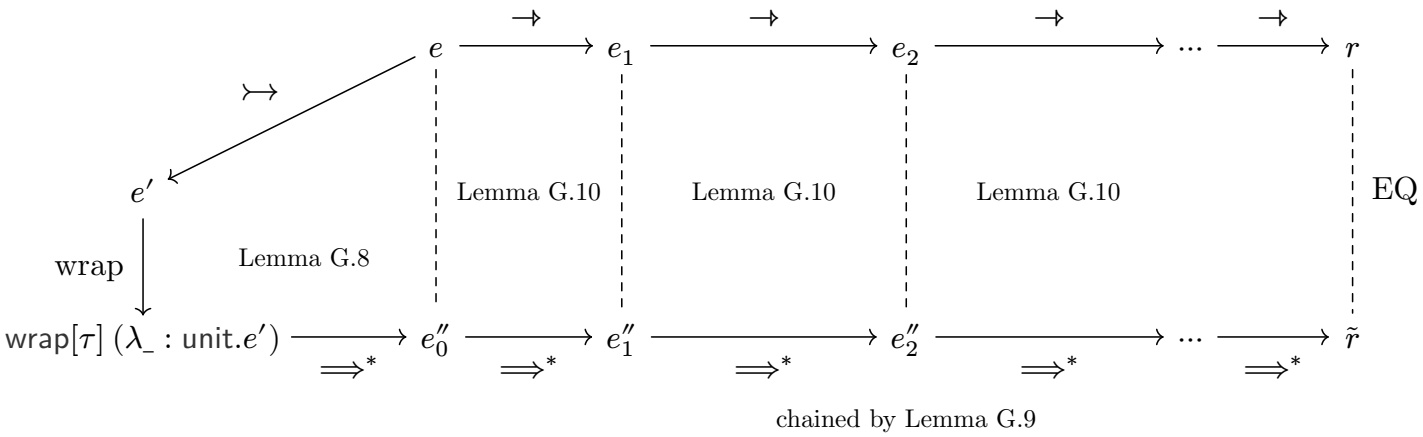


Figure 19: Structure of the compiler correctness proof. Dashed lines rel states by EQ.

Define a predicate between Contracts e and `Effects` e'.

$$\mathrm{EQ}(e, e') = \begin{cases} \text{true if } e \sim e' \text{ and } e \neq E^*\left[\mathsf{check}^{k_1}_{j_1}(e_1, v_1)\right] \\ \text{true if } e = E^+\left[E^*\left[\mathsf{check}^{k_1}_{j_1}(e_1, v_1)\right]\right] \text{ and} \\ \qquad e' = \text{if } \tilde{e}_1 \text{ then } \left(\lambda x.\tilde{E}^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_1 \text{ else } \mathtt{error}_{k_1} \\ \text{true if } e = E\left[\mathsf{blame}^k_j\right] \text{ and } e' = \mathtt{error}_k \\ \text{false otherwise} \end{cases}$$

Theorem G.7. If $\cdot \vdash e : \tau \rightarrowtail^l e'$ and $e^* = \mathrm{untyped}(\mathsf{wrap}[\tau]\ (\lambda_ : \mathsf{unit}.e'))$ and $e \twoheadrightarrow^* r$ then $e^* \Longrightarrow^* \tilde{r}$.

Where r is a value v or blame^k_j.

Proof.

$e^* \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1^*)$	Lemma G.8
$\mathrm{EQ}(e, e_1^*)$	Lemma G.8
$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1^*) \Longrightarrow^* \tilde{r}$	Lemma G.9
$e^* \Longrightarrow^* \tilde{r}$	

□

Lemma G.8. (Wrapped compiled program steps to an equivalent state with source program)

If $\cdot \vdash e : \tau \rightarrowtail^l e'$ and $e_1^* = \mathrm{untyped}(\mathsf{wrap}[\tau]\ (\lambda_ : \mathsf{unit}.e'))$ then it exists e_2^* such that $e_1^* \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_2^*)$ and $\mathrm{EQ}(e, e_2^*)$.

Proof.

$e \sim e'^*$ — Lemma G.3
$\mathrm{EQ}(e, e'^*)$ — Case 1
$e_1^* \Longrightarrow^* \mathsf{H}\, \lambda_.e'^* \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e'^*)$
$e_2^* = e'^*$ — Choose

□

Lemma G.9. If $e \twoheadrightarrow^* r$ and $\mathrm{EQ}(e, e')$ then $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e') \Longrightarrow^* \tilde{r}$.

Proof. By induction on $e \twoheadrightarrow^* r$

- Case $r \twoheadrightarrow^* r$
  - Case $v \twoheadrightarrow^* v$

    $e' = \tilde{v}$ — Case 1 EQ
    $e' \Longrightarrow^* \tilde{v}$ — Reflexivity
  - Case $\mathsf{blame}_j^k \twoheadrightarrow^* \mathsf{blame}_j^k$

    $e' = \mathtt{error}_k$ — Case 1 or 3 by EQ
    $e' \Longrightarrow^* \mathtt{error}_k$ — Reflexivity
- Case $e \twoheadrightarrow e_1$ and $e_1 \twoheadrightarrow^* r$

  $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e') \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1')$ — Lemma G.10
  $\mathrm{EQ}(e_1, e_1')$ — Lemma G.10
  $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \tilde{r}$ — I.H.
  $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e') \Longrightarrow^* \tilde{r}$

□

Lemma G.10. If $e_1 \twoheadrightarrow e_2$ and $\mathrm{EQ}(e_1, e_1')$ then there exists e_2' such that $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_2')$ and $\mathrm{EQ}(e_2, e_2')$.

Proof of Lemma G.10.

By case analysis on $e_1 \twoheadrightarrow e_2$ with respect to $\mathrm{EQ}(e_1, e_1')$.

- Case $E\left[\mathsf{blame}_j^k\right] \twoheadrightarrow \mathsf{blame}_j^k$

  $e_1' = \mathtt{error}_k$ — Case 3 EQ
  $e_1' \Longrightarrow^* \mathtt{error}_k$ — Reflexivity
  $\mathrm{EQ}(e_2, \mathtt{error}_k)$
  $e_2' = \mathtt{error}_k$ — Choose
- Case $E[e_3] \twoheadrightarrow E[e_4]$ and $e_3 \dashrightarrow e_4$ is a redex

Proof Sketch: $E = E^+\left[E^*\left[\mathsf{check}^k_j(\square, v)\right]\right]$, then casing on E^+ structure. At the same time, EQ predicate will allow one case analysis $(e_1 ? = E'\left[\mathsf{check}^{k'}_{j'}(e', v')\right])$. If $E^+ = \square$, use Lemma G.4 to find equivalent $\tilde{E}^*$, else, by EQ case 2 identify the shape of e_1' by E^+ relation.

- Case $e_3 = \mathsf{mon}^{k,l}_j(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}^k_j(e\ v, v) = e_4$

  Actually need $e \neq E'\left[\mathsf{check}^{k'}_{j'}(e', v')\right]$ to obtain $\tilde{e}$. But the proof goes through without stating this fact because we inverse from EQ predicate. When running, this e is never a check, because we don't evaluate inside flat contract.

  $e_1 = E\left[\mathsf{mon}^{k,l}_j(\mathsf{flat}(e), v)\right] = E^+\left[E^*\left[\mathsf{mon}^{k,l}_j(\mathsf{flat}(e), v)\right]\right]$ Lemma G.5

  – Case $E^+ = \square$

  $e_1 = E^*\left[\mathsf{mon}^{k,l}_j(\mathsf{flat}(e), v)\right]$
  $E^* \sim \tilde{E}^*$ Lemma G.4
  $e_3 \sim \tilde{e}_3$ Lemma G.4
  $\tilde{e}_3 = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle$ Inversion
  $e_1' = \tilde{E}^*[\mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle]$ Case 1 EQ
  $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathsf{H}\left(\text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}^*[x]\right)\tilde{v} \text{ else } \mathtt{error}_k\right)$
  $\Longrightarrow^* \mathtt{handle}_s\left(\mathtt{h}_{\mathscr{C}}, \left(\text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}^*[x]\right)\tilde{v} \text{ else } \mathtt{error}_k\right)\right)$
  $e_2 = E^+\left[E^*\left[\mathsf{check}^k_j(e\ v, v)\right]\right]$
  $\mathrm{EQ}\left(e_2, \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}^*[x]\right)\tilde{v} \text{ else } \mathtt{error}_k\right)$ Case 2 EQ
  $e_2' = \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}^*[x]\right)\tilde{v} \text{ else } \mathtt{error}_k$ Choose

  – Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}^{k_1}_{j_1}(\square, v_1)\right]\right]$

  $e_2 =$
  $E_1^+\left[E_1^*\left[\mathsf{check}^{k_1}_{j_1}\left(E^*\left[\mathsf{check}^k_j(e\ v, v)\right], v_1\right)\right]\right] = E_2^+\left[E^*\left[\mathsf{check}^k_j(e\ v, v)\right]\right]$
  $\tilde{E}_2^+ = \text{if } \square \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$
  $E_2^+\left[E^*\left[\mathsf{check}^k_j(\square, v)\right]\right] \sim \text{if } \square \text{ then } \left(\lambda x.\tilde{E}_2^+\left[\tilde{E}^*[x]\right]\right)\tilde{v} \text{ else } \mathtt{error}_k$
  $e_1' = \text{if } \tilde{E}^*[\tilde{e}_3] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$ Case 2 EQ
  $\tilde{e}_3 = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle$ Inversion
  $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1')$
  $\Longrightarrow^* \mathtt{handle}_s\Big(\mathtt{h}_{\mathscr{C}},$

  $\text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\ \text{if } \tilde{E}^*[x] \text{ then } \left(\lambda y.\tilde{E}_1^+\left[\tilde{E}_1^*[y]\right]\right)\tilde{v}_1 \text{ else } \mathtt{error}_{k_1}\right)\tilde{v} \text{ else } \mathtt{error}_k\Big)$

  $= \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}_2^+\left[\tilde{E}^*[x]\right]\right)\tilde{v} \text{ else } \mathtt{error}_k$
  $\mathrm{EQ}\left(e_2, \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}_2^+\left[\tilde{E}^*[x]\right]\right)\tilde{v} \text{ else } \mathtt{error}_k\right)$ Case 2 EQ
  $e_2' = \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}_2^+\left[\tilde{E}^*[x]\right]\right)\tilde{v} \text{ else } \mathtt{error}_k$ Choose

- Case $e_3 = \mathsf{check}^k_j(\mathsf{true}, v) \dashrightarrow v = e_4$

Due to the restriction of EQ, only EQ case 2 matches this case.

$e_1 = E\left[\mathsf{check}_j^k(\mathsf{true}, v)\right] = E^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{true}, v)\right]\right]$ Given and Lemma G.5
$e_2 = E^+[E^*[v]]$

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{check}_j^k(\mathsf{true}, v)\right]$
$e_2 = E^*[v]$
$e_1' = \mathsf{if}\ \mathsf{true}\ \mathsf{then}\ \left(\lambda x.\tilde{E}^*[x]\right)\tilde{v}\ \mathsf{else}\ \mathtt{error}_k$ EQ case 2
$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathtt{handle}_s\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{v}]\right)$
$E^*[v] \sim \tilde{E}^*[\tilde{v}]$ Lemma G.1
$\mathrm{EQ}\left(e_2, \tilde{E}^*[\tilde{v}]\right)$ EQ case 1
$e_2' = \tilde{E}^*[\tilde{v}]$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(E^*[v], v_1)\right]\right]$
$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{check}_j^k(\mathsf{true}, v)\right], v_1\right)\right]\right] =$
$E_2^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{true}, v)\right]\right]$
$e_1' = \mathsf{if}\ \mathsf{true}\ \mathsf{then}\ \left(\lambda y.\tilde{E}_2^+\left[\tilde{E}^*[y]\right]\right)\tilde{v}\ \mathsf{else}\ \mathtt{error}_k$ Case 2 EQ
$E_2^+ \sim \mathsf{if}\ \square\ \mathsf{then}\ \left(\lambda x.\tilde{E}_1^+[E_1^*[x]]\right)\tilde{v}_1\ \mathsf{else}\ \mathtt{error}_{k_1} = \tilde{E}_2^+$
$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathtt{handle}_s\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}_2^+\left[\tilde{E}^*[\tilde{v}]\right]\right)$
$E^*[v] \sim \tilde{E}^*[\tilde{v}]$ Lemma G.1
$\mathrm{EQ}\left(e_2, \mathsf{if}\ \tilde{E}^*[\tilde{v}]\ \mathsf{then}\ \left(\lambda x.\tilde{E}_1^+[E_1^*[x]]\right)\tilde{v}_1\ \mathsf{else}\ \mathtt{error}_{k_1}\right)$ Case 2 EQ
$\mathrm{EQ}\left(e_2, \tilde{E}_2^+\left[\tilde{E}^*[\tilde{v}]\right]\right)$ Replace with $\tilde{E}_2^+$
$e_2' = \tilde{E}_2^+\left[\tilde{E}^*[\tilde{v}]\right]$ Choose

▸ Case $e_3 = \mathsf{check}_j^k(\mathsf{false}, v) \dashrightarrow \mathsf{blame}_j^k = e_4$

$e_1 = E\left[\mathsf{check}_j^k(\mathsf{false}, v)\right] = E^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]\right]$ Given and Lemma G.5
$e_2 = E^+\left[E^*\left[\mathsf{blame}_j^k\right]\right]$

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]$
$e_2 = E^*\left[\mathsf{blame}_j^k\right]$
$e_1' = \mathsf{if}\ \mathsf{false}\ \mathsf{then}\ \left(\lambda x.\tilde{E}^*[x]\right)\tilde{v}\ \mathsf{else}\ \mathtt{error}_k$ EQ case 2
$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{error}_k)$
$\mathrm{EQ}(e_2, \mathtt{error}_k)$ Case 3 EQ
$e_2' = \mathtt{error}_k$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{blame}_j^k\right], v_1\right)\right]\right]$

$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right], v_1\right)\right]\right] =$
$E_2^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]\right]$
$e_1' = \mathsf{if}\ \mathsf{false}\ \mathsf{then}\ \left(\lambda y.\tilde{E}_2^+\left[\tilde{E}^*[y]\right]\right)\tilde{v}\ \mathsf{else}\ \mathtt{error}_k$ Case 2 EQ
$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{error}_k)$
$\mathrm{EQ}(e_2, \mathtt{error}_k)$ Case 3 EQ
$e_2' = \mathtt{error}_k$ Choose

▸ Case $\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)$

$e_1 = E^+\left[E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right]\right]$ Lemma G.5

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right]$
$e_2 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right]$
$E^* \sim \tilde{E}^*$ Lemma G.4
$e_3 \sim \tilde{e}_3$ Lemma G.4
$\tilde{e}_3 = (\lambda f.\tilde{e}_{3a})\ \tilde{v}; e_{3a} = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$
$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma G.1
$e_1' = \tilde{E}^*[\tilde{e}_3]$
$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathtt{handle}_s\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]\right)$
$e_{3a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$
$e_{3a}[f := v] \sim \tilde{e}_{3a}[f := \tilde{v}]$ Lemma G.2
$\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \sim \tilde{e}_{3a}[f := \tilde{v}]$ Relation
$e_2 \sim \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Lemma G.1
$\mathrm{EQ}\left(e_2, \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]\right)$ Case 1 EQ
$e_2' = \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right], v_1\right)\right]\right]$
$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right], v_1\right)\right]\right]$
$E^* \sim \tilde{E}^*$ Lemma G.4
$e_3 \sim \tilde{e}_3$ Lemma G.4
$\tilde{e}_3 = (\lambda f.\tilde{e}_{3a})\ \tilde{v}; e_{3a} = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$
$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma G.1
$e_1' = \mathsf{if}\ \tilde{E}^*[\tilde{e}_3]\ \mathsf{then}\ \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\tilde{v}_1\ \mathsf{else}\ \mathtt{error}_{k_1}$ Case 2 EQ
$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1')$
$\Longrightarrow^* \mathtt{handle}_s\Big(\mathtt{h}_{\mathscr{C}},$
$\quad \mathsf{if}\ \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]\ \mathsf{then}\ \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\tilde{v}_1\ \mathsf{else}\ \mathtt{error}_{k_1}\Big)$

$e_{3a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$

$e_{3a}[f := v] \sim \tilde{e}_{3a}[f := \tilde{v}]$ Lemma G.2

$\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \sim \tilde{e}_{3a}[f := \tilde{v}]$ Relation

$E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Lemma G.1

$\mathrm{EQ}\left(e_2, \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\tilde{v}_1 \text{ else } \mathtt{error}_{k_1}\right)$ Case 2 EQ

$e_2' = \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$ Choose

▸ Case $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)$

Similarly to the functional case. Replacement variable is a fresh f, it does not affect anything.

$e_1 = E^+\left[E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right]\right]$ Lemma G.5

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right]$

$e_2 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right]$

$E^* \sim \tilde{E}^*$ Lemma G.4

$e_3 \sim \tilde{e}_3$ Lemma G.4

$\tilde{e}_3 = (\lambda f.\tilde{e}_{3a})\ \tilde{v}$

$e_{3a} = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right); \kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$

$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma G.1

$e_1' = \tilde{E}^*[\tilde{e}_3]$

$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathtt{handle}_s\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]\right)$

$e_{3a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$

$e_{3a}[f := v] \sim \tilde{e}_{3a}[f := \tilde{v}]$ Lemma G.2

$\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \sim \tilde{e}_{3a}[f := \tilde{v}]$ Relation

$e_2 \sim \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Lemma G.1

$\mathrm{EQ}\left(e_2, \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]\right)$ Case 1 EQ

$e_2' = \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right], v_1\right)\right]\right]$

$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right], v_1\right)\right]\right]$

$E^* \sim \tilde{E}^*$ Lemma G.4

$e_3 \sim \tilde{e}_3$ Lemma G.4

$\tilde{e}_3 = (\lambda f.\tilde{e}_{3a})\ \tilde{v}$

$e_{3a} = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right); \kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$

$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma G.1

$e'_1 = \text{if } \tilde{E}^*[\tilde{e}_3] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right) \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$ Case 2 EQ

$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e'_1)$

$\Longrightarrow^* \mathtt{handle}_s\Big(\mathtt{h}_{\mathscr{C}},$

$\quad \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right) \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}\Big)$

$e_{3a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)$

$e_{3a}[f := v] \sim \tilde{e}_{3a}[f := \tilde{v}]$ Lemma G.2

$\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \sim \tilde{e}_{3a}[f := \tilde{v}]$ Relation

$E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Lemma G.1

$\mathrm{EQ}\Big(e_2, \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right) \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}\Big)$ Case 2 EQ

$e'_2 = \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right) \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$ Choose

▸ Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\Big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\Big) = e_4$

$e_1 = E^+\left[E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2\right]\right]$ Lemma G.5

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2\right]$

$E^* \sim \tilde{E}^*$ Lemma G.4

$e_3 \sim \tilde{e}_3$ Lemma G.4

$\tilde{e}_3 = \lambda x.\tilde{e}_{3a}\ \tilde{v}_2; e_{3a} = \mathsf{mon}_j^{k,l}\Big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)$

$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma G.1

$e'_1 = \tilde{E}^*[\tilde{e}_3]$ Case 1 EQ

$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e'_1) \Longrightarrow^* \mathtt{handle}_s\Big(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]\Big)$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\Big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\Big)$

$e_{3a}[x := v_2] \sim \tilde{e}_{3a}[x := \tilde{v}_2]$ Lemma G.2

$e_2 = E^*\left[\mathsf{mon}_j^{k,l}\Big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\Big)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Lemma G.1

$\mathrm{EQ}\Big(e_2, \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]\Big)$ Case 1 EQ

$e'_2 = \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_3)\right]\right]$

$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\Big(E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2\right], v_3\Big)\right]\right]$

$E^* \sim \tilde{E}^*$ Lemma G.4

$e_3 \sim \tilde{e}_3$ Lemma G.4

$\tilde{e}_3 = \lambda x.\tilde{e}_{3a}\ \tilde{v}_2; e_{3a} = \mathsf{mon}_j^{k,l}\Big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)$

$E^*[\tilde{e}_3] \sim \tilde{E}^*[\tilde{e}_3]$ Lemma G.1

$e'_1 = \text{if } \tilde{E}^*[\tilde{e}_3] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right) \tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$ Case 2 EQ

$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e'_1)$

$\Longrightarrow^* \mathtt{handle}_s\big(\mathtt{h}_{\mathscr{C}},$

if $\tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ then $\left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_3$ else $\mathtt{error}_{k_1}\big)$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)$

$e_{3a}[x := v_2] \sim \tilde{e}_{3a}[x := \tilde{v}_2]$ Lemma G.2

$E^*\left[\mathsf{mon}_j^{k,l}\big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Lemma G.1

$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{mon}_j^{k,l}\big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)\right], v_3\right)\right]\right]$

$\mathrm{EQ}\big(e_2, \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_3 \text{ else } \mathtt{error}_{k_1}\big)$ Case 2 EQ

$e_2' = \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$ Choose

▸ Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big) = e_4$ where $\kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$

Similar to the functional case. The only problem is $\kappa_4 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$ where x is the new argument of the function created for relation. We need to reason $\kappa_4[x := v_2] = \kappa_3$, which is obvious.

$e_1 = E^+\left[E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2\right]\right]$ Lemma G.5

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2\right]$

$E^* \sim \tilde{E}^*$ Lemma G.4

$e_3 \sim \tilde{e}_3$ Lemma G.4

$\tilde{e}_3 = \lambda x.\tilde{e}_{3a}\ \tilde{v}_2$

$e_{3a} = \mathsf{mon}_j^{k,l}\big(\kappa_4, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big); \kappa_4 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$

$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma G.1

$e_1' = \tilde{E}^*[\tilde{e}_3]$ Case 1 EQ

$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathtt{handle}_s\big(\mathtt{h}_{\mathscr{C}}, \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]\big)$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\big(\kappa_4[x := v_2], v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)$

$\kappa_4[x := v_2]$

$= \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right][x := v_2] = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$ $x \notin \mathrm{fv}(\kappa_2, \kappa_1)$

$= \kappa_3$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)$

$e_{3a}[x := v_2] \sim \tilde{e}_{3a}[x := \tilde{v}_2]$ Lemma G.2

$e_2 = E^*\left[\mathsf{mon}_j^{k,l}\big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Lemma G.1

$\mathrm{EQ}\big(e_2, \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]\big)$ Case 1 EQ

$e_2' = \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_3)\right]\right]$

$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2\right], v_3\right)\right]\right]$

$E^* \sim \tilde{E}^*$ Lemma G.4

$e_3 \sim \tilde{e}_3$ Lemma G.4

$\tilde{e}_3 = \lambda x.\tilde{e}_{3a}\ \tilde{v}_2$

$e_{3a} = \mathsf{mon}_j^{k,l}\left(\kappa_4, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right); \kappa_4 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$

$E^*[\tilde{e}_3] \sim \tilde{E}^*[\tilde{e}_3]$ Lemma G.1

$e_1' = \text{if } \tilde{E}^*[\tilde{e}_3] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$ Case 2 EQ

$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1')$

$\Longrightarrow^* \mathtt{handle}_s\Big(\mathtt{h}_{\mathscr{C}},$

$\quad \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_3 \text{ else } \mathtt{error}_{k_1}\Big)$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\left(\kappa_4[x := v_2], v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$

$\kappa_4[x := v_2]$

$\quad = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right][x := v_2] = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$ $x \notin \mathrm{fv}(\kappa_2, \kappa_1)$

$\quad = \kappa_3$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$

$e_{3a}[x := v_2] \sim \tilde{e}_{3a}[x := \tilde{v}_2]$ Lemma G.2

$E^*\left[\mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Lemma G.1

$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)\right], v_3\right)\right]\right]$

$\mathrm{EQ}\left(e_2, \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_3 \text{ else } \mathtt{error}_{k_1}\right)$ Case 2 EQ

$e_2' = \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$ Choose

▸ Other cases follow similarly.

□

CHAPTER H

Contracts to Effects Compiler Correctness (Local Handler)

In this part, we provide the proof for a compiler version using local handler. Instead of a global handler outside, a flat contract is compiled into an effect and handle it on the spot. All other compilation rules stay the same. Semantics of handlers can be either Deep or Shallow. However, because Deep Handlers push a new (unused) handler each time a check is performed, Shallow Handlers are chosen for this compiler implementation.

$$\frac{\Gamma \vdash e_1 : \tau \to \mathsf{bool} \rightarrowtail^l e_3 \qquad \Gamma \vdash e_2 : \tau \rightarrowtail^l e_4}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) : \tau \rightarrowtail^l \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, (\mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_3, e_4\rangle\rangle))}\ \text{Compile-Flat}$$

$$\begin{aligned} \mathtt{h}_{\mathscr{C}} : \mathtt{lab} \quad &= \quad \{\mathscr{C} \to \forall\alpha.\langle \mathtt{blab}, \langle \alpha \to \mathsf{bool}, \alpha\rangle\rangle \to \alpha\} \\ &= \quad \{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.\ \text{if } e\ v \text{ then } k\ v \text{ else } \mathtt{error}_l\} \\ &\quad \text{where } \lambda\langle l, \langle e, v\rangle\rangle \text{ deconstructs the argument as tuple pattern} \end{aligned}$$

H.1 Relations

Before we define relations that will be used in the simulation proof below. We define:

- Context E^* is any context without $\mathsf{check}_j^k(e, v)$ in Contracts.
- Context $E^+ = \square \mid E^+\left[E^*\left[\mathsf{check}_j^k(\square, v)\right]\right]$

$R_v \subseteq \mathsf{Contracts}\ v \times \mathtt{Effects}\ v$

Abuse of notation: If $(v, v') \in R_v$, we write $v \sim_{R_v} v'$ or $v \sim v'$. We write $\tilde{v}$ for any value $\tilde{v}$ satisfying $v \sim \tilde{v}$.

$$
\begin{array}{rcl}
x & \sim & x \\
b & \sim & b \\
N & \sim & N \\
\lambda x : \tau . e & \sim & \lambda x : \tau^* . \tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x : \tau . \mathsf{mon}_j^{k,l}\left(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x : \tau . \mathsf{mon}_j^{k,l}\left(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
 & & \kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]
\end{array}
$$

$R_e \subseteq \mathsf{Contracts}\ e \times \mathtt{Effects}\ e$

Abuse of notation: If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We write $\tilde{e}$ for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

$$
\begin{array}{rcl}
v & \sim & \tilde{v} \\
e_1\ e_2 & \sim & \tilde{e}_1\ \tilde{e}_2 \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) & \sim & \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2 \rangle \rangle) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\left(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \\
 & & \kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right] \\
\mathsf{blame}_j^k & \sim & \mathtt{error}_k
\end{array}
$$

$R_{E^*} \subseteq \mathsf{Contracts}\ E^* \times \mathtt{Effects}\ E^*$

Abuse of notation: If $(E^*, E'^*) \in R_{E^*}$, we write $E^* \sim_{R_{E^*}} E'^*$ or $E^* \sim E'^*$. We write $\tilde{E}^*$ for any expression $\tilde{E}^*$ satisfying $E^* \sim \tilde{E}^*$.

$$
\begin{array}{rcl}
\square & \sim & \square \\
E^*\ e & \sim & \tilde{E}^*\ \tilde{e} \\
v\ E^* & \sim & \tilde{v}\ \tilde{E}^*
\end{array}
$$

$$
\begin{array}{rcl}
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), E^*) & \sim & \mathtt{handle}_s\big(\mathtt{h}_{\mathscr{C}}, \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{E}^* \rangle\rangle\big) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E^*) & \sim & \tilde{e}_1\ \tilde{E}^* \\
 & & e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, E^*) & \sim & \tilde{e}_1\ \tilde{E}^* \\
 & & e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
 & & \kappa_3 = \kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\Big]
\end{array}
$$

$\boxed{R_{E^+} \subseteq \mathsf{Contracts}\ E^+ \times \mathtt{Effects}\ E^+}$

Abuse of notation: If $(E^+, E'^+) \in R_{E^+}$, we write $E^+ \sim_{R_{E^+}} E'^+$ or $E^+ \sim E'^+$. We write $\tilde{E}^+$ for any expression $\tilde{E}^+$ satisfying $E^+ \sim \tilde{E}^+$.

$$
\begin{array}{rcl}
\Box & \sim & \Box \\
E^+\big[E^*\big[\mathsf{check}_j^k(\Box, v)\big]\big] & \sim & \text{if } \Box \text{ then } \big(\lambda x.\tilde{E}^+\big[\tilde{E}^*[x]\big]\big)\ \tilde{v} \text{ else } \mathtt{error}_k
\end{array}
$$

Lemma H.1. (Relation is closed)

If $E^* \sim \tilde{E}^*$, $e \sim \tilde{e}$ and e has no check in evaluation position, then $E^*[e] \sim \tilde{E}^*[\tilde{e}]$.

Proof. By induction on $E^* \sim \tilde{E}^*$. Follows similarly to Lemma F.1 because the relations (R_e, R_E) are the same. □

Lemma H.2. (Relation is closed under substitution)

If $e \sim \tilde{e}$ and $v \sim \tilde{v}$ then $e[x := v] \sim \tilde{e}[x := \tilde{v}]$.

Proof. By induction on $e \sim e'$. Follows Lemma F.3 because relations (R_v, R_e) are the same.

For this compiler, there is a single case that has their relation different from Deep/Shallow compilers.

- Case $\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) \sim \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2 \rangle\rangle)$

  $e_1[x := v] \sim \tilde{e}_1[x := v']$ I.H.
  $e_2[x := v] \sim \tilde{e}_2[x := v']$ I.H.
  $\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2)[x := v] \sim \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2 \rangle\rangle)[x := v']$ Def.

- Other cases follow Lemma F.3 because their relations (R_e) are the same.

□

Lemma H.3. (Relation decomposition through check-free contexts)

If $E^*[e] \sim e'$, then there exist E'^* and e'_0 such that $E^* \sim E'^*$ and $e \sim e'_0$ and $e' = E'^*[e'_0]$.

Proof.

By induction on the check-free context E^*. Follows Lemma F.2 because relations (R_E, R_e) are the same. □

Lemma H.4. (Breaking down Context E)

For all contexts E, uniquely decompose $E = E^+[E^*]$.

Uniqueness observation: Recall that E^* has no check contexts.

- If E does not have check, then it is $E^* = E$, and $E^+ = \square$.
- Else there is only a single decomposition to get out E^* without check contexts

Proof. By induction on the structure of E. Follows Lemma F.4 because the contexts E, E^+, E^* are the same. □

Lemma H.5. (Compiler is subset of Relation)

If $\Gamma \vdash e : \tau \rightarrowtail^l e'$ then $e \sim e'$

Proof.

By induction on structure of $\Gamma \vdash e_1 : \tau \rightarrowtail^l e_2$.

- Case

$$\frac{\Gamma \vdash e_3 : \tau \to \mathsf{bool} \rightarrowtail^l e_5 \qquad \Gamma \vdash e_4 : \tau \rightarrowtail^l e_6}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), e_4) : \tau \rightarrowtail^l \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_5, e_6\rangle\rangle)}$$

$e_2^* = \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{perform}\ \mathscr{C}\ \langle k, \langle e_5^*, e_6^*\rangle\rangle)$

$e_3 \sim e_5^*$ I.H.

$e_4 \sim e_6^*$ I.H.

$e_1 \sim e_2^*$

- Case

$$\frac{\Gamma \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail^l e_4 \qquad \Gamma \vdash \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^l e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail^l e_5\ e_4}$$

$e_2^* = e_5^*\ e_4^*$

$e_3 \sim e_4^*$ I.H.

$\lambda f.\lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right) \sim e_5^*$ I.H.

$e_1 \sim e_2^*$

- Case

$$\frac{\Gamma \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail^l e_4 \qquad \Gamma \vdash \lambda f.\lambda x.\big(\lambda y.\mathsf{mon}_j^{k,l}\big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big)\big)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^l e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail^l e_5\ e_4}$$

$e_2^* = e_5^*\ e_4^*$

$e_3 \sim e_4^*$ I.H.

$\lambda f.\lambda x.\big(\lambda y.\mathsf{mon}_j^{k,l}\big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big)\big)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) \sim e_5^*$ I.H.

$e_1 \sim e_2^*$

- Other cases follow similarly, by I.H.

□

H.2 Correctness Theorem

$$\mathrm{EQ}(e, e') = \begin{cases} \text{true if } e \sim e' \text{ and } e \neq E^*\left[\mathsf{check}_{j_1}^{k_1}(e_1, v_1)\right] \\ \text{true if } e = E^+\left[E^*\left[\mathsf{check}_{j_1}^{k_1}(e_1, v_1)\right]\right] \text{ and} \\ \qquad e' = \text{if } \tilde{e}_1 \text{ then } \left(\lambda x.\tilde{E}^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_1 \text{ else } \mathtt{error}_{k_1} \\ \text{true if } e = E\left[\mathsf{blame}_j^k\right] \text{ and } e' = \mathtt{error}_k \\ \text{false otherwise} \end{cases}$$

Theorem H.6.

If $e \twoheadrightarrow^* b$ and $\cdot \vdash e : \tau \rightarrowtail^l e'$ then $e' \Longrightarrow^* b$

Runs without a wrapper because the local handler resolves the contract effect in-place.

Proof.

$e \sim e'$ Lemma H.5

$\mathrm{EQ}(e, e')$ Case 1 EQ

$e' \Longrightarrow^* \tilde{b} = b$ Lemma H.7

□

Lemma H.7.

If $\mathrm{EQ}(e, e')$ and $e \twoheadrightarrow^* r$ then $e' \Longrightarrow^* \tilde{r}$.

Where $r = v \mid \mathsf{blame}_j^k$ and $\tilde{r} = \tilde{v} \mid \mathtt{error}_k$

Proof.

By induction on the structure of $e \twoheadrightarrow^* r$

- Case $v \twoheadrightarrow^* v$

$\mathrm{EQ}(e, e')$	Given
$e \sim e'$	Case 1 EQ, otherwise invalid
$e' = \tilde{v}$	Inversion
$e' \Longrightarrow^* \tilde{v}$	Reflexivity

- Case $\mathsf{blame}_j^k \twoheadrightarrow^* \mathsf{blame}_j^k$

$\mathrm{EQ}(e, e')$	Given
$e' = \mathtt{error}_k$	Case 3 EQ
$e' \Longrightarrow^* \mathtt{error}_k$	Reflexivity

- Case $e \twoheadrightarrow e_1$ and $e_1 \twoheadrightarrow^* r$

$e' \Longrightarrow^* e_1'$	Lemma H.8
$\mathrm{EQ}(e_1, e_1')$	Lemma H.8
$e_1' \Longrightarrow^* \tilde{r}$	I.H.
$e' \Longrightarrow^* \tilde{r}$	Step

□

Lemma H.8.

If $\mathrm{EQ}(e_1, e_1')$ and $e_1 \twoheadrightarrow e_2$ then exists e_2' that $e_1' \Longrightarrow^* e_2'$ and $\mathrm{EQ}(e_2, e_2')$

Proof.

By case analysis on $e_1 \twoheadrightarrow e_2$.

- $E\left[\mathsf{blame}_j^k\right] \twoheadrightarrow \mathsf{blame}_j^k$, and $E \neq \square$

$e_1' = \mathtt{error}_k$	Case 3 EQ
$e_1' \Longrightarrow^* \mathtt{error}_k$	Step and Reflexivity
$e_2' = \mathtt{error}_k$	Choose

- $E[e_3] \twoheadrightarrow E[e_4]$ where $e_3 \dashrightarrow e_4$ is a redex

  Proof Sketch: $E = E^+\left[E^*\left[\mathsf{check}_j^k(\square, v)\right]\right]$, then casing on E^+ structure. At the same time, EQ predicate will allow one case analysis ($e_1? = E'\left[\mathsf{check}_{j'}^{k'}(e', v')\right]$). If $E^+ = \square$, use Lemma H.3 to find equivalent $\tilde{E}^*$, else, by EQ case 2 identify the shape of e_1' by E^+ relation.

  - Case $e_3 = \mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}_j^k(e\ v, v) = e_4$

    Actually need $e \neq E'\left[\mathsf{check}_{j'}^{k'}(e', v')\right]$ to obtain $\tilde{e}$. But the proof goes through without stating this fact because we inverse from EQ predicate. When running, this e is never a check, because we don't evaluate inside flat contract.

$e_1 = E\left[\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v)\right] = E^+\left[E^*\left[\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v)\right]\right]$ Lemma H.4

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v)\right]$

$E^* \sim \tilde{E}^*$ Lemma H.3

$e_3 \sim \tilde{e}_3$ Lemma H.3

$\tilde{e}_3 = \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle)$ Inversion

$e_1' = \tilde{E}^*[\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle)]$ Case 1 EQ

$e_1' \Longrightarrow^*$ if $\tilde{e}\ \tilde{v}$ then $\left(\lambda x.\tilde{E}^*[x]\right)\ \tilde{v}$ else $\mathtt{error}_k$

$e_2 = E^*\left[\mathsf{check}_j^k(e\ v, v)\right]$

$e\ v \sim \tilde{e}\ \tilde{v}$

$\mathrm{EQ}\left(e_2, \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}^*[x]\right)\ \tilde{v} \text{ else } \mathtt{error}_k\right)$ Case 2 EQ

$e_2' = \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}^*[x]\right)\ \tilde{v} \text{ else } \mathtt{error}_k$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

$e_2 =$
$E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{check}_j^k(e\ v, v)\right], v_1\right)\right]\right] = E_2^+\left[E^*\left[\mathsf{check}_j^k(e\ v, v)\right]\right]$

$\tilde{E}_2^+ = \text{if } \square \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$

$E_2^+\left[E^*\left[\mathsf{check}_j^k(\square, v)\right]\right] \sim \text{if } \square \text{ then } \left(\lambda x.\tilde{E}_2^+\left[\tilde{E}^*[x]\right]\right)\ \tilde{v} \text{ else } \mathtt{error}_k$

$e_1' = \text{if } \tilde{E}^*[\tilde{e}_3] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$ Case 2 EQ

$\tilde{e}_3 = \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle)$ Inversion

$e_1' \Longrightarrow^*$ if $\tilde{e}\ \tilde{v}$ then $\left(\lambda x.\ \text{if } \tilde{E}^*[x] \text{ then } \left(\lambda y.\tilde{E}_1^+\left[\tilde{E}_1^*[y]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}\right)\ \tilde{v}$ else $\mathtt{error}_k$
$= \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}_2^+\left[\tilde{E}^*[x]\right]\right)\ \tilde{v} \text{ else } \mathtt{error}_k$

$\mathrm{EQ}\left(e_2, \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}_2^+\left[\tilde{E}^*[x]\right]\right)\ \tilde{v} \text{ else } \mathtt{error}_k\right)$ Case 2 EQ

$e_2' = \text{if } \tilde{e}\ \tilde{v} \text{ then } \left(\lambda x.\tilde{E}_2^+\left[\tilde{E}^*[x]\right]\right)\ \tilde{v} \text{ else } \mathtt{error}_k$ Choose

▸ Case $e_3 = \mathsf{check}_j^k(\mathsf{true}, v) \dashrightarrow v = e_4$

Due to the restriction of EQ, only EQ case 2 matches this case.

$e_1 = E\left[\mathsf{check}_j^k(\mathsf{true}, v)\right] = E^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{true}, v)\right]\right]$ Given and Lemma H.4

$e_2 = E^+[E^*[v]]$

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{check}_j^k(\mathsf{true}, v)\right]$

$e_2 = E^*[v]$

$e_1' = \text{if true then } \left(\lambda x.\tilde{E}^*[x]\right)\ \tilde{v} \text{ else } \mathtt{error}_k$ EQ case 2

$e_1' \Longrightarrow^* \tilde{E}^*[\tilde{v}]$

$E^*[v] \sim \tilde{E}^*[\tilde{v}]$ Lemma H.1

$\mathrm{EQ}\left(e_2, \tilde{E}^*[\tilde{v}]\right)$ EQ case 1

$e_2' = \tilde{E}^*[\tilde{v}]$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(E^*[v], v_1)\right]\right]$
$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{check}_j^k(\mathsf{true}, v)\right], v_1\right)\right]\right] =$
$E_2^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{true}, v)\right]\right]$
$e_1' = \mathsf{if}\ \mathsf{true}\ \mathsf{then}\ \left(\lambda y.\tilde{E}_2^+\left[\tilde{E}^*[y]\right]\right)\tilde{v}\ \mathsf{else}\ \mathtt{error}_k$ Case 2 EQ
$E_2^+ \sim \mathsf{if}\ \square\ \mathsf{then}\ \left(\lambda x.\tilde{E}_1^+[E_1^*[x]]\right)\tilde{v}_1\ \mathsf{else}\ \mathtt{error}_{k_1} = \tilde{E}_2^+$
$e_1' \Longrightarrow^* \tilde{E}_2^+\left[\tilde{E}^*[\tilde{v}]\right]$
$E^*[v] \sim \tilde{E}^*[\tilde{v}]$ Lemma H.1
$\mathrm{EQ}\left(e_2, \mathsf{if}\ \tilde{E}^*[\tilde{v}]\ \mathsf{then}\ \left(\lambda x.\tilde{E}_1^+[E_1^*[x]]\right)\tilde{v}_1\ \mathsf{else}\ \mathtt{error}_{k_1}\right)$ Case 2 EQ
$\mathrm{EQ}\left(e_2, \tilde{E}_2^+\left[\tilde{E}^*[\tilde{v}]\right]\right)$ Replace with $\tilde{E}_2^+$
$e_2' = \tilde{E}_2^+\left[\tilde{E}^*[\tilde{v}]\right]$ Choose

▸ Case $e_3 = \mathsf{check}_j^k(\mathsf{false}, v) \dashrightarrow \mathsf{blame}_j^k = e_4$

$e_1 = E\left[\mathsf{check}_j^k(\mathsf{false}, v)\right] = E^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]\right]$ Given and Lemma H.4
$e_2 = E^+\left[E^*\left[\mathsf{blame}_j^k\right]\right]$

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]$
$e_2 = E^*\left[\mathsf{blame}_j^k\right]$
$e_1' = \mathsf{if}\ \mathsf{false}\ \mathsf{then}\ \left(\lambda x.\tilde{E}^*[x]\right)\tilde{v}\ \mathsf{else}\ \mathtt{error}_k$ EQ case 2
$e_1' \Longrightarrow^* \mathtt{error}_k$
$\mathrm{EQ}(e_2, \mathtt{error}_k)$ Case 3 EQ
$e_2' = \mathtt{error}_k$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{blame}_j^k\right], v_1\right)\right]\right]$
$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right], v_1\right)\right]\right] =$
$E_2^+\left[E^*\left[\mathsf{check}_j^k(\mathsf{false}, v)\right]\right]$
$e_1' = \mathsf{if}\ \mathsf{false}\ \mathsf{then}\ \left(\lambda y.\tilde{E}_2^+\left[\tilde{E}^*[y]\right]\right)\tilde{v}\ \mathsf{else}\ \mathtt{error}_k$ Case 2 EQ
$e_1' \Longrightarrow^* \mathtt{error}_k$
$\mathrm{EQ}(e_2, \mathtt{error}_k)$ Case 3 EQ
$e_2' = \mathtt{error}_k$ Choose

▸ Case $\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)$

$e_1 = E^+\left[E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right]\right]$ Lemma H.4

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right]$
$e_2 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right]$
$E^* \sim \tilde{E}^*$ Lemma H.3
$e_3 \sim \tilde{e}_3$ Lemma H.3
$\tilde{e}_3 = (\lambda f.\tilde{e}_{3a})\ \tilde{v}; e_{3a} = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$
$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma H.1
$e_1' = \tilde{E}^*[\tilde{e}_3]$
$e_1' \Longrightarrow^* \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$
$e_{3a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$
$e_{3a}[f := v] \sim \tilde{e}_{3a}[f := \tilde{v}]$ Lemma H.2
$\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \sim \tilde{e}_{3a}[f := \tilde{v}]$ Relation
$e_2 \sim \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Lemma H.1
$\mathrm{EQ}\left(e_2, \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]\right)$ Case 1 EQ
$e_2' = \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$

$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right], v_1\right)\right]\right]$
$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right], v_1\right)\right]\right]$
$E^* \sim \tilde{E}^*$ Lemma H.3
$e_3 \sim \tilde{e}_3$ Lemma H.3
$\tilde{e}_3 = (\lambda f.\tilde{e}_{3a})\ \tilde{v}; e_{3a} = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$
$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma H.1
$e_1' = \text{if } \tilde{E}^*[\tilde{e}_3] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$ Case 2 EQ
$e_1' \Longrightarrow^* \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$
$e_{3a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$
$e_{3a}[f := v] \sim \tilde{e}_{3a}[f := \tilde{v}]$ Lemma H.2
$\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) \sim \tilde{e}_{3a}[f := \tilde{v}]$ Relation
$E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Lemma H.1
$\mathrm{EQ}\left(e_2, \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}\right)$ Case 2 EQ
$e_2' = \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$ Choose

▸ Case $\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)$

Similarly to the functional case. Replacement variable is a fresh f, it does not affect anything.

$e_1 = E^+\left[E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right]\right]$ Lemma H.4

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right]$

$e_2 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right]$
$E^* \sim \tilde{E}^*$ Lemma H.3
$e_3 \sim \tilde{e}_3$ Lemma H.3
$\tilde{e}_3 = (\lambda f.\tilde{e}_{3a})\ \tilde{v}$
$e_{3a} = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right); \kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$
$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma H.1
$e_1' = \tilde{E}^*[\tilde{e}_3]$
$e_1' \Longrightarrow^* \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$
$e_{3a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$
$e_{3a}[f := v] \sim \tilde{e}_{3a}[f := \tilde{v}]$ Lemma H.2
$\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \sim \tilde{e}_{3a}[f := \tilde{v}]$ Relation
$e_2 \sim \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Lemma H.1
$\mathrm{EQ}(e_2, \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]])$ Case 1 EQ
$e_2' = \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_1)\right]\right]$
$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right], v_1\right)\right]\right]$
$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right], v_1\right)\right]\right]$
$E^* \sim \tilde{E}^*$ Lemma H.3
$e_3 \sim \tilde{e}_3$ Lemma H.3
$\tilde{e}_3 = (\lambda f.\tilde{e}_{3a})\ \tilde{v}$
$e_{3a} = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right); \kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$
$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma H.1
$e_1' = \text{if } \tilde{E}^*[\tilde{e}_3] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$ Case 2 EQ
$e_1' \Longrightarrow^* \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$
$e_{3a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\left(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$
$e_{3a}[f := v] \sim \tilde{e}_{3a}[f := \tilde{v}]$ Lemma H.2
$\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v) \sim \tilde{e}_{3a}[f := \tilde{v}]$ Relation
$E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]]$ Lemma H.1
$\mathrm{EQ}\left(e_2, \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}\right)$ Case 2 EQ
$e_2' = \text{if } \tilde{E}^*[\tilde{e}_{3a}[f := \tilde{v}]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}_1^*[x]\right]\right)\ \tilde{v}_1 \text{ else } \mathtt{error}_{k_1}$ Choose

▸ Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) = e_4$
$e_1 = E^+\left[E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2\right]\right]$ Lemma H.4

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2\right]$

$E^* \sim \tilde{E}^*$ Lemma H.3

$e_3 \sim \tilde{e}_3$ Lemma H.3

$\tilde{e}_3 = \lambda x.\tilde{e}_{3a}\ \tilde{v}_2; e_{3a} = \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$

$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma H.1

$e_1' = \tilde{E}^*[\tilde{e}_3]$ Case 1 EQ

$e_1' \Longrightarrow^* \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$

$e_{3a}[x := v_2] \sim \tilde{e}_{3a}[x := \tilde{v}_2]$ Lemma H.2

$e_2 = E^*\left[\mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Lemma H.1

$\mathrm{EQ}\left(e_2, \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]\right)$ Case 1 EQ

$e_2' = \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Choose

– Case $E^+ = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}(\square, v_3)\right]\right]$

$e_1 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2\right], v_3\right)\right]\right]$

$E^* \sim \tilde{E}^*$ Lemma H.3

$e_3 \sim \tilde{e}_3$ Lemma H.3

$\tilde{e}_3 = \lambda x.\tilde{e}_{3a}\ \tilde{v}_2; e_{3a} = \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\right)$

$E^*[\tilde{e}_3] \sim \tilde{E}^*[\tilde{e}_3]$ Lemma H.1

$e_1' = \text{if } \tilde{E}^*[\tilde{e}_3] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\ \tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$ Case 2 EQ

$e_1' \Longrightarrow^* \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\ \tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)$

$e_{3a}[x := v_2] \sim \tilde{e}_{3a}[x := \tilde{v}_2]$ Lemma H.2

$E^*\left[\mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)\right] \sim \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Lemma H.1

$e_2 = E_1^+\left[E_1^*\left[\mathsf{check}_{j_1}^{k_1}\left(E^*\left[\mathsf{mon}_j^{k,l}\left(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right)\right], v_3\right)\right]\right]$

$\mathrm{EQ}\left(e_2, \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\ \tilde{v}_3 \text{ else } \mathtt{error}_{k_1}\right)$ Case 2 EQ

$e_2' = \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \left(\lambda x.\tilde{E}_1^+\left[\tilde{E}^*[x]\right]\right)\ \tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$ Choose

▸ Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\left(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\right) = e_4$ where $\kappa_3 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\right]$

Similar to the functional case. The only problem is $\kappa_4 = \kappa_2\left[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\right]$ where x is the new argument of the function created for relation. We need to reason $\kappa_4[x := v_2] = \kappa_3$, which is obvious.

$e_1 = E^+\left[E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2\right]\right]$ Lemma H.4

– Case $E^+ = \square$

$e_1 = E^*\left[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2\right]$

$E^* \sim \tilde{E}^*$ Lemma H.3

$e_3 \sim \tilde{e}_3$ Lemma H.3

$\tilde{e}_3 = \lambda x.\tilde{e}_{3a}\ \tilde{v}_2$

$e_{3a} = \mathsf{mon}_j^{k,l}\big(\kappa_4, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big); \kappa_4 = \kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\big]$

$e_1 \sim \tilde{E}^*[\tilde{e}_3]$ Lemma H.1

$e_1' = \tilde{E}^*[\tilde{e}_3]$ Case 1 EQ

$e_1' \Longrightarrow^* \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\big(\kappa_4[x := v_2], v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)$

$\kappa_4[x := v_2]$

$= \kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\big][x := v_2] = \kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\big]$ $x \notin \mathrm{fv}(\kappa_2, \kappa_1)$

$= \kappa_3$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)$

$e_{3a}[x := v_2] \sim \tilde{e}_{3a}[x := \tilde{v}_2]$ Lemma H.2

$e_2 = E^*\big[\mathsf{mon}_j^{k,l}\big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)\big] \sim \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Lemma H.1

$\mathrm{EQ}\big(e_2, \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]\big)$ Case 1 EQ

$e_2' = \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Choose

– Case $E^+ = E_1^+\big[E_1^*\big[\mathsf{check}_{j_1}^{k_1}(\square, v_3)\big]\big]$

$e_1 = E_1^+\big[E_1^*\big[\mathsf{check}_{j_1}^{k_1}\big(E^*\big[\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2\big], v_3\big)\big]\big]$

$E^* \sim \tilde{E}^*$ Lemma H.3

$e_3 \sim \tilde{e}_3$ Lemma H.3

$\tilde{e}_3 = \lambda x.\tilde{e}_{3a}\ \tilde{v}_2$

$e_{3a} = \mathsf{mon}_j^{k,l}\big(\kappa_4, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big); \kappa_4 = \kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\big]$

$E^*[\tilde{e}_3] \sim \tilde{E}^*[\tilde{e}_3]$ Lemma H.1

$e_1' = \text{if } \tilde{E}^*[\tilde{e}_3] \text{ then } \big(\lambda x.\tilde{E}_1^+\big[\tilde{E}^*[x]\big]\big)\ \tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$ Case 2 EQ

$e_1' \Longrightarrow^* \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \big(\lambda x.\tilde{E}_1^+\big[\tilde{E}^*[x]\big]\big)\ \tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\big(\kappa_4[x := v_2], v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)$

$\kappa_4[x := v_2]$

$= \kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\big][x := v_2] = \kappa_2\big[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\big]$ $x \notin \mathrm{fv}(\kappa_2, \kappa_1)$

$= \kappa_3$

$e_{3a}[x := v_2] = \mathsf{mon}_j^{k,l}\big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)$

$e_{3a}[x := v_2] \sim \tilde{e}_{3a}[x := \tilde{v}_2]$ Lemma H.2

$E^*\big[\mathsf{mon}_j^{k,l}\big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)\big] \sim \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]]$ Lemma H.1

$e_2 = E_1^+\big[E_1^*\big[\mathsf{check}_{j_1}^{k_1}\big(E^*\big[\mathsf{mon}_j^{k,l}\big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\big)\big], v_3\big)\big]\big]$

$\mathrm{EQ}\big(e_2, \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \big(\lambda x.\tilde{E}_1^+\big[\tilde{E}^*[x]\big]\big)\ \tilde{v}_3 \text{ else } \mathtt{error}_{k_1}\big)$ Case 2 EQ

$e_2' = \text{if } \tilde{E}^*[\tilde{e}_{3a}[x := \tilde{v}_2]] \text{ then } \big(\lambda x.\tilde{E}_1^+\big[\tilde{E}^*[x]\big]\big)\ \tilde{v}_3 \text{ else } \mathtt{error}_{k_1}$ Choose

- Other cases follow similarly.

□